[TRƯỜNG ĐẠI HỌC / HỌC VIỆN ĐÀO TẠO — PLACEHOLDER]

[KHOA CÔNG NGHỆ THÔNG TIN — BỘ MÔN HỆ THỐNG THÔNG TIN]

BÁO CÁO BÀI TẬP LỚN HỌC PHẦN CƠ SỞ DỮ LIỆU

ĐỀ TÀI:
HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM (CINEVIET CINEMA CHAIN MANAGEMENT SYSTEM)

Công nghệ: Java 21 LTS • Java Swing (FlatLaf) • MySQL Community Server 8.4 LTS (14 bảng 3NF) • JDBC • PDFBox 3

Thông tin Học phần	Nội dung Đăng ký (Placeholder)
Giảng viên hướng dẫn	[Họ và tên Giảng viên hướng dẫn — Placeholder]
Lớp học phần / Nhóm	[Mã lớp học phần] — [Nhóm XX — Placeholder]
Sinh viên thực hiện 1	[Họ và tên Sinh viên 1] — MSSV: [Mã SV 01]
Sinh viên thực hiện 2	[Họ và tên Sinh viên 2] — MSSV: [Mã SV 02]
Sinh viên thực hiện 3	[Họ và tên Sinh viên 3] — MSSV: [Mã SV 03]
Hệ quản trị CSDL thực thi	**MySQL Community Server 8.4.0 LTS** (`InnoDB`, `utf8mb4_unicode_ci`, 14 bảng 3NF)

Hà Nội, Năm học 2026

LỜI MỞ ĐẦU & MA TRẬN ĐÁP ỨNG YÊU CẦU HỌC PHẦN (R1 – R8)

Báo cáo này trình bày trọn vẹn quá trình phân tích, thiết kế cơ sở dữ liệu **14 bảng đạt chuẩn 3NF nghiêm ngặt**, cài đặt ràng buộc toàn vẹn & giao dịch đồng thời trên **MySQL Community Server 8.4.0 LTS**, phát triển ứng dụng Desktop **Java 21 LTS + Java Swing (FlatLaf)** kết nối qua **MySQL Connector/J** và kiểm chứng thực nghiệm cho **Hệ thống Quản lý Chuỗi Rạp Chiếu Phim CineViet**. Toàn bộ sơ đồ ERD, lược đồ quan hệ, chứng minh 3NF & kết nối không mất thông tin, câu lệnh SQL cài đặt, câu lệnh SQL chức năng R5–R7, biểu mẫu giao diện và báo cáo PDF đều được đối chiếu theo thang điểm chính thức (10,0 điểm):

Mã YC	Nội dung Yêu cầu Đánh giá Chính thức	Điểm	Chương / Mục trong Báo cáo	Trạng thái Kiểm chứng
R1	Mô tả yêu cầu nghiệp vụ và dữ liệu cần lưu	1,0	Chương 1	ĐẠT (Đã kiểm chứng trên MySQL 8.4)
R2	Vẽ và mô tả mô hình thực thể – liên kết (ERD Chen & Crow's Foot)	2,5	Chương 2 (Hình 2.1 & 2.2)	ĐẠT (Đồng bộ 14 bảng chuẩn 3NF)
R3	Mô hình quan hệ và mô tả bảng tương thích với ERD (14 bảng, 17 FK)	2,0	Chương 3 & Phụ lục A1, A2, B	ĐẠT (Đã kiểm chứng trên MySQL 8.4)
R4	Kiểm tra các bảng và đưa về chuẩn 3NF thực sự (xử lý `ticket`)	1,0	Chương 4	ĐẠT (Lossless-Join & 3NF/BCNF)
R5	Hai form nhập liệu, demo kiểm tra dữ liệu đúng / sai	1,0	Chương 5 (Mục 5.2, 5.3 & 5.7)	ĐẠT (40/40 test + GUI EDT + 2 JVM)
R6	Một màn hình tìm kiếm và hiển thị kết quả (phân trang 50 dòng/trang)	0,5	Chương 5 (Mục 5.4 & 5.7)	ĐẠT (Không cắt ngầm `LIMIT 1000`)
R7	Hai báo cáo xem trên màn hình và xuất PDF; nêu rõ người lập & người nhận	1,0	Chương 5 (Mục 5.5, 5.6 & 5.7)	ĐẠT (Sai lệch SQL độc lập = 0,00 VNĐ)
R8	Báo cáo bao phủ đủ R1–R7, kèm SQL cài đặt và SQL chức năng R5–R7	1,0	Chương 5.7 & Phụ lục A1–D2	ĐẠT (Đầy đủ DOCX, PDF & log SQL)

CHƯƠNG 1. PHÂN TÍCH YÊU CẦU NGHIỆP VỤ VÀ DỮ LIỆU CẦN LƯU TRỮ (TIÊU CHÍ R1)

(Tài liệu đáp ứng Tiêu chí R1 — 1.0 điểm)

1. Bối cảnh và Bài toán Quản lý Chuỗi Rạp Chiếu Phim

Doanh nghiệp **CineViet Cinema Chain** vận hành một chuỗi rạp chiếu phim hiện đại tại nhiều tỉnh/thành phố lớn (Hà Nội, TP. Hồ Chí Minh, Đà Nẵng, Hải Phòng...). Mỗi chi nhánh rạp quản lý nhiều phòng chiếu tiêu chuẩn khác nhau (`2D`, `3D`, `IMAX`), mỗi phòng chiếu có sơ đồ ghế vật lý cố định phân chia theo hạng ghế (`STD` – Ghế thường, `VIP` – Ghế VIP, `COUPLE` – Ghế đôi).

Hàng ngày, hệ thống phải xử lý đồng thời các quy trình nghiệp vụ cốt lõi:

1. **Quản lý danh mục phim và lập lịch chiếu (Screening Management):** Phân bổ phim vào từng phòng chiếu thuộc từng chi nhánh theo khung giờ cụ thể, đảm bảo thời lượng suất chiếu đủ cho thời lượng phim cộng thời gian dọn phòng/quảng cáo và tuyệt đối không chồng lấn lịch chiếu trong cùng một phòng.

2. **Bán vé và đặt chỗ theo sơ đồ ghế thời gian thực (Ticket Booking):** Phục vụ cả khách hàng thành viên lẫn khách vãng lai tại quầy. Hệ thống phải hiển thị trực quan ghế trống/ghế đã bán, tính đúng đơn giá vé từng ghế (giá cơ sở của suất chiếu + phụ thu hạng ghế tại thời điểm mua), xử lý thanh toán nguyên tử và ngăn chặn tuyệt đối việc một ghế bị bán hai lần cho cùng một suất chiếu ngay cả khi có nhiều nhân viên thu ngân thao tác đồng thời.

3. **Xử lý hủy vé, hoàn tiền và mở bán lại ghế (Ticket Cancellation & Seat Resale):** Khi đơn đặt vé hoặc vé bị hủy hợp lệ, khóa giữ ghế đang hiệu lực phải được giải phóng ngay để có thể bán lại cho khách hàng khác mà vẫn giữ nguyên lịch sử vé đã hủy phục vụ kiểm toán.

4. **Tra cứu lịch chiếu và kiểm tra ghế trống (Screening Search):** Cho phép nhân viên và khách hàng tìm kiếm nhanh các suất chiếu theo tên phim, chi nhánh, ngày chiếu, khoảng thời gian và xem ngay số lượng ghế còn trống với phân trang tường minh (không cắt ngầm kết quả).

5. **Tổng hợp báo cáo quản trị và tài chính (Business & Financial Reporting):**

• **Báo cáo Doanh thu (Revenue Report):** Do **Kế toán** lập, trình **Giám đốc Kinh doanh chuỗi rạp**, tổng hợp doanh thu thực thu (chỉ tính các vé hợp lệ `VALID` thuộc các đơn `CONFIRMED` đã có giao dịch thanh toán thành công `SUCCESS`), không bị nhân bản dữ liệu khi một đơn có nhiều lần thử thanh toán.

• **Báo cáo Tình hình Bán vé & Lấp đầy (Ticket Sales Report):** Do **Quản lý Chi nhánh** lập, gửi **Ban Điều hành và Quản lý Kinh doanh**, thống kê số vé bán ra, sức chứa phòng, số ghế trống, tỷ lệ lấp đầy (%) và doanh thu theo từng suất chiếu.

2. Phân tích Các Nhóm Người dùng và Phân quyền Nghiệp vụ

Vai trò (Role)	Mã vai trò	Trách nhiệm chính	Màn hình / Báo cáo sử dụng
Thu ngân / Nhân viên bán vé	`CASHIER`	Tra cứu suất chiếu, tư vấn chỗ ngồi, lập đơn đặt vé cho khách thành viên hoặc khách vãng lai, thu tiền và xử lý hủy vé theo quy định.	Màn hình Tra cứu Suất chiếu, Màn hình Đặt vé & Chọn ghế
Quản lý Chi nhánh	`MANAGER`	Lập và điều chỉnh lịch chiếu tại các phòng chiếu của chi nhánh, theo dõi tỷ lệ lấp đầy phòng chiếu và lập Báo cáo Tình hình Bán vé gửi Ban Điều hành & Quản lý Kinh doanh.	Màn hình Quản lý Suất chiếu, Màn hình Tra cứu, Báo cáo Tình hình Bán vé

Vai trò (Role)	Mã vai trò	Trách nhiệm chính	Màn hình / Báo cáo sử dụng
Kế toán Chuỗi rạp	`ACCOUNTANT`	Kiểm soát giao dịch thanh toán, đối soát doanh thu thực tế theo ngày/kỳ hạn và chi nhánh, lập Báo cáo Doanh thu trình Giám đốc Kinh doanh chuỗi rạp.	Báo cáo Doanh thu Chuỗi rạp, Báo cáo Tình hình Bán vé
Giám đốc Kinh doanh & Ban Điều hành	(Người nhận báo cáo)	Nhận và phê duyệt các báo cáo định kỳ từ Kế toán và Quản lý Chi nhánh để ra quyết định phân bổ suất chiếu, điều chỉnh giá vé và chiến lược khai thác.	Bản xem trên màn hình (Preview) & tệp PDF Báo cáo Doanh thu, Báo cáo Tình hình Bán vé

3. Phân tích Chi tiết Dữ liệu Cần Lưu trữ (14 Bảng Quan hệ Chuẩn 3NF)

Để đảm bảo tính toàn vẹn dữ liệu, loại bỏ hoàn toàn phụ thuộc bắc cầu và đạt chuẩn **3NF nghiêm ngặt**, hệ thống tổ chức lưu trữ thành **14 bảng quan hệ** sau đây:

3.1. Nhóm dữ liệu Hạ tầng Rạp & Phòng chiếu

1. **cinema_branch (Chi nhánh rạp):**

• **Ý nghĩa:** Lưu thông tin từng cụm rạp trong chuỗi.

• **Dữ liệu cần lưu:** Mã định danh chi nhánh (`branch_id`), mã nghiệp vụ (`branch_code`, VD: `CN-HN01`), tên chi nhánh (`branch_name`), địa chỉ (`address`), thành phố (`city`), số điện thoại liên hệ (`phone`), trạng thái hoạt động (`active`).

2. **screening_room (Phòng chiếu):**

• **Ý nghĩa:** Lưu các phòng chiếu thuộc từng chi nhánh rạp.

• **Dữ liệu cần lưu:** Mã định danh phòng (`room_id`), mã chi nhánh sở hữu (`branch_id`), tên phòng (`room_name`, VD: `Phòng 01`), loại công nghệ chiếu (`room_type`: `2D`, `3D`, `IMAX`), trạng thái hoạt động (`active`). Lưu ý chuẩn hóa: Tổng số ghế của phòng không lưu cứng để tránh dị thường cập nhật mà được tính tự động từ bảng `seat`.

3. **seat_type (Hạng ghế):**

• **Ý nghĩa:** Danh mục phân loại hạng ghế và mức phụ thu tiêu chuẩn.

• **Dữ liệu cần lưu:** Mã hạng ghế (`seat_type_code`: `STD`, `VIP`, `COUPLE`), tên hiển thị tiếng Việt (`type_name`: Ghế Thường, Ghế VIP, Ghế Đôi), mức phụ thu chuẩn (`surcharge`, VNĐ).

4. **seat (Ghế vật lý):**

• **Ý nghĩa:** Lưu từng vị trí ghế vật lý cố định bên trong một phòng chiếu.

• **Dữ liệu cần lưu:** Mã ghế (`seat_id`), mã phòng chiếu (`room_id`), hàng ghế (`seat_row`: `A`..`H`), số thứ tự ghế trong hàng (`seat_number`: `1`..`10`), mã hạng ghế (`seat_type_code`).

3.2. Nhóm dữ liệu Phim, Thể loại & Lịch chiếu

5. **movie (Phim điện ảnh):**

• **Ý nghĩa:** Lưu thông tin các bộ phim được khai thác tại chuỗi rạp.

• **Dữ liệu cần lưu:** Mã định danh phim (`movie_id`), mã phim (`movie_code`, VD: `MV-001`), tựa đề tiếng Việt (`title`), thời lượng chiếu tính bằng phút (`duration_minutes`), ngày khởi chiếu (`release_date`), phân loại độ tuổi (`age_rating`: `P`, `K`, `T13`, `T16`, `T18`), quốc gia sản xuất (`country`), mô tả tóm tắt (`description`).

6. **genre (Thể loại phim):**

• **Ý nghĩa:** Danh mục các thể loại điện ảnh (Hành động, Tâm lý, Kinh dị, Hoạt hình, Khoa học viễn tưởng...).

• **Dữ liệu cần lưu:** Mã thể loại (`genre_id`), tên thể loại (`genre_name`), mô tả (`description`).

7. **movie_genre (Phân loại Phim – Thể loại):**

• **Ý nghĩa:** Quan hệ nhiều – nhiều (M:N) cho phép một bộ phim thuộc nhiều thể loại và một thể loại có nhiều bộ phim.

• **Dữ liệu cần lưu:** Khóa chính phức hợp (`movie_id`, `genre_id`).

8. **screening (Suất chiếu):**

• **Ý nghĩa:** Một lần trình chiếu cụ thể của một bộ phim tại một phòng chiếu xác định trong một khoảng thời gian.

• **Dữ liệu cần lưu:** Mã suất chiếu (`screening_id`), mã phim (`movie_id`), mã phòng chiếu (`room_id`), thời điểm bắt đầu (`start_time`), thời điểm kết thúc dự kiến bao gồm quảng cáo và vệ sinh phòng (`end_time`), giá vé cơ sở của suất chiếu (`base_price`), trạng thái suất chiếu (`status`: `OPEN`, `CANCELLED`).

3.3. Nhóm dữ liệu Khách hàng, Nhân sự, Đặt vé, Khóa ghế & Thanh toán

9. **customer (Khách hàng thành viên):**

• **Ý nghĩa:** Lưu thông tin khách hàng đăng ký thành viên để tích lũy lịch sử đặt vé và chăm sóc khách hàng.

• **Dữ liệu cần lưu:** Mã khách hàng (`customer_id`), họ và tên (`full_name`), số điện thoại (`phone`), địa chỉ email (`email`), ngày sinh (`birth_date`), thời điểm đăng ký (`created_at`).

10. **employee (Nhân viên):**

• **Ý nghĩa:** Lưu thông tin nhân sự làm việc tại các chi nhánh (Thu ngân, Quản lý chi nhánh, Kế toán).

• **Dữ liệu cần lưu:** Mã nhân viên (`employee_id`), mã chi nhánh làm việc (`branch_id`), mã nhân sự (`employee_code`), họ và tên (`full_name`), số điện thoại (`phone`), vai trò (`role`: `CASHIER`, `MANAGER`, `ACCOUNTANT`), ngày vào làm (`hired_date`).

11. **booking (Đơn đặt vé):**

• **Ý nghĩa:** Ghi nhận một giao dịch đặt mua vé của khách hàng cho đúng một suất chiếu cụ thể.

• **Dữ liệu cần lưu:** Mã đơn (`booking_id`), mã đơn hiển thị (`booking_code`, VD: `BK-20261101-0001`), mã suất chiếu (`screening_id`), mã khách hàng (`customer_id` — cho phép `NULL` đối với quy trình **Khách vãng lai tại quầy**), mã nhân viên thực hiện (`employee_id`), thời điểm lập đơn (`booked_at`), trạng thái đơn (`booking_status`: `CONFIRMED`, `PENDING`, `CANCELLED`).

12. **ticket (Vé xem phim chi tiết theo ghế — Chuẩn 3NF):**

• **Ý nghĩa:** Mỗi dòng lưu một vé gắn với một ghế cụ thể trong một đơn đặt vé (bao gồm cả lịch sử các vé đã hủy `CANCELLED`).

• **Dữ liệu cần lưu:** Mã vé (`ticket_id`), mã đơn đặt vé (`booking_id`), mã ghế (`seat_id`), đơn giá bán thực tế chốt tại thời điểm xuất vé (`unit_price`), trạng thái vé (`ticket_status`: `VALID`, `CANCELLED`), thời điểm hủy (`cancelled_at`). Lưu ý chuẩn hóa 3NF: Không lưu dư thừa `screening_id` trong `ticket` vì mỗi `booking_id` đã xác định duy nhất một `screening_id` trong bảng `booking`.

13. **screening_seat_lock (Khóa ghế hiệu lực theo suất chiếu):**

• **Ý nghĩa:** Lưu danh sách các ghế đang được cấp phát hợp lệ (`VALID`) trong từng suất chiếu để cưỡng chế quy tắc chống bán trùng ghế ngay ở mức khóa chính CSDL mà không làm vi phạm 3NF trên bảng `ticket`.

• **Dữ liệu cần lưu:** Khóa chính phức hợp (`screening_id`, `seat_id`) và mã vé đang giữ khóa (`ticket_id` — duy nhất `UNIQUE`). Khi vé bị hủy (`CANCELLED`), bản ghi khóa ghế tương ứng tự động được giải phóng để cho phép bán lại ghế đó cho đơn mới.

14. **payment (Giao dịch Thanh toán):**

• **Ý nghĩa:** Lưu lịch sử các lần thanh toán gắn với một đơn đặt vé (cho phép ghi nhận cả các lần thanh toán thất bại `FAILED` trước khi thanh toán thành công `SUCCESS`, hoặc hoàn tiền `REFUNDED` khi hủy đơn).

• **Dữ liệu cần lưu:** Mã thanh toán (`payment_id`), mã đơn đặt vé (`booking_id`), số tiền thanh toán (`amount`), phương thức thanh toán (`payment_method`: `CASH`, `CARD`, `TRANSFER`, `EWALLET`), trạng thái thanh toán (`payment_status`: `PENDING`, `SUCCESS`, `FAILED`, `REFUNDED`), thời điểm thanh toán (`paid_at`), mã tham chiếu giao dịch (`transaction_ref`). Ở tầng cài đặt vật lý MySQL 8.4, một cột chỉ mục sinh có điều kiện (`active_success_booking`) được dùng để cưỡng chế tối đa 01 giao dịch `SUCCESS` trên mỗi đơn đặt vé.

4. Khối lượng Dữ liệu Lưu trữ & Quy mô Khởi tạo (Seed Data Scale)

Để phản ánh chân thực hoạt động của chuỗi rạp và phục vụ kiểm chứng các báo cáo quản trị, cơ sở dữ liệu kiểm thử được khởi tạo tất định với quy mô:

• **04 Chi nhánh rạp** tại Hà Nội, TP. Hồ Chí Minh, Đà Nẵng và Hải Phòng.

• **12 Phòng chiếu** (mỗi chi nhánh 3 phòng gồm `2D`, `3D`, `IMAX`), tổng cộng **960 ghế vật lý** (mỗi phòng 8 hàng A–H × 10 ghế = 80 ghế phân bổ `STD`, `VIP`, `COUPLE`).

• **08 Thể loại phim** và **20 Bộ phim** điện ảnh có đầy đủ phân loại độ tuổi, thời lượng thực tế.

• **12 Nhân viên** (thu ngân, quản lý chi nhánh, kế toán) và **45 Khách hàng thành viên** (cùng quy trình khách vãng lai).

• **160 Suất chiếu** phân bổ hợp lệ không trùng lịch trên 12 phòng chiếu.

• **160 Đơn đặt vé** với **373 Vé xem phim** (**327 khóa ghế hiệu lực** trong `screening_seat_lock` gồm 292 ghế đơn `CONFIRMED` + 35 ghế đơn `PENDING`, cùng 46 vé `CANCELLED` trong đó có 23 ghế hủy được bán lại hợp lệ) và **175 Giao dịch thanh toán** bao phủ toàn bộ các kịch bản: thanh toán thành công (`SUCCESS`), đang chờ (`PENDING`), thất bại (`FAILED`), hoàn tiền khi hủy (`REFUNDED`), đơn có thử lại thanh toán sau khi lỗi thẻ, và ghế bị hủy được bán lại thành công cho khách mới.

Quy tắc Nghiệp vụ & Cơ chế Đảm bảo Toàn vẹn (BR-01 .. BR-16)

(Tài liệu đáp ứng Tiêu chí R1 & R3 — Business Rules Specification)

1. Bảng Ánh xạ Quy tắc Nghiệp vụ (Business Rules Traceability)

Mỗi quy tắc nghiệp vụ dưới đây được định danh duy nhất (`BR-01` .. `BR-16`) và được bảo vệ đồng thời ở **Tầng Cơ sở Dữ liệu (DDL / Trigger / Index)** và **Tầng Dịch vụ Ứng dụng (Java Service & Transaction)**:

Mã QT	Phát biểu Quy tắc Nghiệp vụ (Tiếng Việt)	Cơ chế Cưỡng chế tại CSDL (Database Level — 14 bảng chuẩn 3NF)	Cơ chế Cưỡng chế tại Ứng dụng (Service / Transaction)
BR-01	Một chi nhánh rạp (`cinema_branch`) sở hữu nhiều phòng chiếu (`screening_room`). Tên phòng chiếu là duy nhất trong phạm vi cùng một chi nhánh; không được lập lịch vào phòng đang ngừng hoạt động (`active = 0`).	Khóa ngoại `fk_room_branch` (`branch_id NOT NULL`); `uq_room_branch_name (branch_id, room_name)`; Trigger `trg_screening_validate_ins/upd` kiểm tra `screening_room.active = 1`.	Kiểm tra chi nhánh và phòng chiếu tồn tại, `active = 1` trước khi lập lịch hoặc cập nhật suất chiếu.

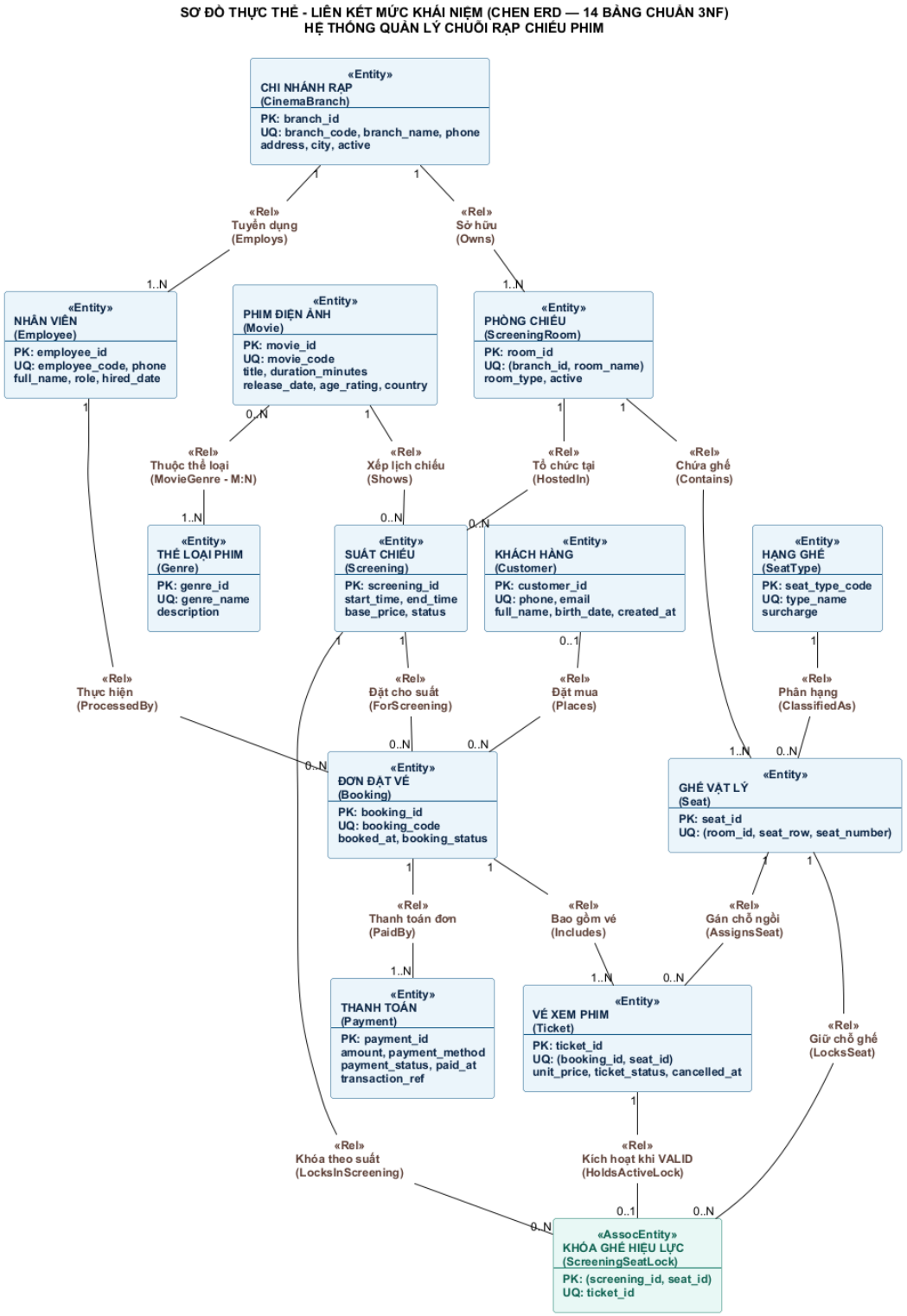
Mã QT	Phát biểu Quy tắc Nghiệp vụ (Tiếng Việt)	Cơ chế Cưỡng chế tại CSDL (Database Level — 14 bảng chuẩn 3NF)	Cơ chế Cưỡng chế tại Ứng dụng (Service / Transaction)
BR-02	Mỗi phòng chiếu chứa nhiều ghế vật lý (`seat`). Tọa độ `(room_id, seat_row, seat_number)` là duy nhất trong mỗi phòng, mỗi ghế thuộc một hạng ghế (`seat_type`) hợp lệ.	Khóa ngoại `fk_seat_room`, `fk_seat_type`; Ràng buộc duy nhất `uq_seat_position (room_id, seat_row, seat_number)`.	Hiển thị sơ đồ ghế trực tiếp từ danh sách ghế của phòng chiếu thuộc suất chiếu được chọn.
BR-03	Một bộ phim (`movie`) có thể thuộc nhiều thể loại (`genre`) và một thể loại có thể gắn với nhiều bộ phim; thời lượng phim phải từ 1 đến 400 phút.	Bảng trung gian `movie_genre` có khóa chính phức hợp `PRIMARY KEY (movie_id, genre_id)`; `CHECK (duration_minutes BETWEEN 1 AND 400)`.	Kiểm tra thời lượng phim khi tính toán thời gian kết thúc tối thiểu của suất chiếu.
BR-04	Một bộ phim có thể được xếp nhiều suất chiếu (`screening`), nhưng mỗi suất chiếu chỉ thuộc duy nhất một phòng chiếu và một bộ phim xác định; giá cơ sở phải dương (`base_price > 0`).	Khóa ngoại `fk_screening_movie`, `fk_screening_room` (`NOT NULL`); `CHECK (base_price > 0)`.	`ScreeningService.validateScreeningInput()` kiểm tra phim, phòng, ngày giờ và giá cơ sở hợp lệ.
BR-05	Thời gian kết thúc suất chiếu (`end_time`) phải lớn hơn thời gian bắt đầu (`start_time`) tối thiểu bằng thời lượng bộ phim (`movie.duration_minutes`).	`CHECK (end_time > start_time)` + Trigger `trg_screening_validate_ins` và `trg_screening_validate_upd` kiểm tra `TIMESTAMPDIFF(MINUTE, NEW.start_time, NEW.end_time) >= duration_minutes`.	`ScreeningService` kiểm tra `endTime >= startTime.plusMinutes(movie.durationMinutes())` và gợi ý tự động giờ kết thúc (+20 phút dọn phòng).
BR-06	**Chống trùng lịch chiếu trong cùng một phòng chiếu (kể cả khi thao tác đồng thời):** Hai suất chiếu có trạng thái `OPEN` trong cùng một `room_id` không được giao nhau về khoảng thời gian: không tồn tại suất chiếu khác có `existing.start_time < new.end_time AND existing.end_time > new.start_time`.	Trigger `trg_screening_validate_ins` & `trg_screening_validate_upd` truy vấn bảng `screening` và phát lỗi `SIGNAL SQLSTATE '45000'` nếu phát hiện chồng lấn.	Trong `ScreeningService`, mở giao dịch và thực hiện `SELECT room_id, active FROM screening_room WHERE room_id = ? FOR UPDATE` để tuần tự hóa mọi yêu cầu lập lịch trên cùng phòng chiếu trước khi kiểm tra trùng lịch và ghi dữ liệu.
BR-07	Mỗi đơn đặt vé (`booking`) phải gắn với một khách hàng thành viên hợp lệ (`customer_id` tham chiếu `customer`) **hoặc** thuộc quy trình **Khách vãng lai tại quầy** (`customer_id IS NULL`) được chọn tường minh trên giao diện; và gắn với nhân viên thực hiện (`employee_id`) cùng suất chiếu (`screening_id`).	Khóa ngoại `fk_booking_customer` (cho phép `NULL`), `fk_booking_employee` (`NOT NULL`), `fk_booking_screening` (`NOT NULL`).	Giao diện Đặt vé có tùy chọn rõ ràng "Khách vãng lai (Mua trực tiếp tại quầy)" hoặc chọn khách hàng thành viên từ danh sách.

Mã QT	Phát biểu Quy tắc Nghiệp vụ (Tiếng Việt)	Cơ chế Cưỡng chế tại CSDL (Database Level — 14 bảng chuẩn 3NF)	Cơ chế Cưỡng chế tại Ứng dụng (Service / Transaction)
BR-08	Một đơn đặt vé (`booking`) có thể bao gồm một hoặc nhiều vé (`ticket`); tất cả các vé trong cùng một đơn tự động thuộc đúng suất chiếu `booking.screening_id` mà không lưu dư thừa `screening_id` trong `ticket`, và không cho phép hủy suất chiếu đã có vé hợp lệ.	Thiết kế chuẩn 3NF: `ticket` tham chiếu `booking_id` (`fk_ticket_booking`) và `UNIQUE (booking_id, seat_id)`; Trigger `trg_ticket_validate_ins/upd` kiểm tra suất chiếu `OPEN`; Trigger `trg_screening_validate_upd` chặn hủy suất chiếu đã có vé `VALID`; cấm đổi `booking.screening_id` khi đã phát sinh vé (`trg_booking_before_upd`).	`BookingService` tạo `booking` và toàn bộ danh sách `ticket(booking_id, seat_id, unit_price, status)` trong cùng một giao dịch nguyên tử; `ScreeningService` chặn hủy suất chiếu đã bán vé.
BR-09	**Ghế trên vé phải thuộc đúng phòng chiếu của suất chiếu (kể cả khi cập nhật suất chiếu):** Mỗi vé (`ticket`) tham chiếu tới một `seat_id` mà `seat.room_id` bắt buộc phải bằng `screening.room_id` (suy từ `booking.screening_id`); đồng thời cấm đổi `room_id` của suất chiếu đã phát sinh vé.	Trigger `trg_ticket_validate_ins/upd` đối chiếu `seat.room_id = screening.room_id` qua `booking`, và Trigger `trg_screening_validate_upd` từ chối cập nhật `NEW.room_id <> OLD.room_id` nếu `EXISTS (SELECT 1 FROM ticket t JOIN booking b ON b.booking_id = t.booking_id WHERE b.screening_id = OLD.screening_id)`.	`BookingService` kiểm tra danh sách ghế thuộc phòng của suất chiếu trước khi lưu; `ScreeningService.updateScreening()` khóa phòng chiếu và chặn đổi phòng khi đã có vé.
BR-10	**Không bán trùng ghế cho cùng một suất chiếu (kể cả khi đặt vé đồng thời):** Cùng một `seat_id` không bao giờ được có quá một vé ở trạng thái `VALID` trong cùng một `screening_id`.	Bảng chuẩn 3NF `screening_seat_lock` với `PRIMARY KEY (screening_id, seat_id)` và `UNIQUE KEY uq_ssl_ticket (ticket_id)` được duy trì tự động bởi bộ Trigger đồng bộ `trg_ticket_after_ins/upd/del` và bảo vệ chống ghi sai lệch trực tiếp bởi `trg_ssl_guard_ins/upd/del`.	`BookingService` khóa dòng suất chiếu `SELECT screening_id, room_id, base_price, status FROM screening WHERE screening_id = ? FOR UPDATE`, kiểm tra các ghế chưa bị khóa trong `screening_seat_lock`, sau đó mới chèn vé trong giao dịch nguyên tử.
BR-11	**Tính nhất quán khi hủy vé và bán lại ghế (Cancellation & Seat Resale):** Khi hủy đơn/vé, `ticket_status` chuyển thành `'CANCELLED'`, `cancelled_at` được ghi nhận; bản ghi khóa ghế tương ứng trong `screening_seat_lock` tự động được xóa bỏ, cho phép ghế đó được đặt lại trong đơn mới mà vẫn giữ nguyên lịch sử vé đã hủy trong `ticket`.	Trigger `trg_booking_status_sync` chuyển toàn bộ vé của đơn hủy sang `'CANCELLED'` và thanh toán `SUCCESS` sang `'REFUNDED'`; Trigger `trg_ticket_after_upd` xóa ngay dòng khóa trong `screening_seat_lock`.	`BookingService.cancelBooking(bookingId)` cập nhật nguyên tử trong một giao dịch: `booking_status = 'CANCELLED'`, `ticket_status = 'CANCELLED'`, và hoàn tiền thanh toán `'REFUNDED'`.

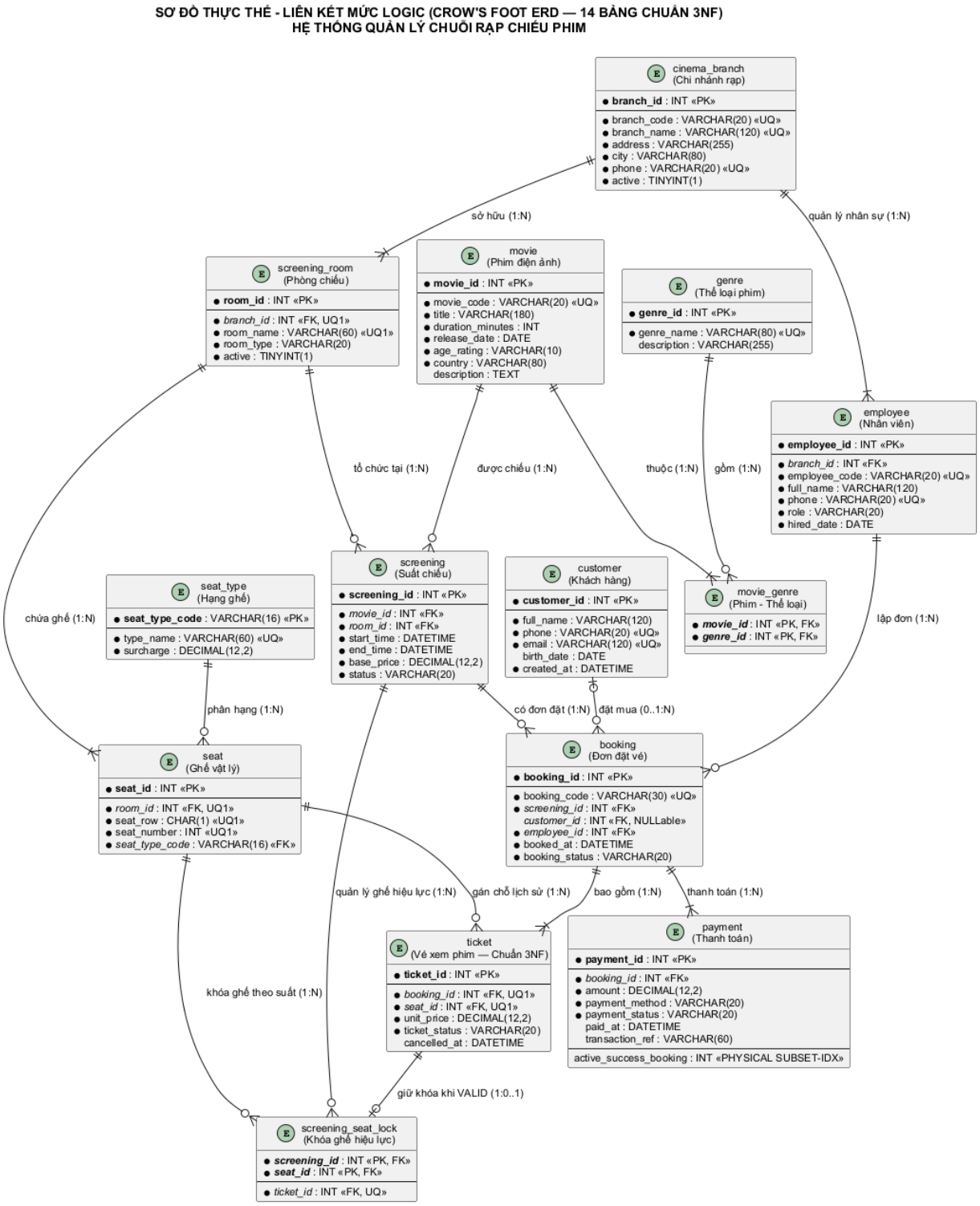
Mã QT	Phát biểu Quy tắc Nghiệp vụ (Tiếng Việt)	Cơ chế Cưỡng chế tại CSDL (Database Level — 14 bảng chuẩn 3NF)	Cơ chế Cưỡng chế tại Ứng dụng (Service / Transaction)
BR-12	**Bảo toàn giá vé lịch sử (Historical Ticket Pricing):** Đơn giá vé `ticket.unit_price` được chốt tại thời điểm bán bằng `screening.base_price + seat_type.surcharge` và lưu trực tiếp vào bảng `ticket`. Mọi thay đổi về sau đối với `screening.base_price` hoặc `seat_type.surcharge` không làm thay đổi doanh thu lịch sử của các vé đã phát hành.	Cột `ticket.unit_price DECIMAL(12,2) NOT NULL CHECK (unit_price >= 0)` lưu giá trị thực tế lúc giao dịch.	`BookingService` tính `unitPrice = screening.basePrice().add(seat.surcharge())` ngay trong giao dịch đặt vé và ghi cố định vào `ticket.unit_price`.
BR-13	**Mỗi đơn đặt vé có tối đa 01 giao dịch thanh toán thành công (SUCCESS), không xác nhận thanh toán cho đơn đã hủy, và số tiền thanh toán phải khớp tổng tiền vé hợp lệ:** Một đơn có thể có các lần thanh toán thất bại (`FAILED`) trước đó, nhưng chỉ được có duy nhất một bản ghi `payment_status = 'SUCCESS'`.	Cột chỉ mục vật lý có điều kiện `active_success_booking INT GENERATED ALWAYS AS (CASE WHEN payment_status = 'SUCCESS' THEN booking_id ELSE NULL END) STORED` kèm `UNIQUE KEY uq_payment_one_success (active_success_booking)` + Trigger `trg_payment_validate_ins/upd` kiểm tra đơn chưa bị `CANCELLED` và `amount` khớp tổng `unit_price` của các vé `VALID`.	`BookingService` đảm bảo tạo và cập nhật thanh toán nhất quán với tổng tiền vé hợp lệ của đơn.
BR-14	**Quy tắc Ghi nhận Doanh thu Chuẩn (Revenue Recognition Rule):** Chỉ những vé có `ticket_status = 'VALID'` thuộc đơn đặt vé `booking_status = 'CONFIRMED'` và có giao dịch thanh toán ở trạng thái `payment_status = 'SUCCESS'` mới được ghi nhận vào doanh thu. Các giao dịch ở trạng thái `PENDING`, `FAILED` hoặc đã hoàn tiền `REFUNDED` (do hủy vé) tuyệt đối bị loại trừ khỏi doanh thu.	View `vw_verified_booking_revenue` và các truy vấn báo cáo lọc chặt chẽ điều kiện `b.booking_status = 'CONFIRMED' AND t.ticket_status = 'VALID' AND p.payment_status = 'SUCCESS'`.	Cả `RevenueReportRepository` và `TicketSalesReportRepository` đều áp dụng cùng một định nghĩa doanh thu và được kiểm chứng chéo bằng SQL độc lập + Known-Answer Fixture trong `ReportAccuracyIT`.
BR-15	**Chống nhân bản doanh thu do kết nối bằng (Anti-Fanout Join Rule):** Khi một đơn đặt vé có nhiều bản ghi trong bảng `payment` (ví dụ: 1 lần `FAILED` do lỗi thẻ, sau đó 1 lần `SUCCESS`), truy vấn báo cáo tuyệt đối không được nhân đôi số vé hoặc tổng tiền vé.	Ràng buộc vật lý `uq_payment_one_success` đảm bảo tối đa 1 dòng `payment` có `payment_status = 'SUCCESS'` cho mỗi `booking_id`; đồng thời truy vấn chỉ kết nối với bản ghi `payment` có `payment_status = 'SUCCESS'` hoặc dùng `EXISTS`.	Được kiểm chứng bằng dữ liệu mẫu có 15 đơn nhiều lần thanh toán (`FAILED` + `SUCCESS`) và bài test `ReportAccuracyIT`.

Mã QT	Phát biểu Quy tắc Nghiệp vụ (Tiếng Việt)	Cơ chế Cưỡng chế tại CSDL (Database Level — 14 bảng chuẩn 3NF)	Cơ chế Cưỡng chế tại Ứng dụng (Service / Transaction)
BR-16	**Trách nhiệm lập và nhận báo cáo quản trị:** - **Báo cáo Doanh thu:** Người lập là **Kế toán chuỗi rạp**, người nhận là **Giám đốc Kinh doanh chuỗi rạp**. - **Báo cáo Tình hình Bán vé:** Người lập là **Quản lý Chi nhánh**, người nhận là **Ban Điều hành và Quản lý Kinh doanh**.	—	Hiển thị rõ thông tin Người lập, Người nhận, Thời điểm kết xuất, Chữ ký xác nhận trên cả giao diện Xem trước (Preview) và tệp PDF xuất ra.

CHƯƠNG 2. THIẾT KẾ SƠ ĐỒ THỰC THỂ – LIÊN KẾT ERD (TIÊU CHÍ R2)



Hình 2.1. Sơ đồ Thực thể – Liên kết mức Khái niệm theo ký hiệu Chen (Conceptual ERD — 14 bảng chuẩn 3NF).



Hình 2.2. Sơ đồ Thực thể – Liên kết mức Logic theo ký hiệu Crow's Foot (Logical ERD — 14 bảng chuẩn 3NF).

(Tài liệu đáp ứng Tiêu chí R2 — 2.5 điểm)

1. Tổng quan Sơ đồ Thực thể – Liên kết (ERD)

Mô hình Thực thể – Liên kết của **Hệ thống Quản lý Chuỗi Rạp Chiếu Phim** được xây dựng gồm **12 tập thực thể nghiệp vụ chính**, **01 quan hệ kết hợp nhiều – nhiều (movie_genre)** và **01 thực thể kết hợp cấp phát ghế hiệu lực (screening_seat_lock)**, tương ứng chính xác với **14 bảng quan hệ đạt chuẩn 3NF nghiêm ngặt** khi ánh xạ sang mô hình quan hệ.

Đặc biệt, trong mô hình ERD và lược đồ quan hệ chuẩn 3NF:

• Thực thể **Ticket (ticket)** liên kết với **Screening (screening)** thông qua **Booking (booking)**, **không** tạo liên kết dư thừa trực tiếp `screening -> ticket` nhằm triệt tiêu hoàn toàn phụ thuộc hàm bắc cầu ticket_id → booking_id → screening_id.

• Thực thể kết hợp **ScreeningSeatLock (screening_seat_lock)** biểu diễn trạng thái cấp phát ghế đang có hiệu lực (`VALID`) của một tấm vé trên cặp `(screening_id, seat_id)`, vừa bảo toàn lịch sử mọi lần hủy vé/đặt lại ghế trong `ticket`, vừa đảm bảo tại mọi thời điểm mỗi ghế trong một suất chiếu chỉ gắn với tối đa một vé hợp lệ ngay ở mức khóa chính CSDL.

Hệ thống cung cấp đồng thời:

1. **Sơ đồ ERD mức khái niệm (Chen Notation):** Mã nguồn tại `docs/erd/erd-chen.puml`, ảnh kết xuất tại `docs/erd/erd-chen.png` và `docs/erd/erd-chen.svg`.

2. **Sơ đồ ERD mức logic (Crow's Foot Notation):** Mã nguồn tại `docs/erd/erd-logical.puml`, ảnh kết xuất tại `docs/erd/erd-logical.png` và `docs/erd/erd-logical.svg`.

3. **Sơ đồ Mermaid ER trực quan:** Hiển thị trực tiếp dưới đây.

2. Thuyết minh Chi tiết 14 Tập Thực thể & Thực thể Kết hợp (Entities)

(Lưu ý học thuật: Cột "Khóa dự tuyển khác (Alternate Keys)" dưới đây chỉ liệt kê các khóa tối thiểu định danh duy nhất trên **toàn bộ các bộ (rows)** của quan hệ; các biểu thức chỉ mục có điều kiện trên tập con trạng thái không phải khóa dự tuyển của quan hệ).

STT	Tên Thực thể (Code / SQL)	Tên Tiếng Việt	Khóa chính (PK)	Khóa dự tuyển khác trên toàn bảng (Alternate Keys)	Giải thích Ý nghĩa Nghiệp vụ
1	`CinemaBranch` (`cinema_branch`)	Chi nhánh rạp	`branch_id`	`branch_code`, `branch_name`, `phone`	Đại diện cho một cụm rạp chiếu phim thuộc chuỗi rạp đặt tại một địa điểm/thành phố cụ thể.
2	`ScreeningRoom` (`screening_room`)	Phòng chiếu	`room_id`	`(branch_id, room_name)`	Không gian chiếu phim vật lý thuộc một chi nhánh rạp, phân loại theo công nghệ `2D`, `3D`, `IMAX`.
3	`SeatType` (`seat_type`)	Hạng ghế	`seat_type_code`	`type_name`	Danh mục phân hạng ghế (`STD` – Ghế Thường, `VIP` – Ghế VIP, `COUPLE` – Ghế Đôi) và phụ thu chuẩn (`surcharge`).
4	`Seat` (`seat`)	Ghế vật lý	`seat_id`	`(room_id, seat_row, seat_number)`	Vị trí chỗ ngồi cố định trong một phòng chiếu, xác định bởi hàng ghế (`A`..`H`) và số thứ tự ghế.

STT	Tên Thực thể (Code / SQL)	Tên Tiếng Việt	Khóa chính (PK)	Khóa dự tuyển khác trên toàn bảng (Alternate Keys)	Giải thích Ý nghĩa Nghiệp vụ
5	`Movie` (`movie`)	Phim điện ảnh	`movie_id`	`movie_code`	Tác phẩm điện ảnh được đưa vào khai thác thương mại với thời lượng (`duration_minutes`) và độ tuổi giới hạn (`age_rating`).
6	`Genre` (`genre`)	Thể loại phim	`genre_id`	`genre_name`	Danh mục thể loại điện ảnh phục vụ phân loại và tìm kiếm phim.
7	`MovieGenre` (`movie_genre`)	Phim – Thể loại	`(movie_id, genre_id)`	—	Thực thể kết hợp giải quyết mối quan hệ M:N giữa Phim và Thể loại.
8	`Screening` (`screening`)	Suất chiếu	`screening_id`	—	Một lịch trình chiếu cụ thể của một bộ phim tại một phòng chiếu trong khoảng thời gian `[start_time, end_time)` với mức giá cơ sở `base_price`.
9	`Customer` (`customer`)	Khách hàng	`customer_id`	`phone`, `email`	Khách hàng đăng ký thành viên của chuỗi rạp để lưu lịch sử mua vé và nhận thông tin.
10	`Employee` (`employee`)	Nhân viên	`employee_id`	`employee_code`, `phone`	Nhân sự làm việc tại chi nhánh rạp gồm Thu ngân (`CASHIER`), Quản lý chi nhánh (`MANAGER`) và Kế toán (`ACCOUNTANT`).
11	`Booking` (`booking`)	Đơn đặt vé	`booking_id`	`booking_code`	Phiên giao dịch đặt mua một hoặc nhiều vé cho một suất chiếu xác định (do khách thành viên hoặc khách vãng lai thực hiện).
12	`Ticket` (`ticket`)	Vé xem phim	`ticket_id`	`(booking_id, seat_id)`	Tấm vé cụ thể gán một ghế (`seat_id`) trong một đơn đặt vé (`booking_id`) với đơn giá lịch sử `unit_price` và trạng thái `VALID` / `CANCELLED`. Suất chiếu được xác định qua `booking` để đảm bảo chuẩn 3NF.

STT	Tên Thực thể (Code / SQL)	Tên Tiếng Việt	Khóa chính (PK)	Khóa dự tuyển khác trên toàn bảng (Alternate Keys)	Giải thích Ý nghĩa Nghiệp vụ
13	ScreeningSeatLock (screening_seat_lock)	Khóa ghế hiệu lực	(screening_id, seat_id)	ticket_id	Thực thể kết hợp lưu sự phân bổ ghế đang có hiệu lực (VALID) của tấm vé ticket_id trên cặp (screening_id, seat_id). Khi vé bị hủy (CANCELLED), bản ghi khóa ghế được giải phóng để cho phép bán lại ghế đó cho đơn mới.
14	Payment (payment)	Giao dịch thanh toán	payment_id	—	Bản ghi giao dịch thanh toán cho đơn đặt vé (PENDING, SUCCESS, FAILED, REFUNDED). Ràng buộc tối đa 01 giao dịch SUCCESS cho mỗi booking_id là ràng buộc duy nhất trên tập con trạng thái, thực thi ở mức vật lý.

3. Thuyết minh Chi tiết Các Mối Quan hệ và Bản số (Relationships & Cardinalities)

1. **cinema_branch (1) — Sở hữu — (1..N) screening_room:**

• Một chi nhánh rạp phải có ít nhất một phòng chiếu và có thể sở hữu nhiều phòng chiếu. Mỗi phòng chiếu thuộc về duy nhất một chi nhánh (screening_room.branch_id NOT NULL).

2. **cinema_branch (1) — Tuyển dụng — (1..N) employee:**

• Một chi nhánh quản lý nhiều nhân viên; mỗi nhân viên biên chế tại một chi nhánh xác định (employee.branch_id NOT NULL).

3. **screening_room (1) — Chứa ghế — (1..N) seat:**

• Mỗi phòng chiếu có nhiều ghế vật lý được lắp đặt cố định; mỗi ghế vật lý thuộc duy nhất một phòng chiếu (seat.room_id NOT NULL).

4. **seat_type (1) — Phân hạng — (0..N) seat:**

• Một hạng ghế (STD, VIP, COUPLE) áp dụng cho nhiều ghế; mỗi ghế vật lý có đúng một hạng ghế (seat.seat_type_code NOT NULL).

5. **movie (0..N) — Thuộc thể loại (movie_genre) — (1..N) genre:**

• Một bộ phim có ít nhất một hoặc nhiều thể loại; một thể loại có thể gắn cho nhiều bộ phim. Biểu diễn qua bảng movie_genre(movie_id, genre_id).

6. **movie (1) — Được xếp lịch — (0..N) screening:**

• Một bộ phim có thể có không hoặc nhiều suất chiếu; mỗi suất chiếu trình chiếu đúng một bộ phim (screening.movie_id NOT NULL).

7. **screening_room (1) — Tổ chức tại — (0..N) screening:**

• Một phòng chiếu tổ chức nhiều suất chiếu theo các khung giờ không giao nhau; mỗi suất chiếu diễn ra tại duy nhất một phòng chiếu (`screening.room_id NOT NULL`).

8. **customer (0..1) — Đặt mua — (0..N) booking:**

• Một khách hàng thành viên có thể đặt nhiều đơn vé. Ngược lại, một đơn đặt vé có thể thuộc về một khách hàng thành viên (`customer_id` khác `NULL`) hoặc là **Khách vãng lai mua tại quầy** (`customer_id IS NULL`), do đó bản số phía `customer` là `0..1`.

9. **employee (1) — Thực hiện — (0..N) booking:**

• Một nhân viên thu ngân lập nhiều đơn đặt vé; mỗi đơn đặt vé được ghi nhận bởi đúng một nhân viên phụ trách (`booking.employee_id NOT NULL`).

10. **screening (1) — Có đơn đặt — (0..N) booking:**

• Một suất chiếu có thể có nhiều đơn đặt vé; mỗi đơn đặt vé đặt chỗ cho đúng một suất chiếu xác định (`booking.screening_id NOT NULL`).

11. **booking (1) — Bao gồm — (1..N) ticket & seat (1) — Gán chỗ lịch sử — (0..N) ticket:**

• Một đơn đặt vé bao gồm từ một đến nhiều vé xem phim; mỗi vé gắn với một ghế vật lý `seat_id` và chỉ xuất hiện tối đa một lần trong cùng một đơn (`UNIQUE(booking_id, seat_id)`). Trên tập lịch sử nhiều đơn khác nhau, một `seat_id` có thể xuất hiện ở nhiều dòng `ticket` (ví dụ: đơn trước bị hủy `CANCELLED`, đơn sau đặt lại ghế đó ở trạng thái `VALID`).

12. **screening (1), seat (1) & ticket (1) — Khóa ghế hiệu lực (screening_seat_lock) — bản số 1:0..N (từ screening, seat) và 1:0..1 (từ ticket):**

• Mỗi tấm vé đang ở trạng thái hợp lệ (`ticket_status = 'VALID'`) giữ đúng `1` bản ghi trong `screening_seat_lock` (`UNIQUE(ticket_id)`), còn vé đã hủy (`CANCELLED`) có `0` bản ghi trong `screening_seat_lock` (bản số `1 : 0..1`).

• Khóa chính `PRIMARY KEY (screening_id, seat_id)` của `screening_seat_lock` đảm bảo tuyệt đối tại mọi thời điểm chỉ có tối đa **01 vé VALID** chiếm giữ một ghế trong một suất chiếu, không cần lưu dư thừa `screening_id` bên trong bảng `ticket`.

13. **booking (1) — Thanh toán — (1..N) payment:**

• Một đơn đặt vé có thể có nhiều bản ghi giao dịch thanh toán theo thời gian (ví dụ: lần 1 quẹt thẻ `FAILED`, lần 2 chuyển khoản `SUCCESS`). Ràng buộc duy nhất có điều kiện ở tầng vật lý đảm bảo mỗi đơn chỉ có tối đa **01 giao dịch ở trạng thái SUCCESS**.

CHƯƠNG 3. LƯỢC ĐỒ CƠ SỞ DỮ LIỆU QUAN HỆ 14 BẢNG & TỪ ĐIỂN DỮ LIỆU (TIÊU CHÍ R3)

(Tài liệu đáp ứng Tiêu chí R3 — 2.0 điểm)

1. Tổng hợp Danh mục 14 Lược đồ Quan hệ Mức Logic (Chuẩn 3NF)

Quy ước ký hiệu trong biểu diễn lược đồ quan hệ mức logic:

• **Gạch chân nét liền (PK)**: Thuộc tính thuộc Khóa chính (Primary Key).

• Chữ nghiêng kèm dấu `#` (`FK`): Thuộc tính đóng vai trò Khóa ngoại (Foreign Key).

• `AK`: Khóa dự tuyển khác trên **toàn bộ các bộ của quan hệ** (Alternate Key — mọi thuộc tính thành phần đều `NOT NULL` và `UNIQUE` trên toàn bảng).

1. **cinema_branch** (`branch_id`, `branch_code` [AK], `branch_name` [AK], `address`, `city`, `phone` [AK], `active`)

2. **screening_room** (`room_id`, `#branch_id`, `room_name`, `room_type`, `active`) — với `AK(branch_id, room_name)`

3. **seat_type** (`seat_type_code`, `type_name` [AK], `surcharge`)

4. **seat** (`seat_id`, `#room_id`, `seat_row`, `seat_number`, `#seat_type_code`) — với `AK(room_id, seat_row, seat_number)`

5. **movie** (`movie_id`, `movie_code` [AK], `title`, `duration_minutes`, `release_date`, `age_rating`, `country`, `description`)

6. **genre** (`genre_id`, `genre_name` [AK], `description`)

7. **movie_genre** (`#movie_id`, `#genre_id`)

8. **screening** (`screening_id`, `#movie_id`, `#room_id`, `start_time`, `end_time`, `base_price`, `status`)

9. **customer** (`customer_id`, `full_name`, `phone` [AK], `email` [AK], `birth_date`, `created_at`)

10. **employee** (`employee_id`, `#branch_id`, `employee_code` [AK], `full_name`, `phone` [AK], `role`, `hired_date`)

11. **booking** (`booking_id`, `booking_code` [AK], `#screening_id`, `#customer_id` [NULLable], `#employee_id`, `booked_at`, `booking_status`)

12. **ticket** (`ticket_id`, `#booking_id`, `#seat_id`, `unit_price`, `ticket_status`, `cancelled_at`) — với `AK(booking_id, seat_id)`

13. **screening_seat_lock** (`#screening_id`, `#seat_id`, `#ticket_id` [AK])

14. **payment** (`payment_id`, `#booking_id`, `amount`, `payment_method`, `payment_status`, `paid_at`, `transaction_ref`)

Phân tách Rõ Mức Logic Quan hệ và Mức Cài đặt Vật lý MySQL 8.4

• **Ở mức mô hình quan hệ logic (Relational Model):** Bảng `payment` chỉ gồm 7 thuộc tính nghiệp vụ nêu trên và có duy nhất một khóa dự tuyển trên toàn bảng là `PK(payment_id)`.

• **Ở mức cài đặt vật lý (Physical DDL trên MySQL 8.4 InnoDB):** Do MySQL 8.4 không hỗ trợ cú pháp `CREATE UNIQUE INDEX ... WHERE payment_status = 'SUCCESS'` như PostgreSQL, tệp `01_schema.sql` cài đặt thêm một cột biểu thức vật lý `active_success_booking INT GENERATED ALWAYS AS (CASE WHEN payment_status = 'SUCCESS' THEN booking_id ELSE NULL END) STORED` kèm `UNIQUE KEY uq_payment_one_success`. Cột này mang giá trị `NULL` đối với các dòng `PENDING`, `FAILED`, `REFUNDED`, nên **tuyệt đối không phải là thuộc tính logic hay khóa dự tuyển của quan hệ payment**, mà chỉ là cấu trúc chỉ mục vật lý để cưỡng chế ràng buộc duy nhất trên tập con các giao dịch thành công.

• Đối với nghiệp vụ vé và giữ ghế: Bảng `ticket` đã loại bỏ hoàn toàn cột dư thừa `screening_id` và biểu thức `active_seat_key`; việc chống bán trùng ghế đang hợp lệ (`VALID`) được mô hình hóa tường minh bằng quan hệ chuẩn 3NF `screening_seat_lock(screening_id, seat_id, ticket_id)`.

2. Đặc tả Chi tiết 14 Bảng Quan hệ (Data Dictionary — Khớp 100% `01_schema.sql`)

Bảng 1: `cinema_branch` — Chi nhánh Rạp chiếu phim

Lưu trữ danh sách các cụm rạp thuộc chuỗi trên toàn quốc.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
`branch_id`	`INT AUTO_INCREMENT`	`NOT NULL`	Auto	`PRIMARY KEY (pk_cinema_branch)`	Mã định danh tự tăng của chi nhánh rạp.
`branch_code`	`VARCHAR(20)`	`NOT NULL`	—	`UNIQUE (uq_branch_code)`	Mã nghiệp vụ của chi nhánh (VD: `CN-HN01`, `CN-HCM01`).
`branch_name`	`VARCHAR(120)`	`NOT NULL`	—	`UNIQUE (uq_branch_name)`	Tên giao dịch chính thức của chi nhánh rạp.
`address`	`VARCHAR(255)`	`NOT NULL`	—	—	Địa chỉ chi tiết của cụm rạp.
`city`	`VARCHAR(80)`	`NOT NULL`	—	—	Tỉnh / Thành phố nơi đặt chi nhánh.
`phone`	`VARCHAR(20)`	`NOT NULL`	—	`UNIQUE (uq_branch_phone)`	Số điện thoại đường dây nóng của chi nhánh.
`active`	`TINYINT(1)`	`NOT NULL`	`1`	`CHECK (active IN (0, 1))`	Trạng thái hoạt động (`1`: Đang hoạt động, `0`: Tạm ngưng).

Bảng 2: `screening_room` — Phòng chiếu phim

Lưu thông tin các phòng chiếu thuộc từng chi nhánh rạp.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
`room_id`	`INT AUTO_INCREMENT`	`NOT NULL`	Auto	`PRIMARY KEY (pk_screening_room)`	Mã định danh tự tăng của phòng chiếu.
`branch_id`	`INT`	`NOT NULL`	—	`FK -> cinema_branch(branch_id)`, `UQ(branch_id, room_name)`	Mã chi nhánh rạp sở hữu phòng chiếu này.
`room_name`	`VARCHAR(60)`	`NOT NULL`	—	`UQ(branch_id, room_name)`	Tên phòng chiếu trong chi nhánh (VD: `Phòng 01 - 2D`).
`room_type`	`VARCHAR(20)`	`NOT NULL`	`'2D'`	`CHECK (room_type IN ('2D', '3D', 'IMAX'))`	Công nghệ phòng chiếu (`2D`, `3D`, `IMAX`).
`active`	`TINYINT(1)`	`NOT NULL`	`1`	`CHECK (active IN (0, 1))`	Trạng thái vận hành của phòng chiếu.

Bảng 3: `seat_type` — Danh mục Hạng ghế

Lưu cấu hình các loại ghế và mức phụ thu cộng thêm vào giá vé cơ sở của suất chiếu.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
`seat_type_code`	`VARCHAR(16)`	`NOT NULL`	—	`PRIMARY KEY (pk_seat_type)`, `CHECK (seat_type_code IN ('STD', 'VIP', 'COUPLE'))`	Mã hạng ghế (`STD`: Thường, `VIP`: VIP, `COUPLE`: Ghế đôi).
`type_name`	`VARCHAR(60)`	`NOT NULL`	—	`UNIQUE (uq_seat_type_name)`	Tên hiển thị tiếng Việt của hạng ghế.
`surcharge`	`DECIMAL(12,2)`	`NOT NULL`	`0.00`	`CHECK (surcharge >= 0)`	Phụ thu hạng ghế (VNĐ) cộng vào `base_price` của suất chiếu.

Bảng 4: `seat` — Ghế vật lý trong Phòng chiếu

Lưu tọa độ từng ghế vật lý thuộc một phòng chiếu xác định.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
`seat_id`	`INT AUTO_INCREMENT`	`NOT NULL`	Auto	`PRIMARY KEY (pk_seat)`	Mã định danh tự tăng của ghế vật lý.
`room_id`	`INT`	`NOT NULL`	—	`FK -> screening_room(room_id)`, `UQ(room_id, seat_row, seat_number)`	Mã phòng chiếu chứa ghế này.
`seat_row`	`CHAR(1)`	`NOT NULL`	—	`UQ(room_id, seat_row, seat_number)`, `CHECK (seat_row >= 'A' AND seat_row <= 'Z')`	Ký hiệu hàng ghế theo bảng chữ cái (`A`, `B`, ..., `H`).
`seat_number`	`INT`	`NOT NULL`	—	`UQ(room_id, seat_row, seat_number)`, `CHECK (seat_number >= 1 AND seat_number <= 50)`	Số thứ tự ghế trong hàng (`1`, `2`, ..., `10`).
`seat_type_code`	`VARCHAR(16)`	`NOT NULL`	`'STD'`	`FK -> seat_type(seat_type_code)`	Mã hạng ghế áp dụng cho vị trí ghế này.

Bảng 5: `movie` — Phim điện ảnh

Lưu danh mục phim được chiếu tại chuỗi rạp.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
`movie_id`	`INT AUTO_INCREMENT`	`NOT NULL`	Auto	`PRIMARY KEY (pk_movie)`	Mã định danh tự tăng của bộ phim.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
movie_code	VARCHAR(20)	NOT NULL	—	UNIQUE (uq_movie_code)	Mã nghiệp vụ của bộ phim (VD: MV-001).
title	VARCHAR(180)	NOT NULL	—	—	Tựa đề chính thức của bộ phim (tiếng Việt).
duration_minutes	INT	NOT NULL	—	CHECK (duration_minutes >= 1 AND duration_minutes <= 400)	Thời lượng chiếu thực tế của bộ phim (tính bằng phút).
release_date	DATE	NOT NULL	—	—	Ngày khởi chiếu chính thức.
age_rating	VARCHAR(10)	NOT NULL	'P'	CHECK (age_rating IN ('P', 'K', 'T13', 'T16', 'T18'))	Phân loại độ tuổi theo quy định Cục Điện ảnh (P, K, T13, T16, T18).
country	VARCHAR(80)	NOT NULL	—	—	Quốc gia sản xuất phim.
description	TEXT	NULL	NULL	—	Tóm tắt nội dung phim.

Bảng 6: genre — Thể loại Phim

Lưu danh mục thể loại điện ảnh.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
genre_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY (pk_genre)	Mã định danh thể loại phim.
genre_name	VARCHAR(80)	NOT NULL	—	UNIQUE (uq_genre_name)	Tên thể loại phim (Hành động, Tâm lý, Kinh dị...).
description	VARCHAR(255)	NULL	NULL	—	Mô tả chi tiết thể loại.

Bảng 7: movie_genre — Liên kết Phim và Thể loại

Bảng trung gian biểu diễn quan hệ nhiều – nhiều (M:N) giữa movie và genre.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
movie_id	INT	NOT NULL	—	PK(movie_id, genre_id), FK -> movie(movie_id)	Mã bộ phim.
genre_id	INT	NOT NULL	—	PK(movie_id, genre_id), FK -> genre(genre_id)	Mã thể loại gắn cho bộ phim.

Bảng 8: screening — Suất chiếu (Lịch chiếu)

Lưu lịch trình chiếu một bộ phim tại một phòng chiếu xác định.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
screening_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY (pk_screening)	Mã định danh suất chiếu.
movie_id	INT	NOT NULL	—	FK -> movie(movie_id)	Mã bộ phim được trình chiếu.
room_id	INT	NOT NULL	—	FK -> screening_room(room_id)	Mã phòng chiếu tổ chức suất chiếu.
start_time	DATETIME	NOT NULL	—	INDEX idx_screening_room_time (room_id, start_time, end_time)	Thời điểm bắt đầu suất chiếu.
end_time	DATETIME	NOT NULL	—	CHECK (end_time > start_time) + Trigger kiểm tra ≥ start_time + duration_minutes và không chồng lấn lịch phòng	Thời điểm kết thúc suất chiếu (đã gồm thời lượng phim và thời gian nghỉ/vệ sinh phòng).
base_price	DECIMAL(12,2)	NOT NULL	—	CHECK (base_price > 0)	Giá vé cơ sở của suất chiếu (VNĐ) áp dụng cho hạng ghế chuẩn STD.
status	VARCHAR(20)	NOT NULL	'OPEN'	CHECK (status IN ('OPEN', 'CANCELLED'))	Trạng thái suất chiếu (OPEN: Đang mở bán, CANCELLED: Đã hủy suất).

Bảng 9: customer — Khách hàng thành viên

Lưu thông tin khách hàng đăng ký thành viên tại chuỗi rạp.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
customer_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY (pk_customer)	Mã định danh khách hàng thành viên.
full_name	VARCHAR(120)	NOT NULL	—	—	Họ và tên đầy đủ của khách hàng.
phone	VARCHAR(20)	NOT NULL	—	UNIQUE (uq_customer_phone)	Số điện thoại liên hệ và định danh thành viên tại quầy.
email	VARCHAR(120)	NOT NULL	—	UNIQUE (uq_customer_email)	Địa chỉ thư điện tử nhận vé của khách hàng.
birth_date	DATE	NULL	NULL	—	Ngày sinh khách hàng (phục vụ kiểm tra độ tuổi phim).
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	—	Thời điểm đăng ký tài khoản thành viên.

Bảng 10: employee — Nhân viên Rạp

Lưu thông tin nhân viên làm việc tại các chi nhánh của chuỗi rạp.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
employee_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY (pk_employee)	Mã định danh tự tăng của nhân viên.
branch_id	INT	NOT NULL	—	FK -> cinema_branch(branch_id)	Mã chi nhánh nơi nhân viên làm việc.
employee_code	VARCHAR(20)	NOT NULL	—	UNIQUE (uq_employee_code)	Mã nhân sự (VD: NV-HN01, KT-001).
full_name	VARCHAR(120)	NOT NULL	—	—	Họ và tên nhân viên.
phone	VARCHAR(20)	NOT NULL	—	UNIQUE (uq_employee_phone)	Số điện thoại liên hệ của nhân viên.
role	VARCHAR(20)	NOT NULL	'CASHIER'	CHECK (role IN ('CASHIER', 'MANAGER', 'ACCOUNTANT'))	Vai trò công việc (CASHIER: Thu ngân, MANAGER: Quản lý chi nhánh, ACCOUNTANT: Kế toán).
hired_date	DATE	NOT NULL	—	—	Ngày chính thức nhận việc.

Bảng 11: booking — Đơn đặt vé

Lưu thông tin chung của một lần giao dịch đặt vé xem phim (mỗi đơn thuộc duy nhất 1 suất chiếu).

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
booking_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY (pk_booking)	Mã định danh tự tăng của đơn đặt vé.
booking_code	VARCHAR(30)	NOT NULL	—	UNIQUE (uq_booking_code)	Mã đơn đặt vé in trên hóa đơn (VD: BK-20261101-0001).
screening_id	INT	NOT NULL	—	FK -> screening(screening_id)	Mã suất chiếu mà đơn đặt vé này mua chỗ.
customer_id	INT	NULL	NULL	FK -> customer(customer_id)	Mã khách hàng thành viên; nhận giá trị NULL khi là **Khách vãng lai mua trực tiếp tại quầy**.
employee_id	INT	NOT NULL	—	FK -> employee(employee_id)	Mã nhân viên thực hiện giao dịch đặt vé.
booked_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	—	Thời điểm khởi tạo đơn đặt vé.
booking_status	VARCHAR(20)	NOT NULL	'CONFIRMED'	CHECK (booking_status IN ('CONFIRMED', 'PENDING', 'CANCELLED'))	Trạng thái đơn (CONFIRMED: Đã xác nhận, PENDING: Chờ thanh toán, CANCELLED: Đã hủy).

Bảng 12: ticket — Vé xem phim chi tiết theo ghế (Chuẩn 3NF)

Lưu từng vé gắn với từng ghế cụ thể trong một đơn đặt vé, bao gồm cả lịch sử các vé đã hủy (CANCELLED). Suất chiếu của vé được suy diễn bảo toàn qua booking.screening_id (không lưu dư thừa screening_id để loại bỏ phụ thuộc hàm bắc cầu).

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
ticket_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY (pk_ticket)	Mã định danh tự tăng của tấm vé.
booking_id	INT	NOT NULL	—	FK -> booking(booking_id), UQ(booking_id, seat_id)	Mã đơn đặt vé chứa vé này.
seat_id	INT	NOT NULL	—	FK -> seat(seat_id), UQ(booking_id, seat_id)	Mã ghế vật lý được gán cho vé (Trigger đảm bảo seat.room_id = screening.room_id).
unit_price	DECIMAL(12,2)	NOT NULL	—	CHECK (unit_price >= 0)	Đơn giá bán thực tế chốt tại thời điểm phát hành vé (base_price + surcharge).
ticket_status	VARCHAR(20)	NOT NULL	'VALID'	CHECK (ticket_status IN ('VALID', 'CANCELLED'))	Trạng thái hiệu lực của vé (VALID: Hợp lệ, CANCELLED: Đã hủy).
cancelled_at	DATETIME	NULL	NULL	—	Thời điểm vé bị hủy (NULL nếu vé đang VALID).

Bảng 13: screening_seat_lock — Khóa phân bổ ghế hiệu lực theo suất chiếu (Chuẩn 3NF)

Lưu sự cấp phát ghế đang có hiệu lực (VALID) của một tấm vé trong một suất chiếu xác định. Được đồng bộ tự động qua Trigger khi ticket thêm mới, hủy (CANCELLED) hoặc xóa, đảm bảo chống bán trùng ghế ở mức khóa chính B-Tree InnoDB mà không làm bẩn chuẩn 3NF của ticket.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
screening_id	INT	NOT NULL	—	PK(screening_id, seat_id), FK -> screening(screening_id)	Mã suất chiếu đang có ghế được giữ hợp lệ.
seat_id	INT	NOT NULL	—	PK(screening_id, seat_id), FK -> seat(seat_id)	Mã ghế vật lý đang được giữ trong suất chiếu đó.
ticket_id	INT	NOT NULL	—	UNIQUE (uq_ssl_ticket), FK -> ticket(ticket_id)	Mã tấm vé đang ở trạng thái VALID nắm giữ khóa ghế này.

Bảng 14: payment — Giao dịch Thanh toán

Lưu lịch sử các lần thanh toán của đơn đặt vé.

Tên cột	Phân loại	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
payment_id	Thuộc tính logic	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY (pk_payment)	Mã định danh giao dịch thanh toán (khóa dự tuyển duy nhất của bảng).
booking_id	Thuộc tính logic	INT	NOT NULL	—	FK -> booking(booking_id)	Mã đơn đặt vé được thanh toán.
amount	Thuộc tính logic	DECIMAL(12,2)	NOT NULL	—	CHECK (amount >= 0)	Số tiền của giao dịch thanh toán (VNĐ).
payment_method	Thuộc tính logic	VARCHAR(20)	NOT NULL	'CASH'	CHECK (payment_method IN ('CASH', 'CARD', 'TRANSFER', 'EWALLET'))	Phương thức thanh toán (CASH, CARD, TRANSFER, EWALLET).
payment_status	Thuộc tính logic	VARCHAR(20)	NOT NULL	'SUCCESS'	CHECK (payment_status IN ('PENDING', 'SUCCESS', 'FAILED', 'REFUNDED'))	Trạng thái thanh toán (PENDING, SUCCESS, FAILED, REFUNDED).
paid_at	Thuộc tính logic	DATETIME	NULL	NULL	—	Thời điểm thanh toán hoàn tất (ghi nhận thời điểm tính doanh thu khi SUCCESS).
transaction_ref	Thuộc tính logic	VARCHAR(60)	NULL	NULL	—	Mã tham chiếu biên lai / ngân hàng / cổng thanh toán.
active_success_booking	Chỉ mục vật lý MySQL 8.4	INT	NULL	Generated	GENERATED ALWAYS AS (CASE WHEN payment_status = 'SUCCESS' THEN booking_id ELSE NULL END) STORED, UNIQUE (uq_payment_one_success)	Cột biểu thức vật lý phục vụ chỉ mục duy nhất có điều kiện trên tập con payment_status = 'SUCCESS' (không phải thuộc tính logic hay khóa dự tuyển của toàn bảng).

3. Tổng hợp 17 Ràng buộc Tham chiếu Khóa ngoại (01b_foreign_keys.sql)

STT	Tên Ràng buộc FK	Bảng Con (Child Table)	Cột FK	Bảng Cha (Parent Table)	Cột PK Tham chiếu	Quy tắc ON DELETE / ON UPDATE
1	fk_room_branch	screening_room	branch_id	cinema_branch	branch_id	RESTRICT / CASCADE
2	fk_seat_room	seat	room_id	screening_room	room_id	RESTRICT / CASCADE

STT	Tên Ràng buộc FK	Bảng Con (Child Table)	Cột FK	Bảng Cha (Parent Table)	Cột PK Tham chiếu	Quy tắc ON DELETE / ON UPDATE
3	fk_seat_type	seat	seat_type_code	seat_type	seat_type_code	RESTRICT / CASCADE
4	fk_mg_movie	movie_genre	movie_id	movie	movie_id	CASCADE / CASCADE
5	fk_mg_genre	movie_genre	genre_id	genre	genre_id	RESTRICT / CASCADE
6	fk_screening_movie	screening	movie_id	movie	movie_id	RESTRICT / CASCADE
7	fk_screening_room	screening	room_id	screening_room	room_id	RESTRICT / CASCADE
8	fk_employee_branch	employee	branch_id	cinema_branch	branch_id	RESTRICT / CASCADE
9	fk_booking_screening	booking	screening_id	screening	screening_id	RESTRICT / CASCADE
10	fk_booking_customer	booking	customer_id	customer	customer_id	SET NULL / CASCADE
11	fk_booking_employee	booking	employee_id	employee	employee_id	RESTRICT / CASCADE
12	fk_ticket_booking	ticket	booking_id	booking	booking_id	RESTRICT / CASCADE
13	fk_ticket_seat	ticket	seat_id	seat	seat_id	RESTRICT / CASCADE
14	fk_ssl_screening	screening_seat_lock	screening_id	screening	screening_id	RESTRICT / CASCADE
15	fk_ssl_seat	screening_seat_lock	seat_id	seat	seat_id	RESTRICT / CASCADE
16	fk_ssl_ticket	screening_seat_lock	ticket_id	ticket	ticket_id	CASCADE / CASCADE
17	fk_payment_booking	payment	booking_id	booking	booking_id	RESTRICT / RESTRICT

CHƯƠNG 4. PHỤ THUỘC HÀM, PHÉP PHÂN RÃ VÀ CHỨNG MINH CHUẨN HÓA 3NF (TIÊU CHÍ R4)

(Tài liệu đáp ứng Tiêu chí R4 — 1.0 điểm)

1. Cơ sở Lý thuyết Chuẩn hóa Quan hệ & Phân biệt Khái niệm Nghiêm ngặt

Cho lược đồ quan hệ R(U, F) với tập thuộc tính U và tập phụ thuộc hàm F xác định trên U:

• **Siêu khóa (Superkey):** Tập thuộc tính X ⊆ U sao cho bao đóng thuộc tính X^+_F = U trên **mọi thể hiện hợp lệ (all valid tuples)** của quan hệ R, và không có thuộc tính nào trong X mang giá trị bất định (`NULL`) làm mất tính định danh bộ.

• **Khóa dự tuyển tối thiểu trên toàn quan hệ (Minimal Candidate Key – K):** Siêu khóa K ⊆ U sao cho không tồn tại tập con thực sự K' ⊂ K nào có (K')^+_F = U. Một khóa dự tuyển được chọn làm **Khóa chính (Primary Key – PK)**, các khóa dự tuyển còn lại trên toàn quan hệ được gọi là **Khóa dự tuyển thay thế (Alternate Keys – AK)**.

• **Thuộc tính khóa (Prime Attribute):** Thuộc tính thuộc về ít nhất một khóa dự tuyển K của R.

• **Thuộc tính không khóa (Non-prime Attribute):** Thuộc tính không thuộc bất kỳ khóa dự tuyển nào của R.

• **Dạng chuẩn 1 (1NF):** Mọi miền giá trị của thuộc tính trong U đều nguyên tố (atomic), không chứa thuộc tính đa trị, danh sách gộp hay nhóm lặp.

• **Dạng chuẩn 2 (2NF):** R đạt 1NF và mọi thuộc tính không khóa A đều phụ thuộc hàm **đầy đủ** vào mọi khóa dự tuyển K của R (tức là không tồn tại khóa dự tuyển phức hợp K, tập con thực sự X ⊂ K (X ≠ ∅) và thuộc tính không khóa A sao cho X → A ∈ F^+).

• **Dạng chuẩn 3 (3NF):** R đạt 1NF và với mọi phụ thuộc hàm không tầm thường X → A ∈ F^+ (A ∉ X), phải thỏa mãn ít nhất một trong hai điều kiện:

1. X là một **siêu khóa** của R, **hoặc**

2. A là một **thuộc tính khóa (Prime attribute)** của R.

(Hệ quả tương đương: Không tồn tại thuộc tính không khóa A phụ thuộc bắc cầu vào khóa dự tuyển K qua một tập không phải siêu khóa Y: K → Y và Y → A với Y ↛ K, A ∉ Y).

• **Dạng chuẩn Boyce–Codd (BCNF):** Mọi phụ thuộc hàm không tầm thường X → A ∈ F^+ đều có X là siêu khóa của R (BCNF ⇒ 3NF ⇒ 2NF ⇒ 1NF).

Hai Phân biệt Lý thuyết Bắt buộc (Tránh Nhầm lẫn Học thuật)

1. **Khóa dự tuyển toàn bảng (Candidate Key) ≠ Ràng buộc duy nhất trên tập con (Partial / Subset Uniqueness Constraint):**

• Một tập thuộc tính chỉ được gọi là Candidate Key hoặc Alternate Key của quan hệ R nếu nó định danh duy nhất **mọi bộ** trong R và không nhận giá trị `NULL`.

• Ngược lại, quy tắc "mỗi `booking_id` chỉ có tối đa một dòng `payment` mang trạng thái `SUCCESS`" chỉ áp dụng trên tập con σ_{payment_status='SUCCESS'}(payment), còn các dòng `PENDING`, `FAILED`, `REFUNDED` có thể lặp lại cùng `booking_id` (và biểu thức `active_success_booking` nhận `NULL` trên các dòng đó). Vì vậy, `booking_id` hay `active_success_booking` **tuyệt đối không phải là khóa dự tuyển / khóa thay thế** của quan hệ `payment`.

2. **Mô hình Quan hệ Mức Logic ≠ Cột Chỉ mục Sinh Mức Vật lý (GENERATED ALWAYS AS ... STORED):**

• Việc một cột được khai báo `GENERATED` hay được bảo vệ bởi Trigger chống sửa sai **không làm biến mất phụ thuộc hàm** giữa các thuộc tính. Chuẩn hóa 1NF/2NF/3NF là tính chất toán học của tập thuộc tính U và tập phụ thuộc hàm F, không phụ thuộc vào việc ứng dụng hay Trigger có chặn lệnh `UPDATE` hay không.

2. Phát hiện Vi phạm 3NF Thực sự trong Thiết kế Cũ của Bảng `ticket` và Phép Phân rã Chuẩn 3NF

2.1. Chẩn đoán Vi phạm 3NF ở Thiết kế Cũ (`ticket` 13 bảng trước đây)

Trong thiết kế 13 bảng ban đầu, bảng `ticket` có lược đồ:

U_{old} = {ticket_id, booking_id, screening_id, seat_id, unit_price, ticket_status, cancelled_at}

Xét các quy tắc nghiệp vụ bất biến:

• Mỗi đơn đặt vé (`booking`) được lập cho **đúng một** suất chiếu xác định: booking_id → screening_id.

• Mỗi vé thuộc về một đơn đặt vé xác định: ticket_id → booking_id, và trong cùng một đơn đặt vé không thể xuất hiện hai dòng vé cho cùng một ghế: {booking_id, seat_id} → ticket_id.

• Khóa dự tuyển của U_{old} là K_1 = {ticket_id} và K_2 = {booking_id, seat_id}. Khi đó tập thuộc tính khóa là {ticket_id, booking_id, seat_id}; thuộc tính `screening_id` là **thuộc tính không khóa (Non-prime attribute)**.

• Xét phụ thuộc hàm không tầm thường:

booking_id → screening_id

• Vế trái `booking_id` **không phải siêu khóa** của `ticket` (vì một đơn đặt vé mua nhiều ghế khác nhau, (booking_id)^+ ≠ U_{old}).

• Vế phải `screening_id` **không phải thuộc tính khóa** (không nằm trong K_1 hay K_2).

• **Kết luận nghiêm ngặt:** Sự hiện diện của cột `screening_id` trong `ticket` tạo ra phụ thuộc hàm bộ phận vào K_2 ({booking_id} ⊂ {booking_id, seat_id}, **vi phạm 2NF**) đồng thời là phụ thuộc hàm bắc cầu từ khóa chính K_1 (ticket_id → booking_id → screening_id, **vi phạm 3NF**). Việc dùng khóa ngoại tổ hợp `FOREIGN KEY (booking_id, screening_id)` hay Trigger đồng bộ ở thiết kế cũ chỉ ngăn dữ liệu mâu thuẫn khi ghi, **không hề xóa bỏ phụ thuộc hàm booking_id → screening_id** nên quan hệ cũ không đạt 3NF về mặt lý thuyết!

2.2. So sánh Các Phương án Thiết kế Chuẩn hóa 3NF cho Nghiệp vụ Vé & Chống Bán Trùng Ghế

Tiêu chí Đánh giá	**Phương án A: Chỉ bỏ `ticket.screening_id`, kiểm tra trùng ghế bằng `SELECT` trong Java / Trigger**	**Phương án B (ĐƯỢC CHỌN): Bỏ ticket.screening_id + Tách quan hệ chuẩn 3NF screening_seat_lock(screening_id, seat_id, ticket_id)**	**Phương án C: Gộp `booking` và `ticket` thành 1 dòng mỗi ghế**
Đạt chuẩn 3NF / BCNF?	Đạt 3NF trên `ticket`.	**Đạt 3NF và BCNF** trên cả `ticket` lẫn `screening_seat_lock`.	Phá vỡ nghiệp vụ 1 hóa đơn nhiều ghế hoặc lặp lại `customer_id, employee_id, booked_at` theo `booking_code` (gây vi phạm 3NF mới).
Chống bán trùng ghế ở mức chỉ mục phần cứng B-Tree InnoDB?	**Không có UNIQUE Index trực tiếp** trên `(screening_id, seat_id)`; nếu chạy dưới `READ COMMITTED` hoặc ghi SQL trực tiếp ngoài ứng dụng dễ hở race condition.	**Có khóa chính vật lý PRIMARY KEY (screening_id, seat_id)** và `UNIQUE (ticket_id)` trực tiếp trong InnoDB — bất khả xâm phạm ngay cả khi 20 luồng giao dịch chạy đồng thời hoặc ghi SQL trực tiếp.	Có `UNIQUE`, nhưng đánh mất thực thể `booking` nhiều ghế.

Tiêu chí Đánh giá	Phương án A: Chỉ bỏ `ticket.screening_id`, kiểm tra trùng ghế bằng `SELECT` trong Java / Trigger	Phương án B (ĐƯỢC CHỌN): Bỏ ticket.screening_id + Tách quan hệ chuẩn 3NF screening_seat_lock(screening_id, seat_id, ticket_id)	Phương án C: Gộp `booking` và `ticket` thành 1 dòng mỗi ghế
Lưu lịch sử vé hủy (CANCELLED) & bán lại ghế?	Lưu được lịch sử trong `ticket`.	**Lưu trọn vẹn lịch sử** mọi lần hủy trong `ticket`, đồng thời tự động giải phóng dòng trong `screening_seat_lock` khi hủy để bán lại ghế ngay lập tức.	Phức tạp khi hủy toàn đơn nhiều ghế và đối chiếu `payment`.

2.3. Chứng minh Kết nối Không Mất Thông tin (Lossless-Join) và Bảo toàn Phụ thuộc / Ràng buộc Liên Quan hệ

1. **Chứng minh Kết nối Không Mất Thông tin (Lossless-Join Decomposition):**

• Xét phép chiếu bỏ cột dư thừa `screening_id` khỏi U_{old} sang hai lược đồ đã tồn tại trong CSDL:

• R_{ticket}(ticket_id, booking_id, seat_id, unit_price, ticket_status, cancelled_at)

• R_{booking}(booking_id, booking_code, screening_id, customer_id, employee_id, booked_at, booking_status)

• Phần giao thuộc tính giữa hai lược đồ là:

R_{ticket} ∩ R_{booking} = {booking_id}

• Vì booking_id là khóa chính (siêu khóa) của R_{booking} nên booking_id → R_{booking} ∈ F^+ (và khóa ngoại `fk_ticket_booking` kèm `booking_id NOT NULL` đảm bảo mọi bộ trong `ticket` đều khớp đúng 1 bộ trong `booking`).

• Theo **Định lý Heath**, phép phân rã bảo toàn thông tin tuyệt đối (**Lossless-Join**): mọi thông tin của bảng `ticket` cũ được khôi phục chính xác 1–1 qua phép kết nối tự nhiên:

π_{ticket_id, booking_id, screening_id, seat_id, unit_price, ticket_status, cancelled_at}(R_{ticket} ⋈ R_{booking}) = r_{old}

• (Thực nghiệm kiểm chứng trên bản sao CSDL thật trong `verification/logs/migration_clone_dryrun.log` cho thấy mã băm SHA-256 trên toàn bộ 373 dòng vé trước và sau khi tách khớp tuyệt đối `ef904b39c8e1b57b812d0b01b1ee76f3c4992c6b0be7f146cc150fb2e0ba3107`).

2. **Chứng minh Bảo toàn Phụ thuộc Hàm (Dependency Preservation) & Cưỡng chế Ràng buộc Liên Quan hệ:**

• Phụ thuộc hàm booking_id → screening_id được **bảo toàn trực tiếp trong quan hệ booking** (nơi `booking_id` là khóa chính).

• Các phụ thuộc hàm của vé (ticket_id → {booking_id, seat_id, unit_price, ticket_status, cancelled_at} và {booking_id, seat_id} → ticket_id) được **bảo toàn trực tiếp trong quan hệ ticket**.

• Các phụ thuộc hàm của khóa ghế hiệu lực ({screening_id, seat_id} → ticket_id và ticket_id → {screening_id, seat_id}) được **bảo toàn trực tiếp trong quan hệ screening_seat_lock** qua `PRIMARY KEY (screening_id, seat_id)` và `UNIQUE (ticket_id)`.

• Hợp của các tập phụ thuộc hàm cục bộ (F_{booking} ∪ F_{ticket} ∪ F_{ssl})^+ suy diễn đầy đủ F^+, tức là phép thiết kế **bảo toàn phụ thuộc hàm (Dependency-Preserving)**.

• Đối với **các ràng buộc toàn vẹn liên quan hệ (Inter-relational Tuple Constraints)** — cụ thể là: (i) `seat.room_id = screening.room_id` (`BR-09`), và (ii) r_{ssl} = π_{screening_id, seat_id, ticket_id}(σ_{ticket_status='VALID'}(ticket ⋈ booking)) (`BR-10`) — trong SQL chuẩn (khi chưa có `CREATE ASSERTION`), các ràng buộc liên bảng này được cưỡng chế đồng bộ 100% tại tầng DBMS bằng

FOREIGN KEY kết hợp bộ Trigger trg_ticket_validate_ins/upd, trg_ticket_after_ins/upd/del, trg_booking_before_upd và trg_ssl_guard_ins/upd/del trong database/schema/02_triggers.sql.

3. Các Quyết định Thiết kế Loại bỏ Dư thừa Dữ liệu Khác

1. **Không lưu capacity (sức chứa) trong screening_room:** Sức chứa bằng tổng số ghế thuộc phòng trong bảng seat, tránh dị thường cập nhật khi cấu hình lại sơ đồ ghế.

2. **Không lưu total_amount trong booking:** Tổng tiền đơn vé được tính từ SUM(ticket.unit_price) của các vé hợp lệ thuộc đơn.

3. **Không lưu room_id trong ticket hoặc booking:** Nếu lưu room_id sẽ sinh phụ thuộc bắc cầu qua screening_id -> room_id và seat_id -> room_id.

4. **Bản chất lịch sử của ticket.unit_price (BR-12):** Tại thời điểm xuất vé, giá bán được tính bằng screening.base_price + seat_type.surcharge hiện hành và chốt cố định vào ticket.unit_price. Khi rạp điều chỉnh bảng giá phụ thu seat_type.surcharge hoặc giá cơ sở cho đợt mới, đơn giá trên các vé lịch sử đã bán không được thay đổi. Do đó trên quan hệ lịch sử ticket, unit_price không phụ thuộc hàm vào giá hiện hành của seat_type hay screening, mà phụ thuộc đầy đủ vào khóa của ticket.

5. **Bản chất vận hành của screening.end_time (BR-05):** Thời điểm kết thúc suất chiếu bao gồm thời lượng phim cộng thời gian chiếu trailer quảng cáo và dọn vệ sinh phòng linh hoạt (15–30 phút tùy khung giờ thường hay suất IMAX), nên end_time không phải hàm tất định thuần túy của (movie_id, start_time) mà chỉ chịu ràng buộc miền giá trị end_time ≥ start_time + duration_minutes.

4. Phân tích Phụ thuộc Hàm, Tìm Khóa Tối thiểu và Chứng minh 1NF, 2NF, 3NF cho Đủ 14 Quan hệ

4.1. Quan hệ 1: cinema_branch

• **Tập thuộc tính:** U_1 = {branch_id, branch_code, branch_name, address, city, phone, active}

• **Tập phụ thuộc hàm cơ sở F_1:**

• f_{1.1}: branch_id → {branch_code, branch_name, address, city, phone, active}

• f_{1.2}: branch_code → {branch_id, branch_name, address, city, phone, active}

• f_{1.3}: branch_name → {branch_id, branch_code, address, city, phone, active}

• f_{1.4}: phone → {branch_id, branch_code, branch_name, address, city, active}

• **Bao đóng & Khóa dự tuyển tối thiểu:** (branch_id)^+ = (branch_code)^+ = (branch_name)^+ = (phone)^+ = U_1 (mọi cột này đều NOT NULL).

• Khóa dự tuyển: K_{1.1} = {branch_id} (**PK**), K_{1.2} = {branch_code} (**AK**), K_{1.3} = {branch_name} (**AK**), K_{1.4} = {phone} (**AK**).

• Thuộc tính khóa: {branch_id, branch_code, branch_name, phone}; Thuộc tính không khóa: {address, city, active} (nhiều chi nhánh có thể cùng một city nên city không xác định chi nhánh).

• **Chứng minh dạng chuẩn:**

• **1NF:** Mọi thuộc tính đều mang giá trị đơn nguyên tố.

• **2NF:** Mọi khóa dự tuyển đều gồm đúng 1 thuộc tính nên không thể tồn tại tập con thực sự khác rỗng ⇒ Không có phụ thuộc hàm bộ phận ⇒ Đạt **2NF**.

• **3NF & BCNF:** Mọi phụ thuộc hàm không tầm thường trong F_1^+ đều có vế trái chứa ít nhất một trong {K_{1.1}, K_{1.2}, K_{1.3}, K_{1.4}} (tức là siêu khóa) ⇒ Đạt **3NF** (và **BCNF**).

4.2. Quan hệ 2: screening_room

• **Tập thuộc tính:** U_2 = {room_id, branch_id, room_name, room_type, active}

• **Tập phụ thuộc hàm cơ sở F_2:**

• f_{2.1}: room_id → {branch_id, room_name, room_type, active}

• f_{2.2}: {branch_id, room_name} → {room_id, room_type, active}

• **Bao đóng & Khóa dự tuyển tối thiểu:**

• (room_id)^+ = U_2.

• ({branch_id})^+ = {branch_id} ≠ U_2 (một chi nhánh có nhiều phòng); ({room_name})^+ = {room_name} ≠ U_2 (nhiều chi nhánh cùng đặt tên `Phòng 01`); ({branch_id, room_name})^+ = U_2.

• Khóa dự tuyển: K_{2.1} = {room_id} (**PK**), K_{2.2} = {branch_id, room_name} (**AK**).

• Thuộc tính khóa: {room_id, branch_id, room_name}; Thuộc tính không khóa: {room_type, active}.

• **Chứng minh dạng chuẩn:**

• **1NF:** Mọi thuộc tính nguyên tố, không lưu danh sách ghế.

• **2NF:** Xét khóa phức hợp K_{2.2} = {branch_id, room_name}: cả ({branch_id})^+ lẫn ({room_name})^+ đều không chứa `room_type` hay `active`, nên mọi thuộc tính không khóa phụ thuộc hàm đầy đủ vào mọi khóa dự tuyển ⇒ Đạt **2NF**.

• **3NF & BCNF:** Mọi FD không tầm thường trong F_2^+ đều có vế trái là siêu khóa (K_{2.1} hoặc K_{2.2}) ⇒ Đạt **3NF** (và **BCNF**).

4.3. Quan hệ 3: `seat_type`

• **Tập thuộc tính:** U_3 = {seat_type_code, type_name, surcharge}

• **Tập phụ thuộc hàm cơ sở F_3:**

• f_{3.1}: seat_type_code → {type_name, surcharge}

• f_{3.2}: type_name → {seat_type_code, surcharge}

• **Khóa dự tuyển tối thiểu:** K_{3.1} = {seat_type_code} (**PK**), K_{3.2} = {type_name} (**AK**). Thuộc tính không khóa: {surcharge}.

• **Chứng minh dạng chuẩn:** Mọi thuộc tính nguyên tố (**1NF**); cả hai khóa đều đơn thuộc tính (**2NF**); mọi FD không tầm thường đều có vế trái là siêu khóa (`surcharge` không xác định ngược hạng ghế) ⇒ Đạt **3NF** (và **BCNF**).

4.4. Quan hệ 4: `seat`

• **Tập thuộc tính:** U_4 = {seat_id, room_id, seat_row, seat_number, seat_type_code}

• **Tập phụ thuộc hàm cơ sở F_4:**

• f_{4.1}: seat_id → {room_id, seat_row, seat_number, seat_type_code}

• f_{4.2}: {room_id, seat_row, seat_number} → {seat_id, seat_type_code}

• **Bao đóng & Khóa dự tuyển tối thiểu:**

• (seat_id)^+ = U_4 và ({room_id, seat_row, seat_number})^+ = U_4.

• Không tập con thực sự nào của {room_id, seat_row, seat_number} suy ra được `seat_id` hoặc `seat_type_code` (vì sơ đồ phân hạng ghế có thể bố trí ghế VIP ở vùng giữa hàng — ví dụ ghế 3..8 là VIP còn ghế 1..2 cùng hàng là Thường — và khác nhau giữa các phòng chiếu).

• Khóa dự tuyển: K_{4.1} = {seat_id} (**PK**), K_{4.2} = {room_id, seat_row, seat_number} (**AK**).

• Thuộc tính khóa: {seat_id, room_id, seat_row, seat_number}; Thuộc tính không khóa: {seat_type_code}.

• **Chứng minh dạng chuẩn:**

• **1NF:** Giá trị nguyên tố.

• **2NF:** `seat_type_code` phụ thuộc hàm đầy đủ vào cả K_{4.1} và K_{4.2} ⇒ Đạt **2NF**.

• **3NF & BCNF:** Không lưu `type_name` hay `surcharge` trong `seat`; mọi FD không tầm thường trong F_4^+ đều có vế trái là siêu khóa ⇒ Đạt **3NF** (và **BCNF**).

4.5. Quan hệ 5: `movie`

• **Tập thuộc tính:** U_5 = {movie_id, movie_code, title, duration_minutes, release_date, age_rating, country, description}

• **Tập phụ thuộc hàm cơ sở F_5:**

• f_{5.1}: movie_id → U_5 \ {movie_id}

• f_{5.2}: movie_code → U_5 \ {movie_code}

• **Khóa dự tuyển tối thiểu:** K_{5.1} = {movie_id} (**PK**), K_{5.2} = {movie_code} (**AK**). (Hai bộ phim sản xuất khác năm/quốc gia có thể trùng `title` nên `title` không phải khóa dự tuyển).

• **Chứng minh dạng chuẩn:** Thuộc tính đa trị thể loại đã tách sang `movie_genre` (**1NF**); các khóa dự tuyển đều gồm 1 thuộc tính (**2NF**); không tồn tại phụ thuộc hàm giữa các thuộc tính không khóa (`title`, `duration_minutes`, `release_date`, `age_rating`, `country`, `description`), mọi FD không tầm thường đều có vế trái là siêu khóa ⇒ Đạt **3NF** (và **BCNF**).

4.6. Quan hệ 6: `genre`

• **Tập thuộc tính:** U_6 = {genre_id, genre_name, description}

• **Tập phụ thuộc hàm cơ sở F_6:**

• f_{6.1}: genre_id → {genre_name, description}

• f_{6.2}: genre_name → {genre_id, description}

• **Khóa dự tuyển tối thiểu:** K_{6.1} = {genre_id} (**PK**), K_{6.2} = {genre_name} (**AK**). Thuộc tính không khóa: {description}.

• **Chứng minh dạng chuẩn:** Thuộc tính nguyên tố (**1NF**); khóa đơn thuộc tính (**2NF**); mọi FD không tầm thường có vế trái là siêu khóa ⇒ Đạt **3NF** (và **BCNF**).

4.7. Quan hệ 7: `movie_genre`

• **Tập thuộc tính:** U_7 = {movie_id, genre_id}

• **Tập phụ thuộc hàm cơ sở F_7:** ∅ (chỉ có các phụ thuộc hàm tầm thường X → Y với Y ⊆ X; một phim có nhiều thể loại nên movie_id ↛ genre_id, một thể loại có nhiều phim nên genre_id ↛ movie_id).

• **Khóa dự tuyển tối thiểu:** Duy nhất K_7 = {movie_id, genre_id} (**PK**). Toàn bộ thuộc tính đều là thuộc tính khóa (U_7 \ K_7 = ∅).

• **Chứng minh dạng chuẩn:** Mỗi dòng chứa cặp mã nguyên tố (**1NF**); không tồn tại bất kỳ phụ thuộc hàm không tầm thường nào ⇒ Hiển nhiên đạt **2NF, 3NF và BCNF**.

4.8. Quan hệ 8: `screening`

• **Tập thuộc tính:** U_8 = {screening_id, movie_id, room_id, start_time, end_time, base_price, status}

• **Tập phụ thuộc hàm cơ sở F_8:**

• f_{8.1}: screening_id → {movie_id, room_id, start_time, end_time, base_price, status}

• **Bao đóng & Khóa dự tuyển tối thiểu:**

• Duy nhất K_8 = {screening_id} (**PK**).

• Vì sao {room_id, start_time} không phải khóa dự tuyển trên toàn bảng? Trên lịch sử vận hành, một suất chiếu ở phòng r lúc 19:00 có thể bị hủy (status = 'CANCELLED') do sự cố kỹ thuật rồi được lập lại một suất chiếu mới status = 'OPEN' chiếu phim khác tại cùng phòng r lúc 19:00. Quy tắc không chồng lấn thời gian chỉ áp dụng trên tập con σ_{status='OPEN'}(screening), không phải phụ thuộc hàm toàn bảng.

• Như đã phân tích tại Mục 3, end_time bao gồm thời gian trailer/vệ sinh linh hoạt và base_price thay đổi theo khung giờ vàng/ngày lễ nên không tồn tại FD {movie_id, start_time} → end_time hay {movie_id, room_id} → base_price.

• **Chứng minh dạng chuẩn:** Thuộc tính nguyên tố (**1NF**); khóa đơn thuộc tính screening_id (**2NF**); FD không tầm thường duy nhất f_{8.1} có vế trái là siêu khóa K_8 ⇒ Đạt **3NF** (và **BCNF**).

4.9. Quan hệ 9: customer

• **Tập thuộc tính:** U_9 = {customer_id, full_name, phone, email, birth_date, created_at}

• **Tập phụ thuộc hàm cơ sở F_9:**

• f_{9.1}: customer_id → {full_name, phone, email, birth_date, created_at}

• f_{9.2}: phone → {customer_id, full_name, email, birth_date, created_at}

• f_{9.3}: email → {customer_id, full_name, phone, birth_date, created_at}

• **Khóa dự tuyển tối thiểu:** Cả customer_id, phone, email đều NOT NULL và duy nhất trên toàn bảng ⇒ K_{9.1} = {customer_id} (**PK**), K_{9.2} = {phone} (**AK**), K_{9.3} = {email} (**AK**). Thuộc tính không khóa: {full_name, birth_date, created_at}.

• **Chứng minh dạng chuẩn:** Mọi thuộc tính nguyên tố (**1NF**); mọi khóa dự tuyển đều đơn thuộc tính (**2NF**); mọi FD không tầm thường đều có vế trái là siêu khóa ⇒ Đạt **3NF** (và **BCNF**).

4.10. Quan hệ 10: employee

• **Tập thuộc tính:** U_{10} = {employee_id, branch_id, employee_code, full_name, phone, role, hired_date}

• **Tập phụ thuộc hàm cơ sở F_{10}:**

• f_{10.1}: employee_id → {branch_id, employee_code, full_name, phone, role, hired_date}

• f_{10.2}: employee_code → {employee_id, branch_id, full_name, phone, role, hired_date}

• f_{10.3}: phone → {employee_id, branch_id, employee_code, full_name, role, hired_date}

• **Khóa dự tuyển tối thiểu:** K_{10.1} = {employee_id} (**PK**), K_{10.2} = {employee_code} (**AK**), K_{10.3} = {phone} (**AK**). Thuộc tính không khóa: {branch_id, full_name, role, hired_date}.

• **Chứng minh dạng chuẩn:** Không lưu tên hay thành phố của chi nhánh trong employee; mọi thuộc tính nguyên tố (**1NF**); mọi khóa đơn thuộc tính (**2NF**); mọi FD không tầm thường đều xuất phát từ siêu khóa ⇒ Đạt **3NF** (và **BCNF**).

4.11. Quan hệ 11: booking

• **Tập thuộc tính:** U_{11} = {booking_id, booking_code, screening_id, customer_id, employee_id, booked_at, booking_status}

• **Tập phụ thuộc hàm cơ sở F_{11}:**

• f_{11.1}: booking_id → {booking_code, screening_id, customer_id, employee_id, booked_at, booking_status}

• f_{11.2}: booking_code → {booking_id, screening_id, customer_id, employee_id, booked_at, booking_status}

• **Khóa dự tuyển tối thiểu:** K_{11.1} = {booking_id} (**PK**), K_{11.2} = {booking_code} (**AK**). Thuộc tính không khóa: {screening_id, customer_id, employee_id, booked_at, booking_status}.

• **Chứng minh dạng chuẩn:** Danh sách ghế/vé đã tách sang `ticket` (**1NF**); cả hai khóa đều đơn thuộc tính (**2NF**); một nhân viên phục vụ nhiều chi nhánh/ca hoặc một khách hàng đặt nhiều suất nên không có phụ thuộc hàm giữa `customer_id`, `employee_id`, `screening_id`, `booked_at`, `booking_status`; mọi FD không tầm thường đều có vế trái là siêu khóa ⇒ Đạt **3NF** (và **BCNF**).

4.12. Quan hệ 12: `ticket` (Sau khi Chuẩn hóa 3NF — Loại bỏ `screening_id`)

• **Tập thuộc tính:** U_{12} = {ticket_id, booking_id, seat_id, unit_price, ticket_status, cancelled_at}

• **Tập phụ thuộc hàm cơ sở F_{12}:**

• f_{12.1}: ticket_id → {booking_id, seat_id, unit_price, ticket_status, cancelled_at}

• f_{12.2}: {booking_id, seat_id} → {ticket_id, unit_price, ticket_status, cancelled_at}

• **Bao đóng & Khóa dự tuyển tối thiểu:**

• (ticket_id)^+ = U_{12} và ({booking_id, seat_id})^+ = U_{12} (cả `booking_id` và `seat_id` đều `NOT NULL` kèm `UNIQUE uq_ticket_booking_seat`).

• Kiểm tra tính tối thiểu của K_{12.2} = {booking_id, seat_id}:

• ({booking_id})^+ = {booking_id}: Một đơn đặt vé mua nhiều ghế khác hạng (`STD`, `VIP`, `COUPLE`) nên chỉ riêng `booking_id` **không** xác định được `seat_id` hay `unit_price`.

• ({seat_id})^+ = {seat_id}: Cùng một ghế vật lý được bán qua hàng trăm suất chiếu/đơn đặt vé khác nhau với `unit_price` và `ticket_status` khác nhau nên chỉ riêng `seat_id` **không** xác định được bất kỳ thuộc tính nào khác.

• Vậy `ticket` có đúng 2 khóa dự tuyển tối thiểu trên toàn bảng: K_{12.1} = {ticket_id} (**PK**) và K_{12.2} = {booking_id, seat_id} (**AK**).

• Thuộc tính khóa: {ticket_id, booking_id, seat_id}; Thuộc tính không khóa: {unit_price, ticket_status, cancelled_at}.

• **Chứng minh dạng chuẩn:**

• **1NF:** Mọi thuộc tính đều nguyên tố.

• **2NF:** Xét khóa phức hợp K_{12.2} = {booking_id, seat_id}: không tồn tại phụ thuộc hàm nào từ tập con thực sự {booking_id} hay {seat_id} sang {unit_price, ticket_status, cancelled_at} (vì `screening_id` đã được loại bỏ hoàn toàn khỏi `ticket`!). Mọi thuộc tính không khóa đều phụ thuộc hàm đầy đủ vào cả K_{12.1} và K_{12.2} ⇒ Đạt **2NF**.

• **3NF & BCNF:** Giữa các thuộc tính không khóa (`unit_price`, `ticket_status`, `cancelled_at`) không có phụ thuộc hàm không tầm thường nào (lưu ý: quy tắc "nếu `ticket_status = 'VALID'` thì `cancelled_at IS NULL`" là ràng buộc miền giá trị có điều kiện cho 1 giá trị trạng thái, còn khi `ticket_status = 'CANCELLED'` thì `cancelled_at` nhận vô số thời điểm khác nhau nên `ticket_status` ↛ `cancelled_at` và `cancelled_at` ↛ `ticket_status` do `NULL` không xác định hàm). Mọi FD không tầm thường trong F_{12}^+ đều có vế trái là siêu khóa (K_{12.1} hoặc K_{12.2}) ⇒ Đạt **3NF** (và **BCNF**).

4.13. Quan hệ 13: `screening_seat_lock` (Quan hệ Khóa Ghế Hiệu lực)

• **Tập thuộc tính:** U_{13} = {screening_id, seat_id, ticket_id}

• **Tập phụ thuộc hàm cơ sở F_{13}:**

• f_{13.1}: {screening_id, seat_id} → ticket_id (tại mọi thời điểm, một ghế trong một suất chiếu chỉ được nắm giữ bởi tối đa 01 tấm vé hợp lệ)

• f_{13.2}: ticket_id → {screening_id, seat_id} (mỗi tấm vé hợp lệ chỉ giữ khóa cho đúng 01 ghế của đúng 01 suất chiếu)

• **Bao đóng & Khóa dự tuyển tối thiểu:**

• ({screening_id, seat_id})^+ = U_{13} (không tập con thực sự nào suy ra `ticket_id` vì 1 suất chiếu có nhiều ghế được bán và 1 ghế được bán ở nhiều suất chiếu).

• (ticket_id)^+ = U_{13} (`ticket_id NOT NULL UNIQUE`).

• Khóa dự tuyển tối thiểu trên toàn bảng: K_{13.1} = {screening_id, seat_id} (**PK**) và K_{13.2} = {ticket_id} (**AK**).

• Thuộc tính khóa: {screening_id, seat_id, ticket_id} (toàn bộ U_{13}); Thuộc tính không khóa: ∅.

• **Chứng minh dạng chuẩn:**

• **1NF:** Mọi thuộc tính nguyên tố.

• **2NF:** Không có thuộc tính không khóa (U_{13} \ (K_{13.1} ∪ K_{13.2}) = ∅) ⇒ Không thể có phụ thuộc bộ phận ⇒ Đạt **2NF**.

• **3NF & BCNF:** Cả hai phụ thuộc hàm không tầm thường f_{13.1} và f_{13.2} đều có vế trái là khóa dự tuyển (siêu khóa) ⇒ Đạt **3NF** (và **BCNF**).

4.14. Quan hệ 14: `payment`

• **Tập thuộc tính mức logic:** U_{14} = {payment_id, booking_id, amount, payment_method, payment_status, paid_at, transaction_ref}

• **Tập phụ thuộc hàm cơ sở F_{14}:**

• f_{14.1}: payment_id → {booking_id, amount, payment_method, payment_status, paid_at, transaction_ref}

• **Bao đóng & Khóa dự tuyển tối thiểu:**

• Duy nhất K_{14} = {payment_id} (**PK**).

• Kiểm tra các cột khác:

• `booking_id` **không phải khóa dự tuyển**: một đơn đặt vé có thể có dòng 1 `FAILED` (thẻ lỗi) và dòng 2 `SUCCESS` (tiền mặt), nên `booking_id` lặp lại trên nhiều dòng của `payment`.

• `transaction_ref` cho phép `NULL` (các giao dịch tiền mặt `CASH` hoặc đang `PENDING` chưa có mã tham chiếu ngân hàng) nên không phải khóa dự tuyển trên toàn bảng.

• Cột vật lý `active_success_booking` trong MySQL 8.4 nhận `NULL` trên mọi dòng khác `SUCCESS`, nên **không thuộc tập khóa dự tuyển** mà chỉ là biểu thức chỉ mục vật lý cho tập con `payment_status = 'SUCCESS'`.

• **Chứng minh dạng chuẩn:**

• **1NF:** Mọi thuộc tính nguyên tố.

• **2NF:** Khóa dự tuyển duy nhất K_{14} = {payment_id} là khóa đơn thuộc tính ⇒ Đạt **2NF**.

• **3NF & BCNF:** Giữa các thuộc tính không khóa (`booking_id`, `amount`, `payment_method`, `payment_status`, `paid_at`, `transaction_ref`) không tồn tại phụ thuộc hàm toàn bảng nào (ví dụ: cùng một `booking_id` vẫn có thể có lần thử thanh toán khác `payment_method`, khác `payment_status` và khác `paid_at`). FD không tầm thường duy nhất f_{14.1} có vế trái là siêu khóa K_{14} ⇒ Đạt **3NF** (và **BCNF**).

5. Bảng Tổng kết Kết quả Chuẩn hóa Đủ 14 Quan hệ

STT	Tên Quan hệ	Khóa chính (PK)	Khóa dự tuyển khác trên toàn bảng (Alternate Keys)	Thuộc tính không khóa (Non-prime)	1NF	2NF	3NF	Kết luận
1	cinema_branch	branch_id	branch_code, branch_name, phone	address, city, active	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
2	screening_room	room_id	(branch_id, room_name)	room_type, active	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
3	seat_type	seat_type_code	type_name	surcharge	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
4	seat	seat_id	(room_id, seat_row, seat_number)	seat_type_code	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
5	movie	movie_id	movie_code	title, duration_minutes, release_date, age_rating, country, description	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
6	genre	genre_id	genre_name	description	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
7	movie_genre	(movie_id, genre_id)	—	(Không có)	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
8	screening	screening_id	—	movie_id, room_id, start_time, end_time, base_price, status	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
9	customer	customer_id	phone, email	full_name, birth_date, created_at	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
10	employee	employee_id	employee_code, phone	branch_id, full_name, role, hired_date	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
11	booking	booking_id	booking_code	screening_id, customer_id, employee_id, booked_at, booking_status	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**

STT	Tên Quan hệ	Khóa chính (PK)	Khóa dự tuyển khác trên toàn bảng (Alternate Keys)	Thuộc tính không khóa (Non-prime)	1NF	2NF	3NF	Kết luận
12	ticket	ticket_id	(booking_id, seat_id)	unit_price, ticket_status, cancelled_at	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
13	screening_seat_lock	(screening_id, seat_id)	ticket_id	(Không có)	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**
14	payment	payment_id	—	booking_id, amount, payment_method, payment_status, paid_at, transaction_ref	Đạt	Đạt	Đạt	**Đạt 3NF (BCNF)**

CHƯƠNG 5. KIẾN TRÚC ỨNG DỤNG, CÁC MÀN HÌNH NGHIỆP VỤ & SQL CHỨC NĂNG (R5, R6, R7, R8)

5.1. Kiến trúc Phân tầng, Phân trang Tường minh & Giao dịch Đồng thời trên MySQL 8.4 LTS

Ứng dụng Desktop được xây dựng trên nền tảng **Java 21 LTS**, **Java Swing + FlatLaf 3.5**, **JDBC (MySQL Connector/J 9.1.0 + PreparedStatement)** kết nối trực tiếp tới **MySQL Community Server 8.4.0 LTS** (`InnoDB`, `utf8mb4_unicode_ci`, `ONLY_FULL_GROUP_BY`, `STRICT_TRANS_TABLES`) và **Apache PDFBox 3**, tuân thủ nghiêm ngặt kiến trúc phân tầng (`ui` → `service` → `repository` → `shared.db`). Bộ kiểm thử kiến trúc tự động `ArchitectureComplianceTest` (ArchUnit) đảm bảo không một lớp giao diện nào được phép truy cập trực tiếp `java.sql.*`. Mọi danh sách dữ liệu lớn đã loại bỏ hoàn toàn giới hạn ngầm `LIMIT 1000` và thay bằng thanh phân trang tường minh **50 dòng/trang** (`PageResult`, `SearchPageResult`) hoặc bộ lọc nhanh hiển thị rõ tổng số dòng.

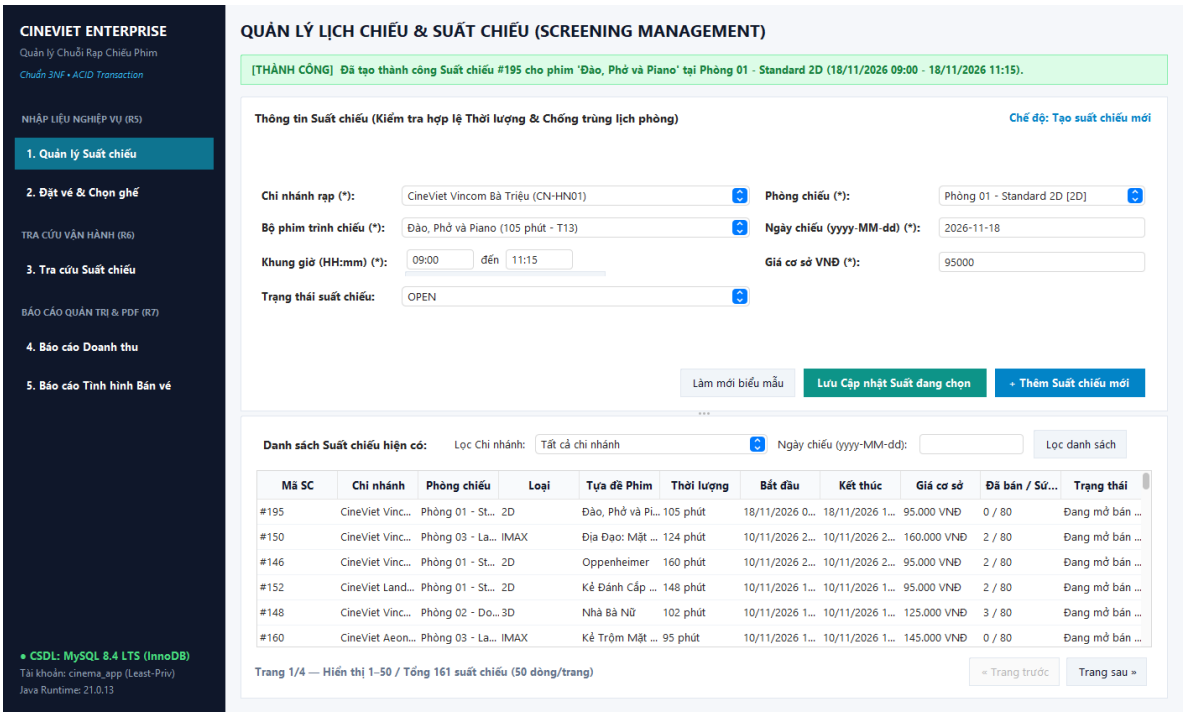
Đối với hai bài toán tranh chấp đồng thời cốt lõi trên lược đồ 14 bảng chuẩn 3NF:

• **Chống chồng lấn lịch chiếu đồng thời (BR-06):** `ScreeningService` khóa bi quan dòng phòng chiếu `SELECT room_id, active FROM screening_room WHERE room_id = ? FOR UPDATE` trước khi kiểm tra giao thoa khung giờ, kết hợp với Trigger `trg_screening_validate_ins/upd` tại tầng MySQL Server.

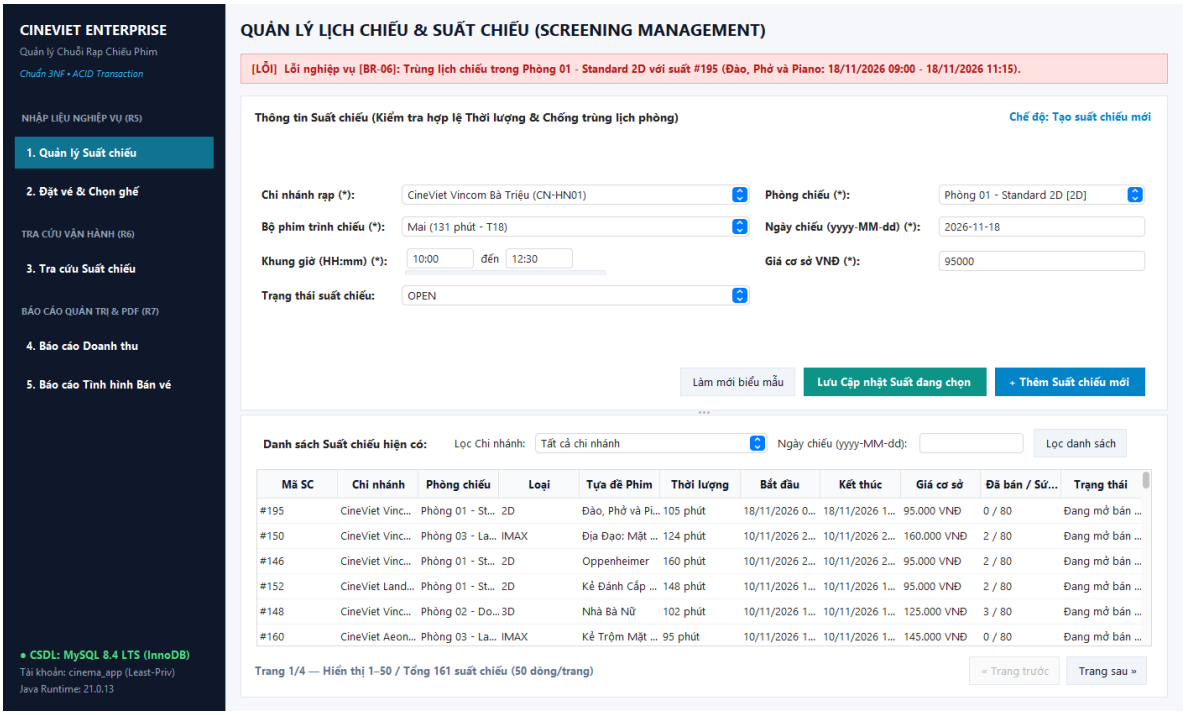
• **Chống bán trùng ghế đồng thời (BR-10):** `BookingService` khóa dòng suất chiếu `SELECT ... FROM screening WHERE screening_id = ? FOR UPDATE`, kiểm tra trạng thái ghế trên `screening_seat_lock`, lưu nguyên tử `booking` + `ticket` (không chứa `screening_id`) + `payment`, và được bảo vệ lớp cuối bởi khóa chính B-Tree `PRIMARY KEY (screening_id, seat_id)` của bảng `screening_seat_lock`.

5.2. Màn hình Nhập liệu 1: Quản lý Lịch chiếu & Suất chiếu — Phân trang 50 dòng/trang (Đáp ứng R5)

Màn hình cho phép Quản lý Chi nhánh tạo mới, cập nhật hoặc hủy suất chiếu với thanh phân trang tường minh 50 dòng/trang; tự động gợi ý giờ kết thúc dựa trên thời lượng bộ phim cộng 20 phút dọn phòng; kiểm tra hợp lệ giá cơ sở dương, thời lượng tối thiểu (`BR-05`), chặn đổi phòng khi đã phát sinh vé (`BR-09`) và từ chối tức thì mọi suất chiếu trùng lịch trong cùng một phòng chiếu (`BR-06`).



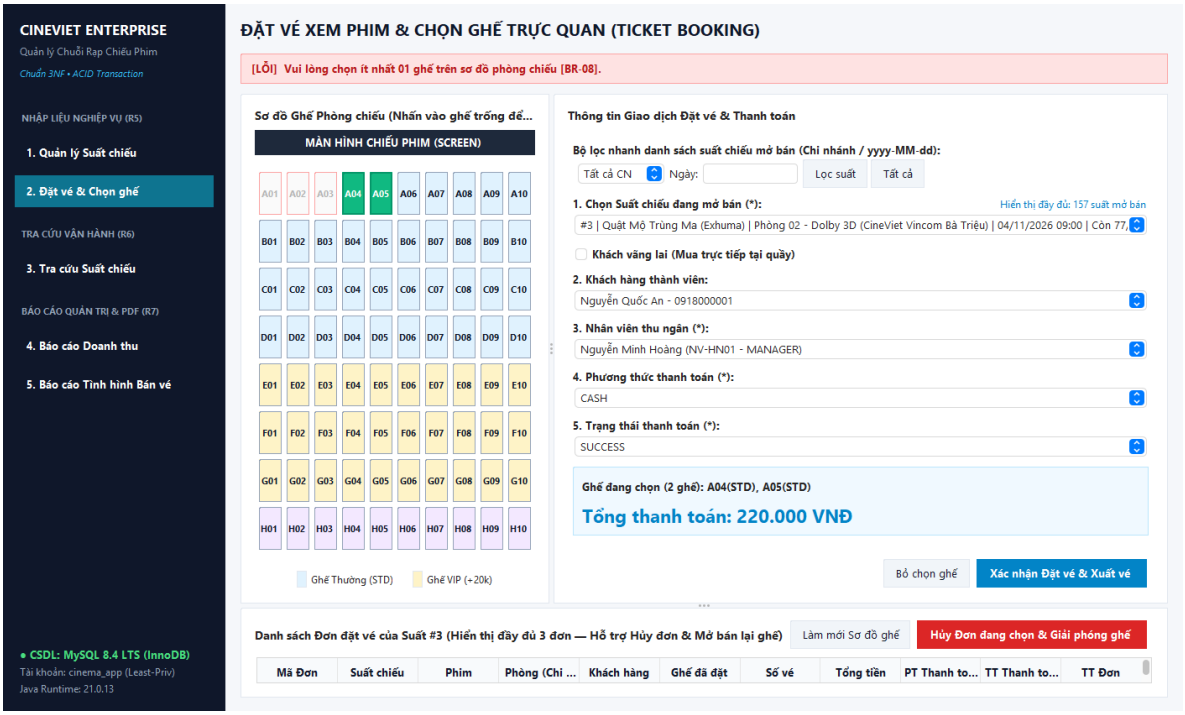
Hình 5.1. Màn hình Quản lý Suất chiếu — Tạo mới thành công suất chiếu hợp lệ và thanh phân trang 50 dòng/trang.



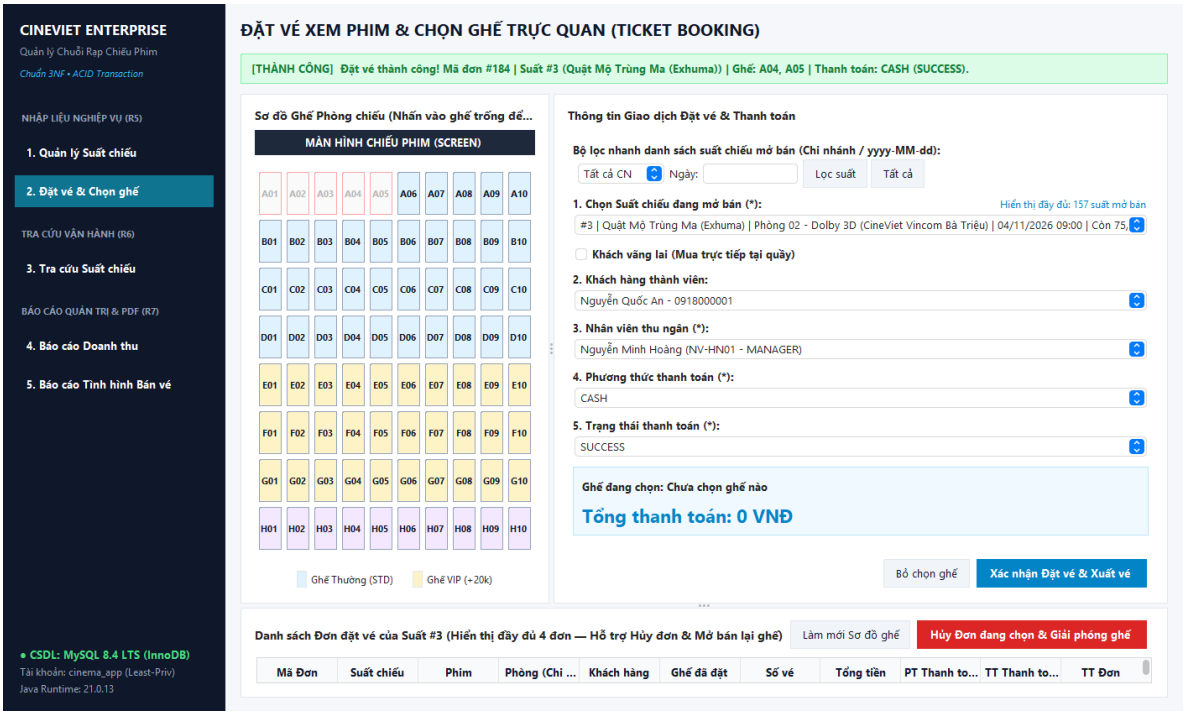
Hình 5.2. Màn hình Quản lý Suất chiếu — Phát hiện và từ chối suất chiếu chồng lấn thời gian trong cùng phòng [BR-06].

5.3. Màn hình Nhập liệu 2: Đặt vé & Chọn ghế theo Sơ đồ Phòng chiếu Chuẩn 3NF (Đáp ứng R5)

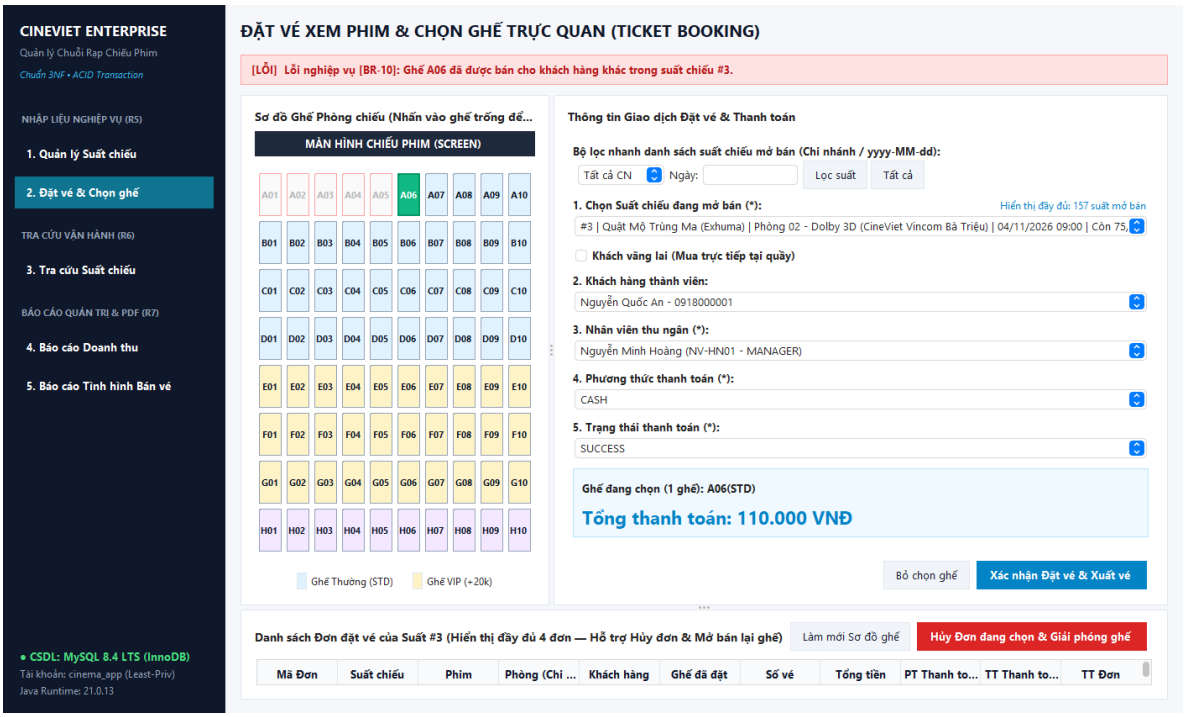
Màn hình cung cấp bộ lọc nhanh Chi nhánh + Ngày chiếu (hiển thị đầy đủ tổng số suất mở bán), phục vụ Thu ngân bán vé cho cả **Khách hàng thành viên** lẫn **Khách vãng lai tại quầy**, hiển thị trực quan sơ đồ 80 ghế dựa trên bảng `screening_seat_lock`, tính tự động đơn giá lịch sử (`base_price + surcharge`), xử lý thanh toán nguyên tử và cho phép **Hủy đơn hoàn tiền để giải phóng khóa ghế mở bán lại (BR-11)**.



Hình 5.3. Màn hình Đặt vé & Chọn ghế — Bộ lọc nhanh suất chiếu và chọn nhiều ghế trống trên sơ đồ 80 ghế.



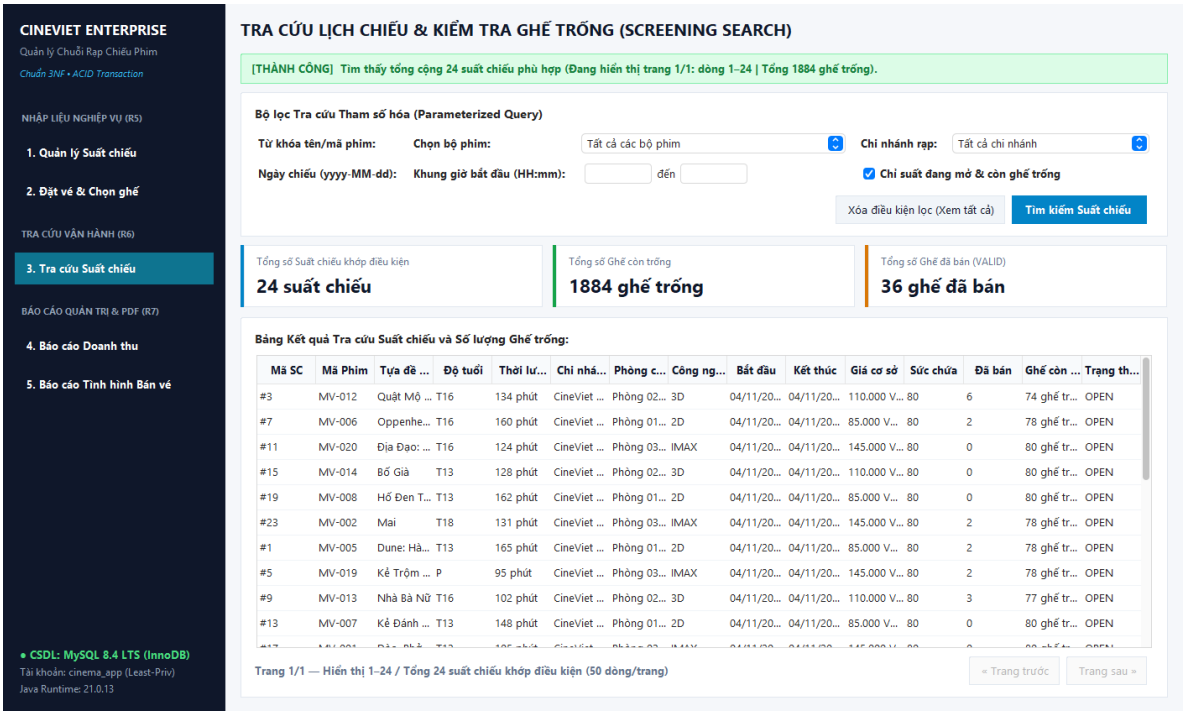
Hình 5.4. Màn hình Đặt vé & Chọn ghế — Xác nhận giao dịch đặt vé chuẩn 3NF và thanh toán thành công.



Hình 5.5. Màn hình Đặt vé & Chọn ghế — Chặn giao dịch khi ghế vừa bị quầy thu ngân khác đặt trước [BR-10].

5.4. Màn hình Tìm kiếm Vận hành: Tra cứu Suất chiếu, KPI & Phân trang (Đáp ứng R6)

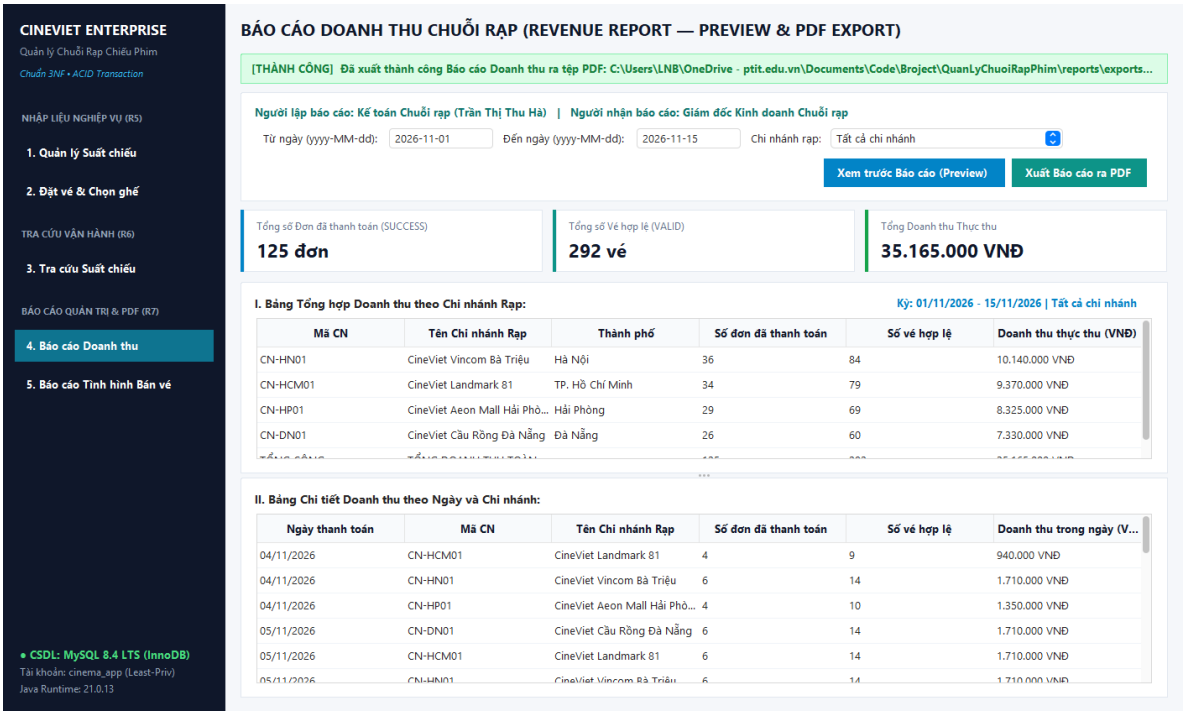
Màn hình sử dụng truy vấn tham số hóa (`PreparedStatement`) trên View `vw_screening_search`, cho phép lọc linh hoạt theo từ khóa tên/mã phim, bộ phim, chi nhánh rạp, ngày chiếu, khung giờ bắt đầu và cờ còn ghế trống; hiển thị tổng hợp KPI trên toàn tập kết quả khớp kèm phân trang 50 dòng/trang.



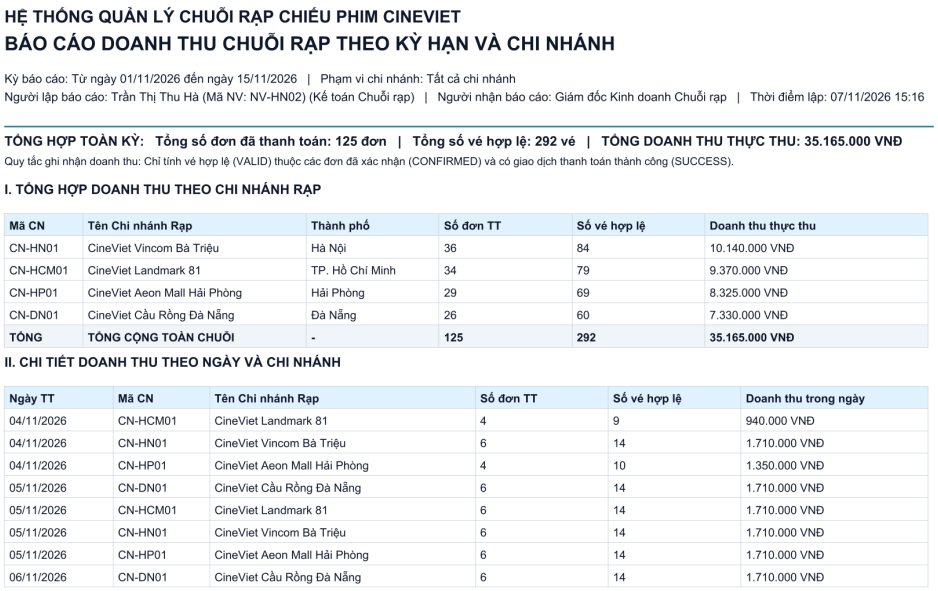
Hình 5.6. Màn hình Tra cứu Suất chiếu — Hiển thị danh sách suất chiếu khớp điều kiện kèm sức chứa, vé đã giữ, ghế trống và thanh phân trang.

5.5. Báo cáo Thực tiễn 1: Báo cáo Doanh thu Chuỗi rạp — Preview & Xuất PDF (Đáp ứng R7)

• **Người lập báo cáo:** Kế toán Chuỗi rạp (`Trần Thị Thu Hà`).
• **Người nhận báo cáo:** Giám đốc Kinh doanh Chuỗi rạp.
• **Quy tắc ghi nhận doanh thu (BR-14, BR-15):** Chỉ tính tổng `ticket.unit_price` của các vé `VALID` thuộc đơn `CONFIRMED` có giao dịch thanh toán `SUCCESS` (loại trừ hoàn toàn các đơn `PENDING`, `FAILED` và `CANCELLED`/`REFUNDED`, đồng thời không bị nhân đôi bởi 15 đơn có lịch sử thanh toán lỗi `FAILED` trước khi `SUCCESS`). Tổng doanh thu thực thu trên tập dữ liệu mẫu đạt chính xác **35.165.000 VNĐ** (khớp 100% truy vấn SQL đối soát độc lập).



Hình 5.7. Màn hình Xem trước (Preview) Báo cáo Doanh thu Chuỗi rạp (nêu rõ Kế toán lập gửi Giám đốc Kinh doanh).



HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM CINEVIET

BÁO CÁO DOANH THU CHUỖI RẠP THEO KỲ HẠN VÀ CHI NHÁNH

Kỳ báo cáo: Từ ngày 01/11/2026 đến ngày 15/11/2026 | Phạm vi chi nhánh: Tất cả chi nhánh

Người lập báo cáo: Trần Thị Thu Hà (Mã NV: NV-HN02) (Kế toán Chuỗi rạp) | Người nhận báo cáo: Giám đốc Kinh doanh Chuỗi rạp | Thời điểm lập: 07/11/2026 15:16

TỔNG HỢP TOÀN KỲ: Tổng số đơn đã thanh toán: 125 đơn | Tổng số vé hợp lệ: 292 vé | TỔNG DOANH THU THỰC THU: 35.165.000 VNĐ

Quy tắc ghi nhận doanh thu: Chỉ tính vé hợp lệ (VALID) thuộc các đơn đã xác nhận (CONFIRMED) và có giao dịch thanh toán thành công (SUCCESS).

I. TỔNG HỢP DOANH THU THEO CHI NHÁNH RẠP

Mã CN	Tên Chi nhánh Rạp	Thành phố	Số đơn TT	Số vé hợp lệ	Doanh thu thực thu
CN-HN01	CineViet Vincom Bà Triệu	Hà Nội	36	84	10.140.000 VNĐ
CN-HCM01	CineViet Landmark 81	TP. Hồ Chí Minh	34	79	9.370.000 VNĐ
CN-HP01	CineViet Aeon Mall Hải Phòng	Hải Phòng	29	69	8.325.000 VNĐ
CN-DN01	CineViet Cầu Rồng Đà Nẵng	Đà Nẵng	26	60	7.330.000 VNĐ
TỔNG	**TỔNG CỘNG TOÀN CHUỖI**	-	**125**	**292**	**35.165.000 VNĐ**

II. CHI TIẾT DOANH THU THEO NGÀY VÀ CHI NHÁNH

Ngày TT	Mã CN	Tên Chi nhánh Rạp	Số đơn TT	Số vé hợp lệ	Doanh thu trong ngày
04/11/2026	CN-HCM01	CineViet Landmark 81	4	9	940.000 VNĐ
04/11/2026	CN-HN01	CineViet Vincom Bà Triệu	6	14	1.710.000 VNĐ
04/11/2026	CN-HP01	CineViet Aeon Mall Hải Phòng	4	10	1.350.000 VNĐ
05/11/2026	CN-DN01	CineViet Cầu Rồng Đà Nẵng	6	14	1.710.000 VNĐ
05/11/2026	CN-HCM01	CineViet Landmark 81	6	14	1.710.000 VNĐ
05/11/2026	CN-HN01	CineViet Vincom Bà Triệu	6	14	1.710.000 VNĐ
05/11/2026	CN-HP01	CineViet Aeon Mall Hải Phòng	6	14	1.710.000 VNĐ
06/11/2026	CN-DN01	CineViet Cầu Rồng Đà Nẵng	6	14	1.710.000 VNĐ

Hình 5.8. Bản kết xuất PDF tiếng Việt Báo cáo Doanh thu Chuỗi rạp (reports/exports/BaoCaoDoanhThu_Mau.pdf).

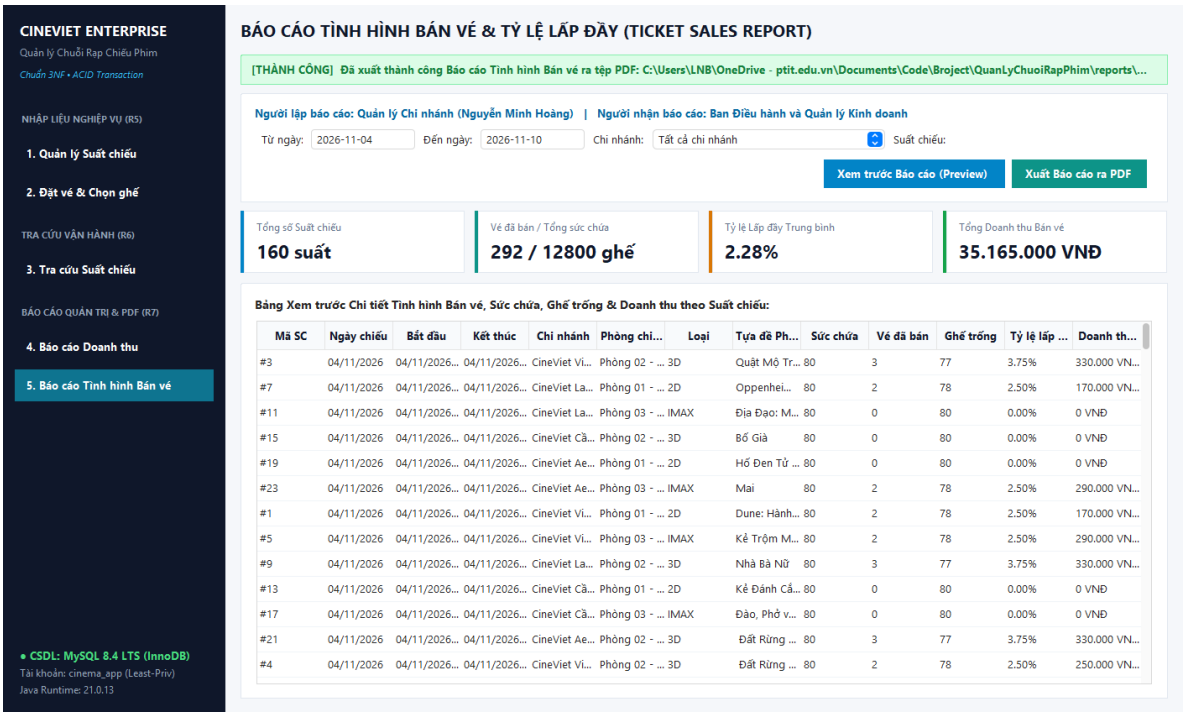
5.6. Báo cáo Thực tiễn 2: Báo cáo Tình hình Bán vé & Tỷ lệ Lấp đầy — Preview & Xuất PDF (Đáp ứng R7)

• **Người lập báo cáo:** Quản lý Chi nhánh (`Nguyễn Minh Hoàng`).

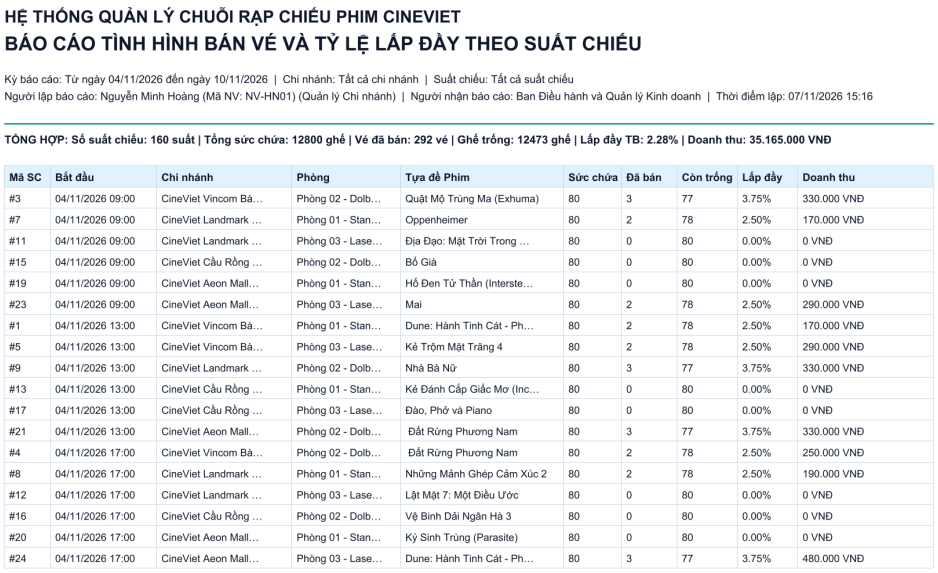
• **Người nhận báo cáo:** Ban Điều hành và Quản lý Kinh doanh.

• **Nội dung quản trị:** Thống kê chi tiết từng suất chiếu từ MySQL 8.4 về sức chứa phòng, số vé đã thanh toán, số ghế còn trống thực tế, tỷ lệ lấp đầy phòng chiếu (`%`) và doanh thu bán vé thực thu; hỗ trợ xem trước

trực quan và kết xuất PDF tiếng Việt.



Hình 5.9. Màn hình Xem trước (Preview) Báo cáo Tình hình Bán vé và Tỷ lệ Lấp đầy (Quản lý Chi nhánh gửi Ban Điều hành).



HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM CINEVIET

BÁO CÁO TÌNH HÌNH BÁN VÉ VÀ TỶ LỆ LẤP ĐẦY THEO SUẤT CHIẾU

Kỳ báo cáo: Từ ngày 04/11/2026 đến ngày 10/11/2026 | Chi nhánh: Tất cả chi nhánh | Suất chiếu: Tất cả suất chiếu

Người lập báo cáo: Nguyễn Minh Hoàng (Mã NV: NV-HN01) (Quản lý Chi nhánh) | Người nhận báo cáo: Ban Điều hành và Quản lý Kinh doanh | Thời điểm lập: 07/11/2026 15:16

TỔNG HỢP: Số suất chiếu: 160 suất | Tổng sức chứa: 12800 ghế | Vé đã bán: 292 vé | Ghế trống: 12473 ghế | Lấp đầy TB: 2.28% | Doanh thu: 35.165.000 VNĐ

Mã SC	Bắt đầu	Chi nhánh	Phòng	Tựa đề Phim	Sức chứa	Đã bán	Còn trống	Lấp đầy	Doanh thu
#3	04/11/2026 09:00	CineViet Vincom Bà…	Phòng 02 - Dolb…	Quật Mộ Trùng Ma (Exhuma)	80	3	77	3.75%	330.000 VNĐ
#7	04/11/2026 09:00	CineViet Landmark …	Phòng 01 - Stan…	Oppenheimer	80	2	78	2.50%	170.000 VNĐ
#11	04/11/2026 09:00	CineViet Landmark …	Phòng 03 - Lase…	Địa Đạo: Mặt Trời Trong …	80	0	80	0.00%	0 VNĐ
#15	04/11/2026 09:00	CineViet Cầu Rồng …	Phòng 02 - Dolb…	Bố Già	80	0	80	0.00%	0 VNĐ
#19	04/11/2026 09:00	CineViet Aeon Mall…	Phòng 01 - Stan…	Hố Đen Tử Thần (Interste…	80	0	80	0.00%	0 VNĐ
#23	04/11/2026 09:00	CineViet Aeon Mall…	Phòng 03 - Lase…	Mai	80	2	78	2.50%	290.000 VNĐ
#1	04/11/2026 13:00	CineViet Vincom Bà…	Phòng 01 - Stan…	Dune: Hành Tinh Cát - Ph…	80	2	78	2.50%	170.000 VNĐ
#5	04/11/2026 13:00	CineViet Vincom Bà…	Phòng 03 - Lase…	Kẻ Trộm Mặt Trăng 4	80	2	78	2.50%	290.000 VNĐ
#9	04/11/2026 13:00	CineViet Landmark …	Phòng 02 - Dolb…	Nhà Bà Nữ	80	3	77	3.75%	330.000 VNĐ
#13	04/11/2026 13:00	CineViet Cầu Rồng …	Phòng 01 - Stan…	Kẻ Đánh Cắp Giấc Mơ (Inc…	80	0	80	0.00%	0 VNĐ
#17	04/11/2026 13:00	CineViet Cầu Rồng …	Phòng 03 - Lase…	Đào, Phở và Piano	80	0	80	0.00%	0 VNĐ
#21	04/11/2026 13:00	CineViet Aeon Mall…	Phòng 02 - Dolb…	Đất Rừng Phương Nam	80	3	77	3.75%	330.000 VNĐ
#4	04/11/2026 17:00	CineViet Vincom Bà…	Phòng 02 - Dolb…	Đất Rừng Phương Nam	80	2	78	2.50%	250.000 VNĐ
#8	04/11/2026 17:00	CineViet Landmark …	Phòng 01 - Stan…	Những Mảnh Ghép Cảm Xúc 2	80	2	78	2.50%	190.000 VNĐ
#12	04/11/2026 17:00	CineViet Landmark …	Phòng 03 - Lase…	Lật Mặt 7: Một Điều Ước	80	0	80	0.00%	0 VNĐ
#16	04/11/2026 17:00	CineViet Cầu Rồng …	Phòng 02 - Dolb…	Vệ Binh Dải Ngân Hà 3	80	0	80	0.00%	0 VNĐ
#20	04/11/2026 17:00	CineViet Aeon Mall…	Phòng 01 - Stan…	Ký Sinh Trùng (Parasite)	80	0	80	0.00%	0 VNĐ
#24	04/11/2026 17:00	CineViet Aeon Mall…	Phòng 03 - Lase…	Dune: Hành Tinh Cát - Ph…	80	3	77	3.75%	480.000 VNĐ

Hình 5.10. Bản kết xuất PDF tiếng Việt Báo cáo Tình hình Bán vé (reports/exports/BaoCaoBanVe_Mau.pdf).

5.7. Đặc tả Chi tiết Các Câu lệnh SQL Tương ứng Chức năng R5, R6, R7 (Đáp ứng R8)

(Tài liệu đáp ứng Tiêu chí R8 — Bổ sung SQL thực thi tương ứng các chức năng ứng dụng)

Nguyên tắc Đối chiếu SQL Mã nguồn Java và SQL Kiểm chứng Thực tế

Toàn bộ các câu lệnh SQL dưới đây được trích xuất trực tiếp từ 5 lớp `*Repository.java` của ứng dụng (kết nối qua `PreparedStatement` chống SQL Injection tuyệt đối) và được kiểm chứng thực thi tự động trên **cinema_chain_test (MySQL Community Server 8.4.0 LTS)** thông qua tệp `database/queries/functional_sql_verification.sql` (kết quả lưu tại `verification/logs/functional_sql_examples_test_db.log`).

Mỗi chức năng đều phân biệt rõ hai dạng biểu diễn:

1. **Câu lệnh SQL tham số hóa (?) trong mã nguồn Java (PreparedStatement):** Mã thực thi thật trong ứng dụng.
2. **Câu lệnh SQL ví dụ cụ thể chạy trực tiếp trên cinema_chain_test:** Dán trực tiếp vào MySQL CLI / MySQL Workbench để kiểm chứng ngay kết quả (mọi ví dụ ghi dữ liệu đều bọc trong `START TRANSACTION ... ROLLBACK` khi kiểm thử để không làm bẩn dữ liệu nền; trong luồng nghiệp vụ thật của ứng dụng, `TransactionTemplate` kết thúc bằng `COMMIT` khi hợp lệ hoặc `ROLLBACK` khi vi phạm quy tắc).

1. Biểu mẫu Nhập liệu 1: Quản lý Lịch chiếu & Suất chiếu (`R5.1`)

• **Mục đích:** Cho phép Quản lý Chi nhánh nạp danh mục chi nhánh/phòng/phim, kiểm tra hợp lệ thời lượng phim (`BR-05`), khóa dòng phòng chiếu chống tranh chấp đồng thời (`FOR UPDATE`), phát hiện xung đột lịch chiếu trong cùng phòng (`BR-06`), thêm mới/cập nhật suất chiếu và phân trang danh sách suất chiếu (`50 dòng/trang`).

• **Đầu vào:** `movieId`, `roomId`, `startTime`, `endTime`, `basePrice`, `status` (`OPEN` / `CANCELLED`), bộ lọc phân trang `(branchId, roomId, movieId, dateFilter, limit, offset)`.

• **Bảng & View liên quan:** `cinema_branch`, `screening_room`, `movie`, `screening`, `booking`, `ticket`, `vw_screening_search`.

• **Lớp mã nguồn Java:** `com.cinema.feature.screening.ScreeningRepository` & `ScreeningService`.

• **Ảnh chụp màn hình demo:** `verification/screenshots/01_screening_management_valid.png` (hợp lệ), `verification/screenshots/02_screening_management_overlap_rejected.png` (từ chối trùng lịch `[BR-06]`).

1.1. Câu lệnh SQL tham số hóa (`?`) trong Java (`ScreeningRepository.java`)

```
-- (a) Nạp phòng chiếu đang hoạt động theo chi nhánh (findRoomsByBranch)
SELECT r.room_id, r.branch_id, b.branch_name, r.room_name, r.room_type, r.active
FROM screening_room r
JOIN cinema_branch b ON b.branch_id = r.branch_id
WHERE r.active = 1 AND r.branch_id = ?
ORDER BY r.branch_id ASC, r.room_id ASC;

-- (b) Trong giao dịch TransactionTemplate: khóa dòng phòng chiếu & kiểm tra active [BR-01, BR-06]
SELECT room_id, active FROM screening_room WHERE room_id = ? FOR UPDATE;

-- (c) Tra cứu thời lượng phim để kiểm tra end_time >= start_time + duration_minutes [BR-05]
SELECT movie_id, movie_code, title, duration_minutes, release_date, age_rating, country, description
FROM movie
WHERE movie_id = ?;

-- (d) Kiểm tra xung đột khung giờ chiếu trong cùng phòng [BR-06] (findOverlappingScreenings)
SELECT screening_id, movie_id, movie_code, movie_title, duration_minutes, age_rating,
       room_id, room_name, room_type, branch_id, branch_code, branch_name,
       start_time, end_time, base_price, screening_status, total_seats, sold_seats, available_seats
FROM vw_screening_search
WHERE room_id = ?
  AND screening_status = 'OPEN'
```

```
  AND start_time < ?
  AND end_time > ?
  AND screening_id <> ?; -- (chỉ thêm điều kiện này khi UPDATE suất chiếu hiện có)

-- (e) Kiểm tra số vé đã phát hành qua quan hệ chuẩn 3NF (ticket JOIN booking) trước khi đổi phòng/hủy [
BR-08, BR-09]
SELECT COUNT(*)
FROM ticket tk
JOIN booking bk ON bk.booking_id = tk.booking_id
WHERE bk.screening_id = ?;

-- (f) Thêm mới suất chiếu (insertScreening) & Cập nhật suất chiếu (updateScreening)
INSERT INTO screening (movie_id, room_id, start_time, end_time, base_price, status)
VALUES (?, ?, ?, ?, ?, ?);

UPDATE screening
SET movie_id = ?, room_id = ?, start_time = ?, end_time = ?, base_price = ?, status = ?
WHERE screening_id = ?;

-- (g) Đếm tổng số dòng và truy vấn phân trang tường minh 50 dòng/trang (countScreenings & findScreening
sPage)
SELECT COUNT(*) FROM vw_screening_search WHERE 1=1 AND branch_id = ? AND screening_date = ?;

SELECT screening_id, movie_id, movie_code, movie_title, duration_minutes, age_rating,
       room_id, room_name, room_type, branch_id, branch_code, branch_name,
       start_time, end_time, base_price, screening_status, total_seats, sold_seats, available_seats
FROM vw_screening_search
WHERE 1=1
ORDER BY start_time DESC, screening_id DESC
LIMIT ? OFFSET ?;
```

1.2. Ví dụ SQL cụ thể chạy trên `cinema_chain_test` & Kết quả mong đợi

```
START TRANSACTION;
SELECT room_id, active FROM screening_room WHERE room_id = 1 FOR UPDATE;
-- Phát hiện trùng lịch với Suất #1 (13:00 -> 16:05 ngày 2026-11-04 tại Phòng 1):
SELECT screening_id, movie_title, room_name, start_time, end_time
FROM vw_screening_search
WHERE room_id = 1 AND screening_status = 'OPEN'
  AND start_time < '2026-11-04 15:00:00'
  AND end_time   > '2026-11-04 13:30:00';
-- Thêm suất hợp lệ vào khung trống ngày 2026-11-19:
INSERT INTO screening (movie_id, room_id, start_time, end_time, base_price, status)
VALUES (1, 1, '2026-11-19 09:00:00', '2026-11-19 11:15:00', 95000.00, 'OPEN');
COMMIT; -- (hoặc ROLLBACK khi chạy kiểm chứng không phá hủy)
```

• **Kết quả mong đợi:** Truy vấn kiểm tra xung đột ngày `2026-11-04` trả về đúng `screening_id = 1` (Dune: Hành Tinh Cát - Phần Hai, `13:00:00 -> 16:05:00`) nên bị từ chối với mã `[BR-06]`; lệnh `INSERT` ngày `2026-11-19` thành công vì không giao nhau và thời lượng `135 phút >= 105 phút`.

2. Biểu mẫu Nhập liệu 2: Đặt vé, Chọn ghế Sơ đồ Phòng & Hủy Hoàn tiền (`R5.2`)

• **Mục đích:** Tải danh sách suất chiếu mở bán (có bộ lọc nhanh Chi nhánh + Ngày chiếu, hiển thị đầy đủ tổng số suất không cắt ngầm), hiển thị sơ đồ 80 ghế trực quan dựa trên bảng chuẩn 3NF `screening_seat_lock`, khóa dòng suất chiếu `FOR UPDATE`, tạo nguyên tử `booking` + `ticket` (không chứa `screening_id`) + `payment`, và xử lý hủy đơn hoàn tiền giải phóng ghế mở bán lại (`BR-10`, `BR-11`, `BR-12`, `BR-13`).

• **Đầu vào:** `screeningId`, `customerId` (`NULL` nếu Khách vãng lai tại quầy), `employeeId`, `seatIds` (danh sách mã ghế), `paymentMethod` (`CASH`/`CARD`/`TRANSFER`/`EWALLET`), `payImmediately` (`true` -> `CONFIRMED/SUCCESS`, `false` -> `PENDING/PENDING`).

• **Bảng liên quan:** `screening`, `seat`, `seat_type`, `screening_seat_lock`, `booking`, `ticket`, `payment`, `customer`, `employee`.

• **Lớp mã nguồn Java:** `com.cinema.feature.booking.BookingRepository` & `BookingService`.

• **Ảnh chụp màn hình demo:** `verification/screenshots/03_ticket_booking_seat_selection.png`, `04_ticket_booking_success.png`, `04b_ticket_booking_occupied_rejected.png`.

2.1. Câu lệnh SQL tham số hóa (?) trong Java (BookingRepository.java)

```
-- (a) Truy vấn sơ đồ 80 ghế của suất chiếu qua bảng khóa ghế chuẩn 3NF `screening_seat_lock` (findSeats
ForScreening)
SELECT
    st.seat_id, st.room_id, st.seat_row, st.seat_number, st.seat_type_code,
    tp.type_name, tp.surcharge,
    (sc.base_price + tp.surcharge) AS final_unit_price,
    CASE WHEN sl.ticket_id IS NOT NULL THEN 1 ELSE 0 END AS is_occupied
FROM screening sc
JOIN seat st      ON st.room_id = sc.room_id
JOIN seat_type tp ON tp.seat_type_code = st.seat_type_code
LEFT JOIN screening_seat_lock sl
       ON sl.screening_id = sc.screening_id
      AND sl.seat_id = st.seat_id
WHERE sc.screening_id = ?
ORDER BY st.seat_row ASC, st.seat_number ASC;

-- (b) Giao dịch nguyên tử Đặt vé & Thanh toán (BookingService.createBooking trong TransactionTemplate):
-- Bước 1: Khóa bi quan dòng suất chiếu để tuần tự hóa các thu ngân đặt cùng suất
SELECT screening_id, room_id, base_price, status
FROM screening
WHERE screening_id = ?
FOR UPDATE;

-- Bước 2: Kiểm tra từng ghế thuộc đúng phòng chiếu và chưa bị giữ trong `screening_seat_lock`
SELECT
    st.seat_id, st.room_id,
    CONCAT(st.seat_row, LPAD(st.seat_number, 2, '0')) AS seat_label,
    tp.surcharge,
    CASE WHEN sl.ticket_id IS NOT NULL THEN 1 ELSE 0 END AS already_occupied
FROM seat st
JOIN seat_type tp ON tp.seat_type_code = st.seat_type_code
LEFT JOIN screening_seat_lock sl
       ON sl.screening_id = ?
      AND sl.seat_id = st.seat_id
WHERE st.seat_id = ?;

-- Bước 3: Tạo đơn đặt vé (customer_id truyền NULL nếu khách vãng lai)
INSERT INTO booking (booking_code, screening_id, customer_id, employee_id, booked_at, booking_status)
VALUES (?, ?, ?, ?, ?, ?);

-- Bước 4: Tạo từng vé chuẩn 3NF (KHÔNG có cột screening_id; Trigger tự chèn khóa vào screening_seat_loc
k)
INSERT INTO ticket (booking_id, seat_id, unit_price, ticket_status)
VALUES (?, ?, ?, ?);

-- Bước 5: Ghi nhận giao dịch thanh toán khớp tổng tiền vé hợp lệ [BR-13]
INSERT INTO payment (booking_id, amount, payment_method, payment_status, paid_at, transaction_ref)
VALUES (?, ?, ?, ?, ?, ?);
-- Kết thúc khối TransactionTemplate: conn.commit() nếu tất cả hợp lệ; conn.rollback() nếu có bất kỳ lỗi
 nào!

-- (c) Giao dịch nguyên tử Hủy đơn vé, Hoàn tiền & Giải phóng khóa ghế (cancelBookingRecords)
SELECT booking_id, booking_status FROM booking WHERE booking_id = ? FOR UPDATE;
UPDATE booking SET booking_status = 'CANCELLED' WHERE booking_id = ?;
UPDATE ticket  SET ticket_status = 'CANCELLED', cancelled_at = CURRENT_TIMESTAMP WHERE booking_id = ?;

-- (Trigger `trg_ticket_after_upd` tự động DELETE dòng tương ứng khỏi `screening_seat_lock` ngay lập tức
)
UPDATE payment SET payment_status = 'REFUNDED' WHERE booking_id = ? AND payment_status = 'SUCCESS';
UPDATE payment SET payment_status = 'FAILED'   WHERE booking_id = ? AND payment_status = 'PENDING';
```

2.2. Ví dụ SQL cụ thể chạy trên cinema_chain_test & Kết quả mong đợi

```
START TRANSACTION;
SELECT screening_id, room_id, base_price, status FROM screening WHERE screening_id = 1 FOR UPDATE;
INSERT INTO booking (booking_code, screening_id, customer_id, employee_id, booked_at, booking_status)
VALUES ('BK-DEMO-3NF-01', 1, NULL, 3, '2026-11-07 10:00:00', 'CONFIRMED');
SET @new_bk := LAST_INSERT_ID();

INSERT INTO ticket (booking_id, seat_id, unit_price, ticket_status)
VALUES (@new_bk, 3, 85000.00, 'VALID'), (@new_bk, 4, 85000.00, 'VALID');

INSERT INTO payment (booking_id, amount, payment_method, payment_status, paid_at, transaction_ref)
VALUES (@new_bk, 170000.00, 'CASH', 'SUCCESS', '2026-11-07 10:00:05', 'TXN-DEMO-3NF-01');
```

```
-- Hủy đơn vừa đặt -> giải phóng khóa ghế để bán lại:
UPDATE booking SET booking_status = 'CANCELLED' WHERE booking_id = @new_bk;
UPDATE ticket  SET ticket_status = 'CANCELLED', cancelled_at = CURRENT_TIMESTAMP WHERE booking_id = @new
_bk;
UPDATE payment SET payment_status = 'REFUNDED' WHERE booking_id = @new_bk AND payment_status = 'SUCCESS'
;
ROLLBACK;
```

• **Kết quả mong đợi:** Ngay sau `INSERT INTO ticket`, bảng `screening_seat_lock` tự động sinh 2 dòng khóa ghế `(1, 3)` và `(1, 4)` (chặn mọi giao dịch thứ hai đặt trùng ghế `A03`, `A04`). Ngay sau `UPDATE ticket SET ticket_status = 'CANCELLED'`, số khóa ghế trong `screening_seat_lock` của 2 vé này về `0` (mở khóa bán lại ghế), trong khi 2 dòng lịch sử `CANCELLED` trong `ticket` và giao dịch `REFUNDED` trong `payment` được giữ nguyên vẹn.

3. Màn hình Tìm kiếm Vận hành: Tra cứu Suất chiếu & Kiểm tra Ghế trống (R6)

• **Mục đích:** Lọc đa tiêu chí theo từ khóa tên/mã phim, bộ phim, chi nhánh, ngày chiếu, khoảng giờ bắt đầu và cờ "Chỉ suất đang mở & còn ghế trống"; hiển thị tổng hợp KPI trên toàn bộ tập kết quả khớp và phân trang bảng kết quả `50 dòng/trang` (loại bỏ hoàn toàn lỗi cắt ngầm `LIMIT 1000`).

• **Đầu vào:** `movieKeyword`, `movieId`, `branchId`, `screeningDate`, `fromTime`, `toTime`, `onlyOpenWithAvailableSeats`, `pageIndex`, `pageSize = 50`.

• **Bảng & View liên quan:** `vw_screening_search` (tổng hợp từ `screening`, `movie`, `screening_room`, `cinema_branch`, `seat`, `screening_seat_lock`).

• **Lớp mã nguồn Java:** `com.cinema.feature.search.ScreeningSearchRepository` & `ScreeningSearchService`.

• **Ảnh chụp màn hình demo:** `verification/screenshots/05_screening_search.png`.

3.1. Câu lệnh SQL tham số hóa (?) trong Java (ScreeningSearchRepository.java)

```
-- (a) Tổng hợp chính xác KPI trên toàn tập kết quả khớp điều kiện lọc (summarizeSearch)
SELECT
    COUNT(*) AS total_count,
    COALESCE(SUM(available_seats), 0) AS sum_available,
    COALESCE(SUM(sold_seats), 0) AS sum_sold
FROM vw_screening_search
WHERE 1=1
  AND (LOWER(movie_title) LIKE ? OR LOWER(movie_code) LIKE ?)
  AND branch_id = ?
  AND screening_date = ?
  AND TIME(start_time) >= ?
  AND TIME(start_time) <= ?
  AND screening_status = 'OPEN' AND available_seats > 0;

-- (b) Truy vấn phân trang kết quả tra cứu với thứ tự ổn định (searchScreeningsPage)
SELECT
    screening_id, movie_id, movie_code, movie_title, duration_minutes, age_rating,
    room_id, room_name, room_type, branch_id, branch_code, branch_name,
    start_time, end_time, base_price, screening_status,
    total_seats, sold_seats, available_seats
FROM vw_screening_search
WHERE 1=1
  AND branch_id = ?
  AND screening_date = ?
  AND TIME(start_time) >= ?
  AND TIME(start_time) <= ?
  AND screening_status = 'OPEN' AND available_seats > 0
ORDER BY start_time ASC, branch_id ASC, room_id ASC, screening_id ASC
LIMIT ? OFFSET ?;
```

3.2. Ví dụ SQL cụ thể chạy trên cinema_chain_test & Kết quả mong đợi

```
SELECT screening_id, movie_code, movie_title, branch_name, room_name,
       start_time, base_price, total_seats, sold_seats, available_seats
FROM vw_screening_search
WHERE branch_id = 1
  AND screening_date = '2026-11-04'
```

```
  AND TIME(start_time) >= '08:00:00'
  AND TIME(start_time) <= '23:00:00'
  AND screening_status = 'OPEN'
  AND available_seats > 0
ORDER BY start_time ASC, screening_id ASC
LIMIT 50 OFFSET 0;
```

• **Kết quả thực tế trên bộ seed chuẩn (functional_sql_examples_test_db.log):** Trả về đúng **06 suất chiếu** tại CineViet Vincom Bà Triệu ngày `04/11/2026` (`total_count = 6`, tổng vé đang giữ `sum_sold = 14`, tổng ghế còn trống `sum_available = 466`, khớp `6 * 80 - 14 = 466`).

4. Báo cáo Quản trị 1: Báo cáo Doanh thu Chuỗi rạp (`R7.1`)

• **Định danh quản trị: Người lập:** Kế toán Chuỗi rạp → **Người nhận:** Giám đốc Kinh doanh Chuỗi rạp.

• **Mục đích:** Tổng hợp số đơn đã thu tiền, số vé hợp lệ và tổng doanh thu thực thu theo từng Chi nhánh và theo Ngày thanh toán (`paid_at`), phục vụ xem trên màn hình (Preview) và kết xuất PDF tiếng Việt (`reports/exports/BaoCaoDoanhThu_Mau.pdf`).

• **Đầu vào:** `fromDate`, `toDate`, `branchId` (tùy chọn `NULL` = toàn chuỗi).

• **Bảng & View liên quan:** `cinema_branch`, `screening_room`, `screening`, `booking`, `ticket`, `payment`, `vw_verified_booking_revenue`, `vw_branch_daily_revenue`.

• **Lớp mã nguồn Java:** `com.cinema.feature.report.revenue.RevenueReportRepository` & `RevenueReportService`.

• **Ảnh chụp màn hình & PDF demo:** `verification/screenshots/06_revenue_report_preview.png`, `08_exported_revenue_pdf_page1.png`, `reports/exports/BaoCaoDoanhThu_Mau.pdf`.

4.1. Quy tắc Hạch toán Minh bạch Các Trạng thái `SUCCESS`, `PENDING`, `CANCELLED`, `REFUNDED`, `FAILED`

Trạng thái Đơn (`booking_status`)	Trạng thái Vé (`ticket_status`)	Trạng thái Thanh toán (`payment_status`)	Giữ Khóa Ghế (`screening_seat_lock`)?	Ghi nhận vào Doanh thu Thực thu (`R7.1` & `R7.2`)?	Giải thích Nghiệp vụ Kế toán & Vận hành
`CONFIRMED`	`VALID`	`SUCCESS` (kể cả đơn từng có dòng `FAILED` trước đó)	**Có** (`292` ghế trên seed)	**CÓ — Ghi nhận đúng 1 lần** (`125` đơn = `35.165.000` VNĐ)	Chỉ tổng hợp `SUM(ticket.unit_price)` của vé `VALID` một lần duy nhất nhờ `vw_verified_booking_revenue` / `EXISTS`, không bị nhân đôi bởi dòng `FAILED` cũ (`BR-14`, `BR-15`).
`PENDING`	`VALID`	`PENDING`	**Có** (`35` ghế đang giữ chỗ tạm)	**KHÔNG (0 VNĐ)**	Đơn đặt giữ chỗ chờ thu tiền: ghế đã trừ khỏi `available_seats`, nhưng chưa thu tiền nên Kế toán chưa ghi nhận doanh thu thực thu.
`CANCELLED`	`CANCELLED`	`REFUNDED`	**Không** (đã xóa khóa, mở bán lại ghế)	**KHÔNG (0 VNĐ)**	Đơn đã thu tiền nhưng khách hủy hợp lệ và đã hoàn tiền (`REFUNDED`): loại trừ hoàn toàn khỏi doanh thu thực thu.
`CANCELLED`	`CANCELLED`	`FAILED`	**Không**	**KHÔNG (0 VNĐ)**	Giao dịch thanh toán thất bại và đơn đã hủy.

4.2. Câu lệnh SQL tham số hóa (?) trong Java (RevenueReportRepository.java)

```
-- (a) Bảng I: Tổng hợp doanh thu thực thu theo từng Chi nhánh (findBranchSummaries)
SELECT
    br.branch_id, br.branch_code, br.branch_name, br.city,
    COUNT(DISTINCT vr.booking_id)              AS paid_bookings,
    COALESCE(SUM(vr.valid_ticket_count), 0)    AS valid_tickets,
    COALESCE(SUM(vr.recognized_revenue), 0.00) AS paid_revenue
FROM cinema_branch br
LEFT JOIN vw_verified_booking_revenue vr
    ON vr.branch_id = br.branch_id
   AND vr.revenue_date BETWEEN ? AND ?
WHERE br.active = 1
GROUP BY br.branch_id, br.branch_code, br.branch_name, br.city
ORDER BY paid_revenue DESC, br.branch_id ASC;

-- (b) Bảng II: Chi tiết doanh thu theo Ngày thanh toán & Chi nhánh (findDailyDetails)
SELECT
    revenue_date, branch_code, branch_name,
    paid_booking_count, paid_ticket_count, total_paid_revenue
FROM vw_branch_daily_revenue
WHERE revenue_date BETWEEN ? AND ?
ORDER BY revenue_date ASC, branch_code ASC;

-- (c) Truy vấn SQL độc lập trực tiếp trên bảng gốc dùng EXISTS để đối soát chéo (queryIndependentVerifi
edRevenue)
SELECT COALESCE(SUM(tk.unit_price), 0.00) AS total_rev
FROM ticket tk
JOIN booking bk        ON bk.booking_id = tk.booking_id
JOIN screening sc      ON sc.screening_id = bk.screening_id
JOIN screening_room rm ON rm.room_id = sc.room_id
WHERE tk.ticket_status = 'VALID'
  AND bk.booking_status = 'CONFIRMED'
  AND EXISTS (
      SELECT 1
      FROM payment pm
      WHERE pm.booking_id = bk.booking_id
        AND pm.payment_status = 'SUCCESS'
        AND DATE(pm.paid_at) BETWEEN ? AND ?
  );
```

4.3. Kết quả chạy thực tế kỳ 2026-11-01 → 2026-11-15 trên cinema_chain_test

- CN-HN01 (CineViet Vincom Bà Triệu): 36 đơn, 84 vé, **10.140.000 VNĐ**.
- CN-HCM01 (CineViet Landmark 81): 34 đơn, 79 vé, **9.370.000 VNĐ**.
- CN-HP01 (CineViet Aeon Mall Hải Phòng): 29 đơn, 69 vé, **8.325.000 VNĐ**.
- CN-DN01 (CineViet Cầu Rồng Đà Nẵng): 26 đơn, 60 vé, **7.330.000 VNĐ**.
- **Tổng cộng toàn chuỗi: 125 đơn**, **292 vé**, **35.165.000 VNĐ** (sai lệch so với SQL độc lập = **0,00 VNĐ**).

5. Báo cáo Quản trị 2: Báo cáo Tình hình Bán vé & Tỷ lệ Lấp đầy (R7.2)

- **Định danh quản trị: Người lập:** Quản lý Chi nhánh → **Người nhận:** Ban Điều hành và Quản lý Kinh doanh.
- **Mục đích:** Thống kê hiệu quả khai thác từng suất chiếu theo ngày chiếu (screening_date) và chi nhánh: sức chứa phòng (room_capacity), số vé đã thanh toán thành công (tickets_sold), số ghế còn trống thực tế (available_seats — trừ cả các ghế đang giữ bởi đơn PENDING), tỷ lệ lấp đầy (occupancy_rate_pct %) và doanh thu bán vé (sales_revenue), xuất PDF tiếng Việt (reports/exports/BaoCaoBanVe_Mau.pdf).
- **Đầu vào:** fromDate, toDate, branchId (tùy chọn), screeningId (tùy chọn).
- **Bảng & View liên quan:** vw_screening_occupancy (tổng hợp từ screening, movie, screening_room, cinema_branch, seat, screening_seat_lock, vw_verified_booking_revenue).
- **Lớp mã nguồn Java:** com.cinema.feature.report.sales.TicketSalesReportRepository & TicketSalesReportService.

- **Ảnh chụp màn hình & PDF demo:** `verification/screenshots/07_ticket_sales_report_preview.png`, `09_exported_sales_pdf_page1.png`, `reports/exports/BaoCaoBanVe_Mau.pdf`.

5.1. Câu lệnh SQL tham số hóa (`?`) trong Java (`TicketSalesReportRepository.java`)

```
-- (a) Truy vấn chi tiết bán vé & tỷ lệ lấp đầy theo Suất chiếu (findScreeningSales)
SELECT
    screening_id, screening_date, start_time, end_time,
    branch_name, room_name, room_type, movie_title,
    room_capacity, tickets_sold, available_seats,
    occupancy_rate_pct, sales_revenue, screening_status
FROM vw_screening_occupancy
WHERE screening_date BETWEEN ? AND ?
  AND branch_id = ?
ORDER BY start_time ASC, screening_id ASC;

-- (b) Truy vấn SQL độc lập trên bảng gốc chuẩn 3NF (ticket -> booking -> screening) để kiểm chứng chéo
SELECT COALESCE(SUM(tk.unit_price), 0.00) AS total_sales_rev
FROM ticket tk
JOIN booking bk        ON bk.booking_id = tk.booking_id
JOIN screening sc      ON sc.screening_id = bk.screening_id
JOIN screening_room rm ON rm.room_id = sc.room_id
WHERE tk.ticket_status = 'VALID'
  AND bk.booking_status = 'CONFIRMED'
  AND DATE(sc.start_time) BETWEEN ? AND ?
  AND EXISTS (
      SELECT 1
      FROM payment pm
      WHERE pm.booking_id = bk.booking_id
        AND pm.payment_status = 'SUCCESS'
  );
```

5.2. Ví dụ SQL cụ thể chạy trên `cinema_chain_test` & Kết quả mong đợi

```
SELECT screening_id, movie_title, room_name, room_capacity,
       tickets_sold, available_seats, occupancy_rate_pct, sales_revenue
FROM vw_screening_occupancy
WHERE screening_date BETWEEN '2026-11-04' AND '2026-11-04'
  AND branch_id = 1
ORDER BY start_time ASC, screening_id ASC;
```

- **Kết quả thực tế (functional_sql_examples_test_db.log):** Trả về 06 suất chiếu ngày `04/11/2026` tại Chi nhánh #1 (VD: Suất `#3` Quật Mộ Trùng Ma: sức chứa `80`, đã bán `3` vé, còn trống `77` ghế, lấp đầy `3.75%`, doanh thu `330.000 VNĐ`; trên toàn kỳ `04/11/2026 -> 10/11/2026` toàn chuỗi khớp chính xác **292 vé** và **35.165.000 VNĐ**).

CHƯƠNG 6. TỔNG HỢP KẾT QUẢ KIỂM THỬ & BẢO TOÀN DỮ LIỆU TRÊN MYSQL 8.4.0 LTS

Hệ thống đã được thực thi và kiểm chứng độc lập trên hai cơ sở dữ liệu tách biệt thuộc cùng máy chủ chính thức **MySQL Community Server 8.4.0 LTS** (`SELECT VERSION() = 8.4.0`, `datadir = C:/Users/LNB/.mysql/data-8.4`, chế độ nghiêm ngặt `ONLY_FULL_GROUP_BY, STRICT_TRANS_TABLES`) và **JDK 21 LTS (Eclipse Temurin)**:

• **Trên CSDL Demo chính thức cinema_chain (14 bảng chuẩn 3NF):** Khởi tạo sạch qua `database/prepare_demo_db.ps1` (`14` bảng, `17` khóa ngoại, `12` Trigger, `04` View; doanh thu mẫu gốc `35.165.000,00 VNĐ`), khởi chạy trực tiếp `com.cinema.app.Main` theo cấu hình `config/application.properties` (`Mode=NORMAL_READ_ONLY_INIT | DestructiveReset=DISABLED`), kiểm chứng đầy đủ kịch bản hợp lệ & bị từ chối trên cả 5 màn hình, xuất 2 tệp PDF đối chiếu SQL độc lập (`35.375.000,00 VNĐ`, sai lệch `0,00 VNĐ`) và đóng/mở ứng dụng qua 2 tiến trình JVM xác nhận dữ liệu demo vừa nhập được giữ nguyên 100%.

• **Trên CSDL Kiểm thử tự động cô lập cinema_chain_test (14 bảng chuẩn 3NF):** Chạy tươi tại thời điểm `2026-11-07 15:15:43` (sau khi xóa sạch toàn bộ `TEST-*.xml` cũ) đạt **40/40 bài kiểm thử PASS (0 Skipped)**; đồng thời dấu vân tay nội dung SHA-256 của CSDL demo `cinema_chain` trước và sau kiểm thử đều khớp tuyệt đối `7af5d0052ade56eedd37184d82ce7a00a178bbcaa7806f828e06ed23aaa66d37`.

Báo cáo phân biệt minh bạch 3 cấp độ kiểm chứng: (a) Kiểm thử tích hợp tầng `Service`/`Repository` trên MySQL 8.4 thật; (b) Kiểm chứng giao diện Swing tự động trên Event Dispatch Thread qua 2 tiến trình JVM độc lập; và (c) Thao tác bấm chuột/bàn phím vật lý của người trình bày theo checklist tại `DEMO_GUIDE.md`.

Bộ Kiểm thử (Test Class)	Cổng Kiểm chứng	Số Test	Kết quả	Nội dung Kiểm chứng Cốt lõi trên MySQL 8.4.0 LTS
`ArchitectureComplianceTest`	Gate B (Kiến trúc)	05	**PASS (5/5)**	Cấm SQL trong UI, tách biệt phân tầng, cấm SQL phá hủy (`DROP`/`TRUNCATE`/`DELETE`) & gọi tiến trình trong `src/main/java`
`TestSafetyGuardTest`	Gate B (An toàn & CLI)	06	**PASS (6/6)**	Chặn `cinema_chain`/MariaDB/`root`; `setup.ps1` chặn `-CleanReset`; `verify_all.ps1` chỉ đọc; `mysql_spy` chứng minh không lộ mật khẩu CLI/env
`ScreeningValidationTest`	Gate A (Đơn vị)	05	**PASS (5/5)**	Kiểm tra hợp lệ thời lượng suất chiếu, giá cơ sở dương, danh sách ghế, thông báo tiếng Việt
`DatabaseIntegrityIT`	Gate C (3NF & MySQL 8.4)	15	**PASS (15/15)**	Kiểm chứng `ticket` chuẩn 3NF & `screening_seat_lock`, SHA-256 phát hiện sửa 1 dòng, 12 Trigger chống trùng lịch/sai phòng/đổi phòng đã có vé/thanh toán đúp, giá lịch sử, UTF-8

Bộ Kiểm thử (Test Class)	Cổng Kiểm chứng	Số Test	Kết quả	Nội dung Kiểm chứng Cốt lõi trên MySQL 8.4.0 LTS
`ScreeningConcurrencyIT`	Gate C (Đồng thời)	02	**PASS (2/2)**	12 luồng `CREATE` đồng thời và 8 luồng `UPDATE` đồng thời suất chiếu trùng giờ → đúng 01 thắng + chặn đổi phòng
`BookingConcurrencyIT`	Gate C (Đồng thời)	01	**PASS (1/1)**	20 luồng đặt đồng thời cùng 1 ghế → đúng 01 thắng, 0 bản ghi mồ côi; hủy đơn giải phóng `screening_seat_lock` để bán lại ghế
`ScreeningSearchIT`	Gate D (Tra cứu & Trang)	03	**PASS (3/3)**	Tra cứu đa điều kiện kèm số ghế trống, hiển thị đủ suất `OPEN` và duyệt phân trang `50 dòng/trang` không trùng/sót dòng
`ReportAccuracyIT`	Gate E (Báo cáo & PDF)	03	**PASS (3/3)**	Đối chiếu 2 báo cáo & PDF tiếng Việt với SQL độc lập (`35.165.000 VNĐ`, sai lệch `0.00 VNĐ`) và Known-Answer Fixture
TỔNG CỘNG	**Gates A – E**	**40**	**PASS (40/40)**	**0 lỗi biên dịch, 0 Failed, 0 Errors, 0 Skipped (2026-11-07 15:15:43), bảo toàn SHA-256 cinema_chain**

PHỤ LỤC A1. MÃ LỆNH DDL KHỞI TẠO 14 BẢNG QUAN HỆ CHUẨN 3NF TRÊN MYSQL 8.4 (01_schema.sql)

```
-- ========================================================================
-- TỆP: 01_schema.sql
-- DỰ ÁN: HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM
-- BƯỚC KHỞI TẠO: [Bước 1 - Tạo Bảng] Định nghĩa 14 bảng đạt chuẩn 3NF thực sự với
--                PRIMARY KEY, UNIQUE, CHECK, chỉ mục vật lý có điều kiện và INDEX.
-- THIẾT KẾ CHUẨN MỰC:
--   - KHÔNG sử dụng `DROP TABLE` hay `SET FOREIGN_KEY_CHECKS = 0` để tránh
--     xóa nhầm dữ liệu khi chạy ở chế độ khởi tạo không phá hủy.
--   - Bảng `ticket` đã loại bỏ hoàn toàn thuộc tính bắc cầu `screening_id`
--     (vì `ticket_id -> booking_id -> screening_id`), đảm bảo đạt chuẩn 3NF chặt chẽ.
--   - Bảng `screening_seat_lock` (quan hệ phân bổ ghế đang hiệu lực) bảo đảm
--     chống bán trùng ghế `(screening_id, seat_id)` ngay ở mức B-Tree PRIMARY KEY
--     của MySQL 8.4 InnoDB đồng thời cho phép bán lại ghế sau khi vé hủy (`CANCELLED`).
--   - Các ràng buộc Khóa ngoại (17 FOREIGN KEY) được tách sang `01b_foreign_keys.sql`.
-- TƯƠNG THÍCH: MySQL Community Server 8.4 LTS (InnoDB, utf8mb4_unicode_ci)
-- ========================================================================

SET NAMES utf8mb4;

-- ------------------------------------------------------------------------
-- 1. Bảng cinema_branch (Chi nhánh rạp chiếu phim)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS cinema_branch (
    branch_id    INT          NOT NULL AUTO_INCREMENT,
    branch_code  VARCHAR(20)  NOT NULL,
    branch_name  VARCHAR(120) NOT NULL,
    address      VARCHAR(255) NOT NULL,
    city         VARCHAR(80)  NOT NULL,
    phone        VARCHAR(20)  NOT NULL,
    active       TINYINT(1)   NOT NULL DEFAULT 1,
    CONSTRAINT pk_cinema_branch  PRIMARY KEY (branch_id),
    CONSTRAINT uq_branch_code    UNIQUE (branch_code),
    CONSTRAINT uq_branch_name    UNIQUE (branch_name),
    CONSTRAINT uq_branch_phone   UNIQUE (phone),
    CONSTRAINT chk_branch_active CHECK (active IN (0, 1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 2. Bảng screening_room (Phòng chiếu phim thuộc chi nhánh)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS screening_room (
    room_id    INT         NOT NULL AUTO_INCREMENT,
    branch_id  INT         NOT NULL,
    room_name  VARCHAR(60) NOT NULL,
    room_type  VARCHAR(20) NOT NULL DEFAULT '2D',
    active     TINYINT(1)  NOT NULL DEFAULT 1,
    CONSTRAINT pk_screening_room   PRIMARY KEY (room_id),
    CONSTRAINT uq_room_branch_name UNIQUE (branch_id, room_name),
    CONSTRAINT chk_room_type       CHECK (room_type IN ('2D', '3D', 'IMAX')),
    CONSTRAINT chk_room_active     CHECK (active IN (0, 1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 3. Bảng seat_type (Danh mục hạng ghế & mức phụ thu chuẩn)
-- ------------------------------------------------------------------------

CREATE TABLE IF NOT EXISTS seat_type (
    seat_type_code VARCHAR(16)   NOT NULL,
    type_name      VARCHAR(60)   NOT NULL,
    surcharge      DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    CONSTRAINT pk_seat_type       PRIMARY KEY (seat_type_code),
    CONSTRAINT uq_seat_type_name  UNIQUE (type_name),
    CONSTRAINT chk_seat_type_code CHECK (seat_type_code IN ('STD', 'VIP', 'COUPLE')),
    CONSTRAINT chk_seat_surcharge CHECK (surcharge >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 4. Bảng seat (Ghế vật lý cố định trong từng phòng chiếu)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS seat (
```

```
    seat_id        INT         NOT NULL AUTO_INCREMENT,
    room_id        INT         NOT NULL,
    seat_row       CHAR(1)     NOT NULL,
    seat_number    INT         NOT NULL,
    seat_type_code VARCHAR(16) NOT NULL DEFAULT 'STD',
    CONSTRAINT pk_seat          PRIMARY KEY (seat_id),
    CONSTRAINT uq_seat_position UNIQUE (room_id, seat_row, seat_number),
    CONSTRAINT chk_seat_row     CHECK (seat_row >= 'A' AND seat_row <= 'Z'),
    CONSTRAINT chk_seat_number  CHECK (seat_number >= 1 AND seat_number <= 50),
    INDEX idx_seat_type (seat_type_code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------------
-- 5. Bảng movie (Phim điện ảnh)
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS movie (
    movie_id         INT          NOT NULL AUTO_INCREMENT,
    movie_code       VARCHAR(20)  NOT NULL,
    title            VARCHAR(180) NOT NULL,
    duration_minutes INT          NOT NULL,
    release_date     DATE         NOT NULL,
    age_rating       VARCHAR(10)  NOT NULL DEFAULT 'P',
    country          VARCHAR(80)  NOT NULL,
    description      TEXT         NULL,
    CONSTRAINT pk_movie         PRIMARY KEY (movie_id),
    CONSTRAINT uq_movie_code    UNIQUE (movie_code),
    CONSTRAINT chk_movie_dur    CHECK (duration_minutes >= 1 AND duration_minutes <= 400),
    CONSTRAINT chk_movie_rating CHECK (age_rating IN ('P', 'K', 'T13', 'T16', 'T18'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------------
-- 6. Bảng genre (Thể loại phim)
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS genre (
    genre_id    INT          NOT NULL AUTO_INCREMENT,
    genre_name  VARCHAR(80)  NOT NULL,
    description VARCHAR(255) NULL,
    CONSTRAINT pk_genre      PRIMARY KEY (genre_id),
    CONSTRAINT uq_genre_name UNIQUE (genre_name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------------
-- 7. Bảng movie_genre (Liên kết N-N giữa Phim và Thể loại)
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS movie_genre (
    movie_id INT NOT NULL,
    genre_id INT NOT NULL,
    CONSTRAINT pk_movie_genre PRIMARY KEY (movie_id, genre_id),
    INDEX idx_mg_genre (genre_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------------
-- 8. Bảng screening (Suất chiếu)
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS screening (
    screening_id INT           NOT NULL AUTO_INCREMENT,
    movie_id     INT           NOT NULL,
    room_id      INT           NOT NULL,
    start_time   DATETIME      NOT NULL,
    end_time     DATETIME      NOT NULL,
    base_price   DECIMAL(12,2) NOT NULL,
    status       VARCHAR(20)   NOT NULL DEFAULT 'OPEN',
    CONSTRAINT pk_screening        PRIMARY KEY (screening_id),
    CONSTRAINT chk_screening_time  CHECK (end_time > start_time),
    CONSTRAINT chk_screening_price CHECK (base_price > 0),
    CONSTRAINT chk_screening_stat  CHECK (status IN ('OPEN', 'CANCELLED')),
    INDEX idx_screening_room_time (room_id, start_time, end_time),
    INDEX idx_screening_movie_time (movie_id, start_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------------
-- 9. Bảng customer (Khách hàng thành viên)
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS customer (
    customer_id INT          NOT NULL AUTO_INCREMENT,
```

```
    full_name   VARCHAR(120) NOT NULL,
    phone       VARCHAR(20)  NOT NULL,
    email       VARCHAR(120) NOT NULL,
    birth_date  DATE         NULL,
    created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_customer       PRIMARY KEY (customer_id),
    CONSTRAINT uq_customer_phone UNIQUE (phone),
    CONSTRAINT uq_customer_email UNIQUE (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 10. Bảng employee (Nhân viên làm việc tại chi nhánh)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS employee (
    employee_id   INT          NOT NULL AUTO_INCREMENT,
    branch_id     INT          NOT NULL,
    employee_code VARCHAR(20)  NOT NULL,
    full_name     VARCHAR(120) NOT NULL,
    phone         VARCHAR(20)  NOT NULL,
    role          VARCHAR(20)  NOT NULL DEFAULT 'CASHIER',
    hired_date    DATE         NOT NULL,

    CONSTRAINT pk_employee       PRIMARY KEY (employee_id),
    CONSTRAINT uq_employee_code  UNIQUE (employee_code),
    CONSTRAINT uq_employee_phone UNIQUE (phone),
    CONSTRAINT chk_employee_role CHECK (role IN ('CASHIER', 'MANAGER', 'ACCOUNTANT')),
    INDEX idx_employee_branch (branch_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 11. Bảng booking (Đơn đặt vé; customer_id NULLable cho Khách vãng lai)
--     Mỗi booking gắn với đúng 1 screening_id (booking_id -> screening_id).
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS booking (
    booking_id     INT         NOT NULL AUTO_INCREMENT,
    booking_code   VARCHAR(30) NOT NULL,
    screening_id   INT         NOT NULL,
    customer_id    INT         NULL,
    employee_id    INT         NOT NULL,
    booked_at      DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    booking_status VARCHAR(20) NOT NULL DEFAULT 'CONFIRMED',
    CONSTRAINT pk_booking         PRIMARY KEY (booking_id),
    CONSTRAINT uq_booking_code    UNIQUE (booking_code),
    CONSTRAINT chk_booking_status CHECK (booking_status IN ('CONFIRMED', 'PENDING', 'CANCELLED')),
    INDEX idx_booking_screening (screening_id, booking_status),
    INDEX idx_booking_customer (customer_id),
    INDEX idx_booking_employee (employee_id),
    INDEX idx_booking_booked_at (booked_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 12. Bảng ticket (Vé xem phim chi tiết theo ghế – Đạt chuẩn 3NF / BCNF)
--     - Không lưu dư thừa `screening_id` (suy ra bảo toàn qua `booking.screening_id`).
--     - Khóa dự tuyển tối tiểu trên toàn bảng: `{ticket_id}` (PK) và `{booking_id, seat_id}` (UQ).
--     - Lưu giữ đầy đủ lịch sử vé hủy (`CANCELLED`) và đơn giá chốt lịch sử (`unit_price`).
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ticket (
    ticket_id     INT           NOT NULL AUTO_INCREMENT,
    booking_id    INT           NOT NULL,
    seat_id       INT           NOT NULL,
    unit_price    DECIMAL(12,2) NOT NULL,
    ticket_status VARCHAR(20)   NOT NULL DEFAULT 'VALID',
    cancelled_at  DATETIME      NULL,
    CONSTRAINT pk_ticket              PRIMARY KEY (ticket_id),
    CONSTRAINT uq_ticket_booking_seat UNIQUE (booking_id, seat_id),
    CONSTRAINT chk_ticket_price       CHECK (unit_price >= 0),
    CONSTRAINT chk_ticket_status      CHECK (ticket_status IN ('VALID', 'CANCELLED')),
    INDEX idx_ticket_booking_status (booking_id, ticket_status),
    INDEX idx_ticket_seat (seat_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 13. Bảng screening_seat_lock (Phân bổ khóa ghế đang hiệu lực theo suất chiếu – 3NF / BCNF)
--     - Biểu diễn tập ghế đang bị giữ bởi các vé có `ticket_status = 'VALID'`.
--     - Khóa dự tuyển tối tiểu trên toàn bảng: `{screening_id, seat_id}` (PK) và `{ticket_id}` (UQ).
```

```
--     - Đảm bảo chống bán trùng ghế ở mức chỉ mục B-Tree InnoDB cho cả giao dịch đồng thời,
--       đồng thời tự động giải phóng khóa ghế khi vé chuyển sang `CANCELLED` để bán lại ghế.

-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS screening_seat_lock (
    screening_id INT NOT NULL,
    seat_id      INT NOT NULL,
    ticket_id    INT NOT NULL,
    CONSTRAINT pk_screening_seat_lock PRIMARY KEY (screening_id, seat_id),
    CONSTRAINT uq_ssl_ticket          UNIQUE (ticket_id),
    INDEX idx_ssl_seat (seat_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------------
-- 14. Bảng payment (Giao dịch thanh toán)
--     - Khóa dự tuyển tối tiểu của quan hệ logic: duy nhất `{payment_id}` (PK).
--     - Cột sinh vật lý `active_success_booking` là cơ chế chỉ mục duy nhất có điều kiện
--       của MySQL 8.4 trên tập con `payment_status = 'SUCCESS'` (không phải khóa của toàn bảng),
--       nhằm ngăn thanh toán thành công trùng lặp cho cùng một `booking_id`.
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS payment (
    payment_id             INT           NOT NULL AUTO_INCREMENT,
    booking_id             INT           NOT NULL,
    amount                 DECIMAL(12,2) NOT NULL,
    payment_method         VARCHAR(20)   NOT NULL DEFAULT 'CASH',
    payment_status         VARCHAR(20)   NOT NULL DEFAULT 'SUCCESS',
    paid_at                DATETIME      NULL,
    transaction_ref        VARCHAR(60)   NULL,
    active_success_booking INT           GENERATED ALWAYS AS (
        CASE WHEN payment_status = 'SUCCESS' THEN booking_id ELSE NULL END
    ) STORED,
    CONSTRAINT pk_payment             PRIMARY KEY (payment_id),
    CONSTRAINT uq_payment_one_success UNIQUE (active_success_booking),
    CONSTRAINT chk_payment_amount     CHECK (amount >= 0),
    CONSTRAINT chk_payment_method     CHECK (payment_method IN ('CASH', 'CARD', 'TRANSFER', 'EWALLET')),
    CONSTRAINT chk_payment_status     CHECK (payment_status IN ('PENDING', 'SUCCESS', 'FAILED', 'REFUNDE
D')),
    INDEX idx_payment_status_paid_at (payment_status, paid_at),
    INDEX idx_payment_booking (booking_id, payment_status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

PHỤ LỤC A2. MÃ LỆNH THIẾT LẬP 17 RÀNG BUỘC KHÓA NGOẠI (01b_foreign_keys.sql)

```
-- ========================================================================
-- TỆP: 01b_foreign_keys.sql
-- DỰ ÁN: HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM
-- BƯỚC KHỞI TẠO: [Bước 2 - Tạo Khóa ngoại] Thiết lập toàn bộ 17 ràng buộc
--                tham chiếu FOREIGN KEY giữa 14 bảng chuẩn 3NF.
-- LƯU Ý TƯƠNG THÍCH MYSQL 8.4 LTS:
--   - Bảng `payment` sử dụng cột sinh vật lý (`GENERATED ALWAYS AS ... STORED`)
--     phục vụ chỉ mục duy nhất có điều kiện cho dòng `SUCCESS`. Theo quy định
--     của MySQL 8.4 InnoDB (mã lỗi 3108), cột cơ sở `booking_id` tham gia biểu thức
--     STORED GENERATED dùng `ON DELETE RESTRICT ON UPDATE RESTRICT`.
-- ========================================================================

SET NAMES utf8mb4;

-- 1. Phòng chiếu thuộc Chi nhánh rạp (1 - N)
ALTER TABLE screening_room
    ADD CONSTRAINT fk_room_branch FOREIGN KEY (branch_id)
        REFERENCES cinema_branch (branch_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 2. Ghế vật lý thuộc Phòng chiếu & phân loại theo Hạng ghế
ALTER TABLE seat
    ADD CONSTRAINT fk_seat_room FOREIGN KEY (room_id)
        REFERENCES screening_room (room_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    ADD CONSTRAINT fk_seat_type FOREIGN KEY (seat_type_code)
        REFERENCES seat_type (seat_type_code)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 3. Bảng trung gian Phim - Thể loại (N - N)
ALTER TABLE movie_genre
    ADD CONSTRAINT fk_mg_movie FOREIGN KEY (movie_id)
        REFERENCES movie (movie_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    ADD CONSTRAINT fk_mg_genre FOREIGN KEY (genre_id)
        REFERENCES genre (genre_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 4. Suất chiếu gắn với Phim và Phòng chiếu
ALTER TABLE screening
    ADD CONSTRAINT fk_screening_movie FOREIGN KEY (movie_id)
        REFERENCES movie (movie_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    ADD CONSTRAINT fk_screening_room FOREIGN KEY (room_id)
        REFERENCES screening_room (room_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 5. Nhân viên làm việc tại Chi nhánh rạp
ALTER TABLE employee
    ADD CONSTRAINT fk_employee_branch FOREIGN KEY (branch_id)
        REFERENCES cinema_branch (branch_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 6. Đơn đặt vé gắn với Suất chiếu, Khách hàng (NULLable cho khách vãng lai) & Nhân viên
ALTER TABLE booking

    ADD CONSTRAINT fk_booking_screening FOREIGN KEY (screening_id)
        REFERENCES screening (screening_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    ADD CONSTRAINT fk_booking_customer FOREIGN KEY (customer_id)
        REFERENCES customer (customer_id)
        ON DELETE SET NULL ON UPDATE CASCADE,
    ADD CONSTRAINT fk_booking_employee FOREIGN KEY (employee_id)
        REFERENCES employee (employee_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 7. Vé xem phim chuẩn 3NF gắn với Đơn đặt vé & Ghế vật lý
ALTER TABLE ticket
    ADD CONSTRAINT fk_ticket_booking FOREIGN KEY (booking_id)
        REFERENCES booking (booking_id)
```

```
        ON DELETE RESTRICT ON UPDATE CASCADE,
    ADD CONSTRAINT fk_ticket_seat FOREIGN KEY (seat_id)
        REFERENCES seat (seat_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 8. Khóa phân bổ ghế hiệu lực gắn với Suất chiếu, Ghế vật lý & Vé hợp lệ
ALTER TABLE screening_seat_lock
    ADD CONSTRAINT fk_ssl_screening FOREIGN KEY (screening_id)
        REFERENCES screening (screening_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    ADD CONSTRAINT fk_ssl_seat FOREIGN KEY (seat_id)
        REFERENCES seat (seat_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    ADD CONSTRAINT fk_ssl_ticket FOREIGN KEY (ticket_id)
        REFERENCES ticket (ticket_id)
        ON DELETE CASCADE ON UPDATE CASCADE;

-- 9. Giao dịch thanh toán gắn với Đơn đặt vé
ALTER TABLE payment
    ADD CONSTRAINT fk_payment_booking FOREIGN KEY (booking_id)
        REFERENCES booking (booking_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT;
```

PHỤ LỤC B. MÃ LỆNH 12 TRIGGER ĐỒNG BỘ KHÓA GHẾ & BẢO VỆ TOÀN VẸN NGHIỆP VỤ (02_triggers.sql)

```
-- ============================================================================
-- TỆP: 02_triggers.sql
-- DỰ ÁN: HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM
-- BƯỚC KHỞI TẠO: [Bước 3 - Tạo Trigger] Thiết lập 12 Trigger bảo vệ toàn vẹn
--                nghiệp vụ liên bảng trực tiếp ở tầng Hệ quản trị MySQL 8.4 LTS:
--   1. Chống trùng lịch chiếu trong cùng phòng & đảm bảo thời lượng tối thiểu.
--   2. Khóa không cho đổi `booking.screening_id` khi đơn đặt vé đã phát sinh vé.
--   3. Đảm bảo ghế trên vé thuộc đúng phòng chiếu của suất chiếu (qua `booking`).
--   4. Tự động đồng bộ bảng khóa ghế `screening_seat_lock` khi thêm/hủy vé và
--      ngăn chặn mọi thao tác can thiệp sai lệch trực tiếp vào `screening_seat_lock`.
--   5. Đảm bảo tính nhất quán của thời điểm hủy vé và thời điểm thanh toán.
-- ============================================================================

SET NAMES utf8mb4;

DROP TRIGGER IF EXISTS trg_screening_validate_ins;
DROP TRIGGER IF EXISTS trg_screening_validate_upd;
DROP TRIGGER IF EXISTS trg_booking_validate_upd;
DROP TRIGGER IF EXISTS trg_ticket_validate_ins;
DROP TRIGGER IF EXISTS trg_ticket_validate_upd;
DROP TRIGGER IF EXISTS trg_ticket_lock_after_ins;
DROP TRIGGER IF EXISTS trg_ticket_lock_after_upd;
DROP TRIGGER IF EXISTS trg_ssl_validate_ins;
DROP TRIGGER IF EXISTS trg_ssl_validate_upd;
DROP TRIGGER IF EXISTS trg_ssl_validate_del;
DROP TRIGGER IF EXISTS trg_payment_validate_ins;
DROP TRIGGER IF EXISTS trg_payment_validate_upd;

DELIMITER $$

-- ----------------------------------------------------------------------------
-- 1. Trigger kiểm tra khi THÊM MỚI suất chiếu (screening)
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_screening_validate_ins
BEFORE INSERT ON screening
FOR EACH ROW
BEGIN
    DECLARE v_duration INT DEFAULT 0;
    DECLARE v_room_active TINYINT DEFAULT 0;
    DECLARE v_overlap_count INT DEFAULT 0;

    IF NEW.end_time <= NEW.start_time THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-05]: Thoi gian ket thuc phai lon hon thoi gian bat dau
.';
    END IF;

    IF NEW.base_price <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-04]: Gia ve co so cua suat chieu phai lon hon 0.';
    END IF;

    SELECT duration_minutes INTO v_duration
    FROM movie
    WHERE movie_id = NEW.movie_id;

    IF TIMESTAMPDIFF(MINUTE, NEW.start_time, NEW.end_time) < v_duration THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-05]: Khoang thoi gian suat chieu nho hon thoi luong ph
im.';
    END IF;

    SELECT active INTO v_room_active
    FROM screening_room
    WHERE room_id = NEW.room_id;

    IF v_room_active = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-01]: Phong chieu dang tam ngung hoat dong.';
    END IF;
```

```
    IF NEW.status = 'OPEN' THEN
        SELECT COUNT(*) INTO v_overlap_count
        FROM screening s
        WHERE s.room_id = NEW.room_id
          AND s.status = 'OPEN'
          AND s.start_time < NEW.end_time
          AND s.end_time > NEW.start_time;

        IF v_overlap_count > 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Loi nghiep vu [BR-06]: Trung lich chieu voi mot suat chieu khac tron
g cung phong chieu.';
        END IF;
    END IF;
END$$

-- ------------------------------------------------------------------------
-- 2. Trigger kiểm tra khi CẬP NHẬT suất chiếu (screening)
-- ------------------------------------------------------------------------
CREATE TRIGGER trg_screening_validate_upd
BEFORE UPDATE ON screening
FOR EACH ROW
BEGIN
    DECLARE v_duration INT DEFAULT 0;
    DECLARE v_room_active TINYINT DEFAULT 0;
    DECLARE v_overlap_count INT DEFAULT 0;
    DECLARE v_ticket_count INT DEFAULT 0;
    DECLARE v_valid_ticket_count INT DEFAULT 0;

    IF NEW.end_time <= NEW.start_time THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-05]: Thoi gian ket thuc phai lon hon thoi gian bat dau
.';
    END IF;

    IF NEW.base_price <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-04]: Gia ve co so cua suat chieu phai lon hon 0.';
    END IF;

    SELECT active INTO v_room_active
    FROM screening_room
    WHERE room_id = NEW.room_id;

    IF v_room_active = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-01]: Phong chieu dang tam ngung hoat dong.';
    END IF;

    IF NEW.room_id <> OLD.room_id THEN
        SELECT COUNT(*) INTO v_ticket_count
        FROM ticket tk
        JOIN booking bk ON bk.booking_id = tk.booking_id
        WHERE bk.screening_id = OLD.screening_id;

        IF v_ticket_count > 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Loi nghiep vu [BR-09]: Khong the doi phong chieu cho suat chieu da p
hat sinh ve.';
        END IF;
    END IF;

    IF OLD.status = 'OPEN' AND NEW.status = 'CANCELLED' THEN
        SELECT COUNT(*) INTO v_valid_ticket_count
        FROM ticket tk
        JOIN booking bk ON bk.booking_id = tk.booking_id
        WHERE bk.screening_id = OLD.screening_id
          AND tk.ticket_status = 'VALID';

        IF v_valid_ticket_count > 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Loi nghiep vu [BR-08]: Khong the huy suat chieu dang co ve hop le ch
ua duoc hoan huy.';
        END IF;
    END IF;
```

```
    SELECT duration_minutes INTO v_duration
    FROM movie
    WHERE movie_id = NEW.movie_id;

    IF TIMESTAMPDIFF(MINUTE, NEW.start_time, NEW.end_time) < v_duration THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-05]: Khoang thoi gian suat chieu nho hon thoi luong ph
im.';
    END IF;

    IF NEW.status = 'OPEN' THEN
        SELECT COUNT(*) INTO v_overlap_count
        FROM screening s
        WHERE s.room_id = NEW.room_id
          AND s.screening_id <> NEW.screening_id
          AND s.status = 'OPEN'
          AND s.start_time < NEW.end_time
          AND s.end_time > NEW.start_time;

        IF v_overlap_count > 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Loi nghiep vu [BR-06]: Trung lich chieu voi mot suat chieu khac tron
g cung phong chieu.';
        END IF;
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 3. Trigger bảo vệ bất biến đơn đặt vé khi CẬP NHẬT booking
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_booking_validate_upd
BEFORE UPDATE ON booking
FOR EACH ROW
BEGIN
    DECLARE v_ticket_count INT DEFAULT 0;

    IF NEW.screening_id <> OLD.screening_id THEN
        SELECT COUNT(*) INTO v_ticket_count
        FROM ticket
        WHERE booking_id = OLD.booking_id;

        IF v_ticket_count > 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Loi nghiep vu [BR-08]: Khong the doi suat chieu cua don dat ve da ph
at sinh ve.';
        END IF;
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 4. Trigger kiểm tra khi THÊM MỚI vé xem phim (ticket — 3NF)
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_ticket_validate_ins
BEFORE INSERT ON ticket
FOR EACH ROW
BEGIN
    DECLARE v_booking_scr_id INT DEFAULT NULL;
    DECLARE v_scr_room_id INT DEFAULT NULL;
    DECLARE v_scr_status VARCHAR(20) DEFAULT NULL;
    DECLARE v_seat_room_id INT DEFAULT NULL;

    SELECT screening_id INTO v_booking_scr_id
    FROM booking
    WHERE booking_id = NEW.booking_id;

    IF v_booking_scr_id IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-08]: Don dat ve khong ton tai.';
    END IF;

    SELECT room_id, status INTO v_scr_room_id, v_scr_status
    FROM screening
    WHERE screening_id = v_booking_scr_id;

    SELECT room_id INTO v_seat_room_id
```

```
    FROM seat
    WHERE seat_id = NEW.seat_id;

    IF v_scr_room_id IS NULL OR v_seat_room_id IS NULL OR v_scr_room_id <> v_seat_room_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-09]: Ghe duoc chon khong thuoc phong chieu cua suat ch
ieu nay.';
    END IF;

    IF NEW.ticket_status = 'VALID' AND v_scr_status <> 'OPEN' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-08]: Khong the phat hanh ve hop le cho suat chieu da b
i huy.';
    END IF;

    IF NEW.ticket_status = 'CANCELLED' AND NEW.cancelled_at IS NULL THEN
        SET NEW.cancelled_at = CURRENT_TIMESTAMP;
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 5. Trigger kiểm tra khi CẬP NHẬT vé xem phim (ticket – 3NF)
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_ticket_validate_upd
BEFORE UPDATE ON ticket
FOR EACH ROW
BEGIN
    DECLARE v_booking_scr_id INT DEFAULT NULL;
    DECLARE v_scr_room_id INT DEFAULT NULL;
    DECLARE v_scr_status VARCHAR(20) DEFAULT NULL;
    DECLARE v_seat_room_id INT DEFAULT NULL;

    SELECT screening_id INTO v_booking_scr_id
    FROM booking
    WHERE booking_id = NEW.booking_id;

    SELECT room_id, status INTO v_scr_room_id, v_scr_status
    FROM screening
    WHERE screening_id = v_booking_scr_id;

    SELECT room_id INTO v_seat_room_id
    FROM seat
    WHERE seat_id = NEW.seat_id;

    IF v_scr_room_id IS NULL OR v_seat_room_id IS NULL OR v_scr_room_id <> v_seat_room_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-09]: Ghe duoc chon khong thuoc phong chieu cua suat ch
ieu nay.';
    END IF;

    IF NEW.ticket_status = 'VALID' AND v_scr_status <> 'OPEN' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-08]: Khong the khoi phuc ve hop le cho suat chieu da b
i huy.';
    END IF;

    IF NEW.ticket_status = 'CANCELLED' AND NEW.cancelled_at IS NULL THEN
        SET NEW.cancelled_at = CURRENT_TIMESTAMP;
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 6. Trigger đồng bộ khóa ghế sau khi THÊM MỚI vé hợp lệ (ticket -> screening_seat_lock)
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_ticket_lock_after_ins
AFTER INSERT ON ticket
FOR EACH ROW
BEGIN
    DECLARE v_booking_scr_id INT DEFAULT NULL;

    IF NEW.ticket_status = 'VALID' THEN
        SELECT screening_id INTO v_booking_scr_id
        FROM booking
        WHERE booking_id = NEW.booking_id;
```

```
        INSERT INTO screening_seat_lock (screening_id, seat_id, ticket_id)
        VALUES (v_booking_scr_id, NEW.seat_id, NEW.ticket_id);
    END IF;
END$$

-- ------------------------------------------------------------------------
-- 7. Trigger đồng bộ khóa ghế sau khi CẬP NHẬT vé (giải phóng ghế khi CANCELLED)
-- ------------------------------------------------------------------------
CREATE TRIGGER trg_ticket_lock_after_upd
AFTER UPDATE ON ticket
FOR EACH ROW
BEGIN
    DECLARE v_booking_scr_id INT DEFAULT NULL;

    IF OLD.ticket_status = 'VALID' AND NEW.ticket_status = 'CANCELLED' THEN
        DELETE FROM screening_seat_lock
        WHERE ticket_id = OLD.ticket_id;
    ELSEIF OLD.ticket_status = 'CANCELLED' AND NEW.ticket_status = 'VALID' THEN
        SELECT screening_id INTO v_booking_scr_id
        FROM booking
        WHERE booking_id = NEW.booking_id;

        INSERT INTO screening_seat_lock (screening_id, seat_id, ticket_id)
        VALUES (v_booking_scr_id, NEW.seat_id, NEW.ticket_id);
    ELSEIF OLD.ticket_status = 'VALID' AND NEW.ticket_status = 'VALID'
       AND (OLD.seat_id <> NEW.seat_id OR OLD.booking_id <> NEW.booking_id) THEN
        SELECT screening_id INTO v_booking_scr_id
        FROM booking
        WHERE booking_id = NEW.booking_id;

        UPDATE screening_seat_lock
        SET screening_id = v_booking_scr_id,
            seat_id = NEW.seat_id

        WHERE ticket_id = OLD.ticket_id;
    END IF;
END$$

-- ------------------------------------------------------------------------
-- 8. Trigger bảo vệ toàn vẹn khi THÊM MỚI trực tiếp vào screening_seat_lock
-- ------------------------------------------------------------------------
CREATE TRIGGER trg_ssl_validate_ins
BEFORE INSERT ON screening_seat_lock
FOR EACH ROW
BEGIN
    DECLARE v_tk_seat_id INT DEFAULT NULL;
    DECLARE v_tk_status VARCHAR(20) DEFAULT NULL;
    DECLARE v_bk_scr_id INT DEFAULT NULL;

    SELECT tk.seat_id, tk.ticket_status, bk.screening_id
    INTO v_tk_seat_id, v_tk_status, v_bk_scr_id
    FROM ticket tk
    JOIN booking bk ON bk.booking_id = tk.booking_id
    WHERE tk.ticket_id = NEW.ticket_id;

    IF v_tk_status IS NULL OR v_tk_status <> 'VALID' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-10]: Chi duoc phep khoa ghe cho ve dang o trang thai V
ALID.';
    END IF;

    IF v_tk_seat_id <> NEW.seat_id OR v_bk_scr_id <> NEW.screening_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-10]: Thong tin khoa ghe khong khop voi suat chieu va g
he cua ve.';
    END IF;
END$$

-- ------------------------------------------------------------------------
-- 9. Trigger bảo vệ toàn vẹn khi CẬP NHẬT trực tiếp vào screening_seat_lock
-- ------------------------------------------------------------------------
CREATE TRIGGER trg_ssl_validate_upd
BEFORE UPDATE ON screening_seat_lock
FOR EACH ROW
BEGIN
```

```
    DECLARE v_tk_seat_id INT DEFAULT NULL;
    DECLARE v_tk_status VARCHAR(20) DEFAULT NULL;
    DECLARE v_bk_scr_id INT DEFAULT NULL;

    SELECT tk.seat_id, tk.ticket_status, bk.screening_id
    INTO v_tk_seat_id, v_tk_status, v_bk_scr_id
    FROM ticket tk
    JOIN booking bk ON bk.booking_id = tk.booking_id
    WHERE tk.ticket_id = NEW.ticket_id;

    IF v_tk_status IS NULL OR v_tk_status <> 'VALID' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-10]: Chi duoc phep khoa ghe cho ve dang o trang thai V
ALID.';
    END IF;

    IF v_tk_seat_id <> NEW.seat_id OR v_bk_scr_id <> NEW.screening_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-10]: Thong tin khoa ghe khong khop voi suat chieu va g
he cua ve.';
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 10. Trigger chặn xóa trái phép khóa ghế khi vé vẫn đang VALID
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_ssl_validate_del
BEFORE DELETE ON screening_seat_lock
FOR EACH ROW
BEGIN
    DECLARE v_tk_status VARCHAR(20) DEFAULT NULL;

    SELECT ticket_status INTO v_tk_status
    FROM ticket
    WHERE ticket_id = OLD.ticket_id;

    IF v_tk_status = 'VALID' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-10]: Khong the xoa khoa ghe khi ve van dang o trang th
ai VALID.';
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 11. Trigger kiểm tra khi THÊM MỚI giao dịch thanh toán (payment)
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_payment_validate_ins
BEFORE INSERT ON payment
FOR EACH ROW
BEGIN
    IF NEW.amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-13]: So tien thanh toan phai lon hon 0.';
    END IF;

    IF NEW.payment_status = 'SUCCESS' AND NEW.paid_at IS NULL THEN
        SET NEW.paid_at = CURRENT_TIMESTAMP;
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 12. Trigger kiểm tra khi CẬP NHẬT giao dịch thanh toán (payment)
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_payment_validate_upd
BEFORE UPDATE ON payment
FOR EACH ROW
BEGIN
    IF NEW.amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-13]: So tien thanh toan phai lon hon 0.';
    END IF;

    IF NEW.payment_status = 'SUCCESS' AND NEW.paid_at IS NULL THEN
```

```
        SET NEW.paid_at = CURRENT_TIMESTAMP;
    END IF;
END$$

DELIMITER ;
```

PHỤ LỤC C. MÃ LỆNH 4 VIEW TRA CỨU & ĐỐI SOÁT DOANH THU CHUẨN 3NF (03_views.sql)

```
-- ========================================================================
-- TỆP: 03_views.sql
-- DỰ ÁN: HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM
-- BƯỚC KHỞI TẠO: [Bước 4 - Tạo View] Định nghĩa 04 khung nhìn (Views) trên
--                lược đồ 14 bảng chuẩn 3NF phục vụ Tra cứu Suất chiếu,
--                Báo cáo Doanh thu và Báo cáo Tình hình Bán vé.
-- TƯƠNG THÍCH MYSQL 8.4 LTS:
--   - Sử dụng `CREATE OR REPLACE VIEW` chuẩn của MySQL 8.4, tuân thủ nghiêm ngặt
--     chế độ `ONLY_FULL_GROUP_BY`.
-- ========================================================================

SET NAMES utf8mb4;

-- ------------------------------------------------------------------------
-- 1. Khung nhìn vw_screening_search (Tra cứu suất chiếu & ghế trống trên lược đồ 3NF)
--    Ghế đang bị giữ được đếm trực tiếp từ `screening_seat_lock` (mọi vé VALID
--    của cả đơn CONFIRMED lẫn PENDING đều đang giữ chỗ vật lý trên sơ đồ ghế).
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW vw_screening_search AS
SELECT
    s.screening_id,
    s.movie_id,
    m.movie_code,
    m.title AS movie_title,
    m.duration_minutes,
    m.age_rating,
    s.room_id,
    r.room_name,
    r.room_type,
    r.branch_id,
    b.branch_code,
    b.branch_name,
    b.city,
    s.start_time,
    s.end_time,
    DATE(s.start_time) AS screening_date,
    s.base_price,
    s.status AS screening_status,
    (SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id) AS total_seats,
    (SELECT COUNT(*) FROM screening_seat_lock sl WHERE sl.screening_id = s.screening_id) AS sold_seats,
    (
        (SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id) -
        (SELECT COUNT(*) FROM screening_seat_lock sl WHERE sl.screening_id = s.screening_id)
    ) AS available_seats
FROM screening s
JOIN movie m          ON m.movie_id = s.movie_id
JOIN screening_room r ON r.room_id = s.room_id
JOIN cinema_branch b  ON b.branch_id = r.branch_id;

-- ------------------------------------------------------------------------
-- 2. Khung nhìn vw_verified_booking_revenue (Đơn đặt vé đủ điều kiện doanh thu)
--    Quy tắc BR-14 & BR-15:
--    - Chỉ tính đơn có booking_status = 'CONFIRMED'
--    - Có đúng 1 giao dịch payment_status = 'SUCCESS'
--    - Chỉ cộng các vé có ticket_status = 'VALID'
--    - Đơn PENDING, CANCELLED hoặc thanh toán FAILED / REFUNDED bị loại trừ 100%
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW vw_verified_booking_revenue AS
SELECT
    bk.booking_id,
    bk.booking_code,
    bk.screening_id,
    sc.movie_id,
    mv.title AS movie_title,
    sc.room_id,
    rm.room_name,
    rm.branch_id,
    br.branch_code,
    br.branch_name,
```

```
    bk.customer_id,
    COALESCE(cu.full_name, 'Khách vãng lai') AS customer_name,
    bk.employee_id,
    em.full_name AS employee_name,
    bk.booked_at,
    pm.payment_id,
    pm.payment_method,
    pm.paid_at,
    DATE(pm.paid_at) AS revenue_date,
    COUNT(tk.ticket_id) AS valid_ticket_count,
    COALESCE(SUM(tk.unit_price), 0.00) AS recognized_revenue
FROM booking bk
JOIN screening sc      ON sc.screening_id = bk.screening_id
JOIN movie mv          ON mv.movie_id = sc.movie_id
JOIN screening_room rm ON rm.room_id = sc.room_id
JOIN cinema_branch br  ON br.branch_id = rm.branch_id
LEFT JOIN customer cu  ON cu.customer_id = bk.customer_id
JOIN employee em       ON em.employee_id = bk.employee_id
JOIN payment pm        ON pm.booking_id = bk.booking_id AND pm.payment_status = 'SUCCESS'
JOIN ticket tk         ON tk.booking_id = bk.booking_id AND tk.ticket_status = 'VALID'
WHERE bk.booking_status = 'CONFIRMED'
GROUP BY
    bk.booking_id,
    bk.booking_code,
    bk.screening_id,
    sc.movie_id,
    mv.title,
    sc.room_id,
    rm.room_name,
    rm.branch_id,
    br.branch_code,
    br.branch_name,
    bk.customer_id,
    cu.full_name,
    bk.employee_id,
    em.full_name,
    bk.booked_at,
    pm.payment_id,
    pm.payment_method,
    pm.paid_at;

-- ------------------------------------------------------------------------
-- 3. Khung nhìn vw_branch_daily_revenue (Doanh thu theo ngày thanh toán & chi nhánh)
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW vw_branch_daily_revenue AS
SELECT
    revenue_date,
    branch_id,
    branch_code,
    branch_name,
    COUNT(DISTINCT booking_id) AS paid_booking_count,
    SUM(valid_ticket_count)    AS paid_ticket_count,
    SUM(recognized_revenue)    AS total_paid_revenue
FROM vw_verified_booking_revenue
GROUP BY
    revenue_date,
    branch_id,
    branch_code,
    branch_name;

-- ------------------------------------------------------------------------
-- 4. Khung nhìn vw_screening_occupancy (Báo cáo bán vé & tỷ lệ lấp đầy suất chiếu)
--    - `tickets_sold`, `occupancy_rate_pct`, `sales_revenue`: chỉ tính vé VALID
--      thuộc đơn CONFIRMED đã có thanh toán SUCCESS.
--    - `available_seats`: số ghế trống thực tế trên phòng (trừ mọi ghế đang giữ chỗ
--      trong `screening_seat_lock`, bao gồm cả đơn PENDING đang chờ thanh toán).
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW vw_screening_occupancy AS
SELECT
    s.screening_id,
    DATE(s.start_time) AS screening_date,
    s.start_time,
    s.end_time,
    s.status AS screening_status,
    s.base_price,
```

```
    m.movie_id,
    m.movie_code,
    m.title AS movie_title,
    r.room_id,
    r.room_name,
    r.room_type,
    b.branch_id,
    b.branch_code,
    b.branch_name,
    (SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id) AS room_capacity,
    (
        SELECT COUNT(*)
        FROM ticket tk
        JOIN booking bk ON bk.booking_id = tk.booking_id
        JOIN payment pm ON pm.booking_id = bk.booking_id AND pm.payment_status = 'SUCCESS'
        WHERE bk.screening_id = s.screening_id
          AND tk.ticket_status = 'VALID'
          AND bk.booking_status = 'CONFIRMED'
    ) AS tickets_sold,
    (
        (SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id) -
        (SELECT COUNT(*) FROM screening_seat_lock sl WHERE sl.screening_id = s.screening_id)
    ) AS available_seats,
    ROUND(
        100.0 * (
            SELECT COUNT(*)
            FROM ticket tk
            JOIN booking bk ON bk.booking_id = tk.booking_id
            JOIN payment pm ON pm.booking_id = bk.booking_id AND pm.payment_status = 'SUCCESS'
            WHERE bk.screening_id = s.screening_id
              AND tk.ticket_status = 'VALID'
              AND bk.booking_status = 'CONFIRMED'
        ) / NULLIF((SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id), 0),
        2
    ) AS occupancy_rate_pct,
    COALESCE((
        SELECT SUM(tk.unit_price)
        FROM ticket tk
        JOIN booking bk ON bk.booking_id = tk.booking_id
        JOIN payment pm ON pm.booking_id = bk.booking_id AND pm.payment_status = 'SUCCESS'
        WHERE bk.screening_id = s.screening_id
          AND tk.ticket_status = 'VALID'
          AND bk.booking_status = 'CONFIRMED'
    ), 0.00) AS sales_revenue
FROM screening s
JOIN movie m          ON m.movie_id = s.movie_id
JOIN screening_room r ON r.room_id = s.room_id
JOIN cinema_branch b  ON b.branch_id = r.branch_id;
```

PHỤ LỤC D1. BỘ CÂU LỆNH SQL VÍ DỤ CHẠY TRỰC TIẾP TƯƠNG ỨNG 5 MÀN HÌNH R5 – R7 (functional_sql_verification.sql)

```
-- ============================================================================
-- TỆP: functional_sql_verification.sql
-- MỤC ĐÍCH: Bộ câu lệnh SQL ví dụ chạy được trên `cinema_chain_test` tương ứng
--           1-1 với các câu lệnh tham số hóa (?) trong mã nguồn Java của 5 màn hình
--           chấm điểm R5 (2 Form nhập liệu), R6 (Tìm kiếm), R7 (2 Báo cáo).
-- LƯU Ý AN TOÀN: Mọi ví dụ ghi (INSERT/UPDATE) đều bọc trong giao dịch và kết thúc
--                bằng ROLLBACK để bảo toàn nguyên trạng dữ liệu mẫu kiểm thử.
-- ============================================================================

SET NAMES utf8mb4;

SELECT '=== [R5.1] FORM 1: QUẢN LÝ SUẤT CHIẾU (KIỂM TRA PHÒNG, CHỐNG TRÙNG LỊCH, INSERT & UPDATE) ===' A
S section_banner;

START TRANSACTION;

-- 1a. Khóa dòng phòng chiếu và kiểm tra trạng thái hoạt động [BR-01] (ScreeningRepository.lockRoomAndCh
eckActive)
SELECT room_id, active
FROM screening_room
WHERE room_id = 1
FOR UPDATE;

-- 1b. Tra cứu thời lượng bộ phim để kiểm tra thời lượng suất chiếu tối thiểu [BR-05] (ScreeningReposito
ry.findMovieById)
SELECT movie_id, movie_code, title, duration_minutes, release_date, age_rating
FROM movie
WHERE movie_id = 1;

-- 1c. Kiểm tra xung đột lịch chiếu trong cùng phòng [BR-06] (ScreeningRepository.findOverlappingScreeni
ngs)
--     Ví dụ khung giờ trống hợp lệ ngày 2026-11-19 (09:00 -> 11:15) tại Phòng 1 -> Kỳ vọng 0 dòng xung
đột:
SELECT screening_id, movie_title, room_name, start_time, end_time, screening_status
FROM vw_screening_search
WHERE room_id = 1
  AND screening_status = 'OPEN'
  AND start_time < '2026-11-19 11:15:00'
  AND end_time   > '2026-11-19 09:00:00';

-- 1d. Kiểm tra phát hiện trùng lịch với Suất #1 hiện có ngày 2026-11-04 (13:00 -> 15:31) tại Phòng 1 ->
 Kỳ vọng >= 1 dòng xung đột:
SELECT screening_id, movie_title, room_name, start_time, end_time, screening_status
FROM vw_screening_search
WHERE room_id = 1
  AND screening_status = 'OPEN'
  AND start_time < '2026-11-04 15:00:00'
  AND end_time   > '2026-11-04 13:30:00';

-- 1e. Thêm mới suất chiếu hợp lệ (ScreeningRepository.insertScreening)
INSERT INTO screening (movie_id, room_id, start_time, end_time, base_price, status)
VALUES (1, 1, '2026-11-19 09:00:00', '2026-11-19 11:15:00', 95000.00, 'OPEN');

SET @demo_new_screening_id := LAST_INSERT_ID();

-- 1f. Kiểm tra số vé đã phát hành trước khi cho phép đổi phòng hoặc hủy suất (ScreeningRepository.count
TicketsForScreening)
SELECT COUNT(*) AS existing_tickets_for_new_screening
FROM ticket tk
JOIN booking bk ON bk.booking_id = tk.booking_id
WHERE bk.screening_id = @demo_new_screening_id;

-- 1g. Cập nhật giá cơ sở và khung giờ của suất chiếu vừa tạo (ScreeningRepository.updateScreening)
UPDATE screening
SET movie_id = 1,
    room_id = 1,
    start_time = '2026-11-19 09:30:00',
    end_time = '2026-11-19 11:45:00',
    base_price = 105000.00,
    status = 'OPEN'
```

```
WHERE screening_id = @demo_new_screening_id;

SELECT screening_id, movie_title, room_name, start_time, end_time, base_price, screening_status
FROM vw_screening_search
WHERE screening_id = @demo_new_screening_id;

ROLLBACK;

SELECT '=== [R5.2] FORM 2: ĐẶT VÉ & CHỌN GHẾ (SƠ ĐỒ GHẾ 3NF, FOR UPDATE, TẠO ĐƠN/VÉ/THANH TOÁN, HỦY/HOÀN
 TIỀN) ===' AS section_banner;

-- 2a. Truy vấn sơ đồ ghế trực quan của Suất #1 qua bảng khóa ghế 3NF `screening_seat_lock` (BookingRepo
sitory.findSeatsForScreening)
SELECT
    st.seat_id,
    st.room_id,
    st.seat_row,
    st.seat_number,
    st.seat_type_code,
    tp.type_name,
    tp.surcharge,
    (sc.base_price + tp.surcharge) AS final_unit_price,
    CASE WHEN sl.ticket_id IS NOT NULL THEN 1 ELSE 0 END AS is_occupied
FROM screening sc
JOIN seat st      ON st.room_id = sc.room_id
JOIN seat_type tp ON tp.seat_type_code = st.seat_type_code
LEFT JOIN screening_seat_lock sl
       ON sl.screening_id = sc.screening_id
      AND sl.seat_id = st.seat_id
WHERE sc.screening_id = 1
ORDER BY st.seat_row ASC, st.seat_number ASC
LIMIT 12;

-- 2b. Giao dịch nguyên tử đặt vé + thanh toán + hủy hoàn tiền trên suất chiếu thử nghiệm (mô phỏng đầy
đủ BookingService)
START TRANSACTION;

INSERT INTO screening (movie_id, room_id, start_time, end_time, base_price, status)
VALUES (1, 1, '2026-11-20 18:00:00', '2026-11-20 20:15:00', 100000.00, 'OPEN');
SET @bk_demo_scr_id := LAST_INSERT_ID();

-- Khóa dòng suất chiếu chống tranh chấp đồng thời (BookingRepository.lockScreeningForUpdate)
SELECT screening_id, room_id, base_price, status
FROM screening
WHERE screening_id = @bk_demo_scr_id
FOR UPDATE;

-- Kiểm tra ghế #1 (A01 - STD) và ghế #45 (F05 - VIP) trước khi chèn vé (BookingRepository.inspectSeatFo
rBooking)
SELECT
    st.seat_id,
    st.room_id,
    CONCAT(st.seat_row, LPAD(st.seat_number, 2, '0')) AS seat_label,
    tp.surcharge,
    CASE WHEN sl.ticket_id IS NOT NULL THEN 1 ELSE 0 END AS already_occupied
FROM seat st
JOIN seat_type tp ON tp.seat_type_code = st.seat_type_code
LEFT JOIN screening_seat_lock sl
       ON sl.screening_id = @bk_demo_scr_id
      AND sl.seat_id = st.seat_id
WHERE st.seat_id IN (1, 45);

-- Tạo đơn đặt vé (BookingRepository.insertBooking)
INSERT INTO booking (booking_code, screening_id, customer_id, employee_id, booked_at, booking_status)
VALUES ('BK-SQL-DEMO-001', @bk_demo_scr_id, 1, 3, '2026-11-07 10:00:00', 'CONFIRMED');
SET @bk_demo_id := LAST_INSERT_ID();

-- Tạo các dòng vé chuẩn 3NF — KHÔNG lưu cột dư thừa screening_id trên ticket (BookingRepository.insertT
icket)
-- Trigger `trg_ticket_lock_after_ins` tự động ghi khóa ghế vào `screening_seat_lock`
INSERT INTO ticket (booking_id, seat_id, unit_price, ticket_status)
VALUES
    (@bk_demo_id, 1,  100000.00, 'VALID'),
    (@bk_demo_id, 45, 125000.00, 'VALID');
```

```
-- Ghi nhận giao dịch thanh toán thành công (BookingRepository.insertPayment)
INSERT INTO payment (booking_id, amount, payment_method, payment_status, paid_at, transaction_ref)
VALUES (@bk_demo_id, 225000.00, 'CARD', 'SUCCESS', '2026-11-07 10:00:05', 'TXN-SQL-DEMO-001');

-- Kiểm tra trạng thái khóa ghế tự động trong `screening_seat_lock` (Kỳ vọng: 2 dòng khóa cho ghế 1 và 4
5)
SELECT screening_id, seat_id, ticket_id
FROM screening_seat_lock
WHERE screening_id = @bk_demo_scr_id
ORDER BY seat_id ASC;

-- Minh họa Hủy đơn & Hoàn tiền giải phóng ghế để bán lại (BookingRepository.cancelBookingRecords)
UPDATE booking SET booking_status = 'CANCELLED' WHERE booking_id = @bk_demo_id;
UPDATE ticket  SET ticket_status = 'CANCELLED', cancelled_at = CURRENT_TIMESTAMP WHERE booking_id = @bk_
demo_id;
UPDATE payment SET payment_status = 'REFUNDED' WHERE booking_id = @bk_demo_id AND payment_status = 'SUCC
ESS';
UPDATE payment SET payment_status = 'FAILED'   WHERE booking_id = @bk_demo_id AND payment_status = 'PEND
ING';

-- Kiểm tra sau khi hủy: khóa ghế được giải phóng về 0, vé CANCELLED và giao dịch REFUNDED vẫn lưu lịch
sử
SELECT
    (SELECT COUNT(*) FROM screening_seat_lock WHERE screening_id = @bk_demo_scr_id) AS active_locks_afte
r_cancel,
    (SELECT COUNT(*) FROM ticket WHERE booking_id = @bk_demo_id AND ticket_status = 'CANCELLED') AS canc
elled_history_tickets,
    (SELECT payment_status FROM payment WHERE booking_id = @bk_demo_id) AS refunded_payment_status;

-- Trong ứng dụng thật, giao dịch hợp lệ kết thúc bằng COMMIT; ở kịch bản kiểm chứng này dùng ROLLBACK đ
ể giữ sạch DB test
ROLLBACK;

SELECT '=== [R6] MÀN HÌNH 3: TRA CỨU SUẤT CHIẾU & KIỂM TRA GHẾ TRỐNG (TỔNG HỢP KPI + PHÂN TRANG) ===' AS
 section_banner;

-- 3a. Tổng hợp KPI toàn tập kết quả khớp bộ lọc (ScreeningSearchRepository.summarizeSearch)
SELECT
    COUNT(*) AS total_count,
    COALESCE(SUM(available_seats), 0) AS sum_available,
    COALESCE(SUM(sold_seats), 0) AS sum_sold
FROM vw_screening_search
WHERE 1 = 1
  AND branch_id = 1
  AND screening_date = '2026-11-04'
  AND TIME(start_time) >= '08:00:00'
  AND TIME(start_time) <= '23:00:00'
  AND screening_status = 'OPEN'
  AND available_seats > 0;

-- 3b. Truy vấn trang 1 danh sách suất chiếu khớp điều kiện (ScreeningSearchRepository.searchScreeningsP
age)
SELECT
    screening_id, movie_code, movie_title, age_rating, duration_minutes,
    branch_name, room_name, room_type, start_time, end_time,
    base_price, total_seats, sold_seats, available_seats, screening_status
FROM vw_screening_search
WHERE 1 = 1
  AND branch_id = 1
  AND screening_date = '2026-11-04'
  AND TIME(start_time) >= '08:00:00'
  AND TIME(start_time) <= '23:00:00'
  AND screening_status = 'OPEN'
  AND available_seats > 0
ORDER BY start_time ASC, branch_id ASC, room_id ASC, screening_id ASC
LIMIT 50 OFFSET 0;

SELECT '=== [R7.1] MÀN HÌNH 4: BÁO CÁO DOANH THU CHUỖI RẠP (KẾ TOÁN LẬP -> GỬI GIÁM ĐỐC KINH DOANH) ===='
 AS section_banner;

-- 4a. Bảng I — Tổng hợp doanh thu thực thu theo Chi nhánh kỳ 01/11/2026 - 15/11/2026 (RevenueReportRepo
sitory.findBranchSummaries)
SELECT
    br.branch_id,
    br.branch_code,
```

```
    br.branch_name,
    br.city,
    COUNT(DISTINCT vr.booking_id)              AS paid_bookings,
    COALESCE(SUM(vr.valid_ticket_count), 0)    AS valid_tickets,
    COALESCE(SUM(vr.recognized_revenue), 0.00) AS paid_revenue
FROM cinema_branch br
LEFT JOIN vw_verified_booking_revenue vr
    ON vr.branch_id = br.branch_id
   AND vr.revenue_date BETWEEN '2026-11-01' AND '2026-11-15'
WHERE br.active = 1
GROUP BY br.branch_id, br.branch_code, br.branch_name, br.city
ORDER BY paid_revenue DESC, br.branch_id ASC;

-- 4b. Bảng đối soát minh bạch cách xử lý các trạng thái thanh toán & đơn vé (SUCCESS vs PENDING vs CANC
ELLED/REFUNDED vs FAILED)
SELECT
    bk.booking_status,
    pm.payment_status,
    COUNT(DISTINCT bk.booking_id) AS booking_count,
    COUNT(DISTINCT tk.ticket_id)  AS ticket_rows,
    CASE
        WHEN bk.booking_status = 'CONFIRMED' AND pm.payment_status = 'SUCCESS'
            THEN 'ĐƯỢC GHI NHẬN DOANH THU (vé VALID, không nhân bản khi có retry FAILED)'
        WHEN bk.booking_status = 'PENDING' AND pm.payment_status = 'PENDING'
            THEN 'LOẠI TRỪ DOANH THU THỰC THU (chưa thu tiền; vẫn giữ khóa ghế tạm thời)'
        WHEN bk.booking_status = 'CANCELLED' AND pm.payment_status = 'REFUNDED'
            THEN 'LOẠI TRỪ DOANH THU THỰC THU (đã hủy vé & hoàn tiền, giải phóng ghế)'
        ELSE 'LOẠI TRỪ DOANH THU THỰC THU (giao dịch lỗi / đơn hủy chưa thu tiền)'
    END AS accounting_treatment
FROM booking bk
JOIN payment pm ON pm.booking_id = bk.booking_id
LEFT JOIN ticket tk ON tk.booking_id = bk.booking_id
GROUP BY bk.booking_status, pm.payment_status
ORDER BY bk.booking_status, pm.payment_status;

SELECT '=== [R7.2] MÀN HÌNH 5: BÁO CÁO TÌNH HÌNH BÁN VÉ & TỶ LỆ LẤP ĐẦY (QUẢN LÝ CHI NHÁNH -> BAN ĐIỀU H
ÀNH/QL KINH DOANH) ===' AS section_banner;

-- 5a. Chi tiết bán vé, sức chứa, ghế trống & tỷ lệ lấp đầy theo Suất chiếu tại Chi nhánh #1 ngày 04/11/
2026 (TicketSalesReportRepository.findScreeningSales)
SELECT
    screening_id,
    screening_date,
    start_time,
    end_time,
    branch_name,
    room_name,
    room_type,
    movie_title,
    room_capacity,
    tickets_sold,
    available_seats,
    occupancy_rate_pct,
    sales_revenue,
    screening_status
FROM vw_screening_occupancy
WHERE screening_date BETWEEN '2026-11-04' AND '2026-11-04'
  AND branch_id = 1
ORDER BY start_time ASC, screening_id ASC;
```

PHỤ LỤC D2. CÁC TRUY VẤN NGHIỆP VỤ, GIAO DỊCH & BÁO CÁO MỞ RỘNG (business_queries.sql)

```
-- ============================================================================
-- TỆP: business_queries.sql
-- MỤC ĐÍCH: Bộ câu lệnh SQL mẫu chạy được trực tiếp trên `cinema_chain_test`
--           tương ứng 100% với 5 chức năng nghiệp vụ R5, R6, R7 (Phụ lục R8).
-- ============================================================================

SET NAMES utf8mb4;

-- ----------------------------------------------------------------------------
-- Q1. Kiểm tra xung đột lịch chiếu trong cùng một phòng chiếu (Form 1 - R5)
-- ----------------------------------------------------------------------------
SELECT
    s.screening_id,
    m.title AS movie_title,
    s.start_time,
    s.end_time,
    s.status
FROM screening s
JOIN movie m ON m.movie_id = s.movie_id
WHERE s.room_id = 1
  AND s.status = 'OPEN'
  AND s.start_time < '2026-11-04 11:30:00'
  AND s.end_time > '2026-11-04 09:00:00';

-- ----------------------------------------------------------------------------
-- Q2. Truy vấn sơ đồ ghế và trạng thái ghế theo suất chiếu qua bảng khóa 3NF
--     `screening_seat_lock` (Form 2 - R5)
-- ----------------------------------------------------------------------------
SELECT
    st.seat_id,
    st.seat_row,
    st.seat_number,
    CONCAT(st.seat_row, LPAD(st.seat_number, 2, '0')) AS seat_label,
    st.seat_type_code,
    tp.type_name,
    tp.surcharge,
    (sc.base_price + tp.surcharge) AS final_ticket_price,
    CASE
        WHEN sl.ticket_id IS NOT NULL THEN 'OCCUPIED'
        ELSE 'AVAILABLE'
    END AS seat_availability
FROM screening sc
JOIN seat st                   ON st.room_id = sc.room_id
JOIN seat_type tp              ON tp.seat_type_code = st.seat_type_code
LEFT JOIN screening_seat_lock sl
       ON sl.screening_id = sc.screening_id
      AND sl.seat_id = st.seat_id
WHERE sc.screening_id = 1
ORDER BY st.seat_row, st.seat_number
LIMIT 20;

-- ----------------------------------------------------------------------------
-- Q3. Đếm tổng số kết quả và phân trang tra cứu suất chiếu (Màn hình 3 - R6)
-- ----------------------------------------------------------------------------
SELECT COUNT(*) AS total_matching_screenings

FROM vw_screening_search v
WHERE v.screening_status = 'OPEN'
  AND v.screening_date = '2026-11-04';

SELECT
    v.screening_id,
    v.movie_code,
    v.movie_title,
    v.duration_minutes,
    v.age_rating,
    v.branch_name,
    v.room_name,
    v.room_type,
    v.start_time,
```

```
    v.end_time,
    v.base_price,
    v.total_seats,
    v.sold_seats,
    v.available_seats,
    v.screening_status
FROM vw_screening_search v
WHERE v.screening_status = 'OPEN'
  AND v.screening_date = '2026-11-04'
ORDER BY v.start_time ASC, v.branch_name ASC, v.room_name ASC
LIMIT 15 OFFSET 0;

-- ------------------------------------------------------------------------
-- Q4. Báo cáo Doanh thu theo Chi nhánh và Khoảng thời gian thanh toán (Báo cáo 1 - R7)
--     Người lập: Kế toán | Người nhận: Giám đốc Kinh doanh
--     Chỉ ghi nhận đơn CONFIRMED có thanh toán SUCCESS và vé VALID;
--     loại trừ hoàn toàn đơn PENDING, CANCELLED, thanh toán FAILED và REFUNDED.
-- ------------------------------------------------------------------------
SELECT
    br.branch_id,
    br.branch_code,
    br.branch_name,
    br.city,
    COUNT(DISTINCT vr.booking_id)              AS total_paid_bookings,
    COALESCE(SUM(vr.valid_ticket_count), 0)    AS total_valid_tickets,
    COALESCE(SUM(vr.recognized_revenue), 0.00) AS total_paid_revenue
FROM cinema_branch br
LEFT JOIN vw_verified_booking_revenue vr
    ON vr.branch_id = br.branch_id
   AND vr.revenue_date BETWEEN '2026-11-01' AND '2026-11-15'
GROUP BY br.branch_id, br.branch_code, br.branch_name, br.city
ORDER BY total_paid_revenue DESC, br.branch_id ASC;

-- ------------------------------------------------------------------------
-- Q5. Chi tiết doanh thu theo ngày thanh toán và chi nhánh (Báo cáo 1 - R7)
-- ------------------------------------------------------------------------
SELECT
    revenue_date,
    branch_code,
    branch_name,
    paid_booking_count,
    paid_ticket_count,
    total_paid_revenue
FROM vw_branch_daily_revenue
WHERE revenue_date BETWEEN '2026-11-01' AND '2026-11-15'
ORDER BY revenue_date ASC, branch_code ASC;

-- ------------------------------------------------------------------------
-- Q6. Báo cáo Tình hình Bán vé & Tỷ lệ Lấp đầy theo Suất chiếu (Báo cáo 2 - R7)
--     Người lập: Quản lý Chi nhánh | Người nhận: Ban Điều hành / Quản lý Kinh doanh
-- ------------------------------------------------------------------------
SELECT
    screening_id,
    screening_date,
    TIME(start_time) AS start_clock,
    TIME(end_time)   AS end_clock,
    branch_name,
    room_name,
    room_type,
    movie_title,
    room_capacity,
    tickets_sold,
    available_seats,
    occupancy_rate_pct,
    sales_revenue
FROM vw_screening_occupancy
WHERE screening_date BETWEEN '2026-11-04' AND '2026-11-10'
ORDER BY screening_date ASC, start_time ASC, branch_id ASC
LIMIT 20;

-- ------------------------------------------------------------------------
-- Q7. Đối soát toàn vẹn Doanh thu Độc lập (Kiểm chứng chống nhân bản doanh thu):
--     Chứng minh tổng doanh thu từ bảng ticket (thuộc đơn CONFIRMED có payment SUCCESS)
--     khớp tuyệt đối với tổng doanh thu trên View báo cáo.
```

```sql
-- ----------------------------------------------------------------------------
SELECT
    (SELECT SUM(tk.unit_price)
     FROM ticket tk
     JOIN booking bk ON bk.booking_id = tk.booking_id
     WHERE tk.ticket_status = 'VALID'
       AND bk.booking_status = 'CONFIRMED'
       AND EXISTS (
           SELECT 1 FROM payment pm
           WHERE pm.booking_id = bk.booking_id
             AND pm.payment_status = 'SUCCESS'
       )
    ) AS independent_ticket_revenue_sum,
    (SELECT SUM(total_paid_revenue)
     FROM vw_branch_daily_revenue
    ) AS view_report_revenue_sum;
```